Università di Pisa



Corso di Laurea Magistrale
in Matematica

Applications of Reinforcement Learning to Conjectures over Large Discrete Spaces

Tesi di Laurea Magistrale
in Matematica

Candidato:
Davide Orecchioni

Relatore:
Francesco Morandin

Anno Accademico 2026 - 2027

Contents

Introduction **2**

1 Theory of Markov Decision Processes **4**
1.1 Markov Decision Processes . . . 4
1.2 Return, Episodes, and Continuing Tasks . . . 7
1.3 Policies, Value and Optimality . . . 9
1.4 Optimal Policies for Discounted MDPs . . . 12
1.5 Optimal Policies for Finite Time Horizon MDPs . . . 18

2 RL Models for Maximization **23**
2.1 Modeling a MDP . . . 23
2.2 Modeling discrete maximization . . . 24
2.2.1 Structure of the MDP . . . 25
2.2.2 Convergence to a maximum . . . 26
2.2.3 Additional considerations . . . 27
2.3 Maximization over binary strings . . . 28
2.3.1 Continuous, Full reward . . . 29
2.3.2 Continuous, Incremental reward . . . 34
2.3.3 FTH, Full reward . . . 38
2.3.4 FTH, Incremental reward . . . 39
2.3.5 FTH, Episodic reward . . . 40
2.3.6 Continuous, Episodic reward . . . 44

3 DP, Tabular Methods and Approximation **48**
3.1 Dynamic Programming . . . 48
3.1.1 Evaluation and Improvement . . . 49
3.1.2 Generalized Policy Iteration . . . 52
3.2 Tabular Methods . . . 53
3.3 Approximation Methods . . . 56
3.3.1 Linear Methods . . . 58
3.3.2 Convergence of Linear Sarsa . . . 59

4 Computational results **63**
4.1 Theoretical setting . . . 63
4.2 Experimental setup . . . 65
4.3 Results . . . 68

Bibliography **76**

Introduction

Reinforcement Learning (RL) is one of the three basic machine learning paradigms, together with supervised and unsupervised learning. In the case of supervised and unsupervised learning, the goal is to train a model on a certain amount of data, to uncover and exploit hidden patterns in the data. Reinforcement learning is quite different in this regard, as it does not rely on data to solve a given task; instead, it aims to train an *agent* that can autonomously choose how to operate in order to solve the original task. This agent is free to move within a certain *environment*, which is a dynamic space in which our task takes place. As the name suggests, the agent is allowed to choose between a set of predetermined actions, and its goal is to choose the best possible action. Each action influences the environment, which reacts by transitioning to a new configuration. The agent can detect and quantify changes in the environment, and use that information to determine which action is the best at any given time. To guide the decision process, every time the environment changes as a result of the agent's action they receives a numeric, real-valued *reward*: an action will be good if it results in a high reward, and suboptimal or bad otherwise. The main goal of reinforcement learning is to teach our agent the best way to interact with the environment in order to maximize the total reward it receives during the entirety of its training. This goes further than maximizing the immediate reward at each step: some actions may yield less reward in the short term, but make the environment transition to new configurations where even greater rewards can be achieved.

Among all learning paradigms, RL is the one closest to how human beings learn. We can think of our agent as a human playing a game and trying to discern which moves are the best, not only to maximize the game score at each time, but to learn a strategy that allows it to ultimately win. Of course, we do not know which move is the best from the beginning; we have to play the game multiple times, taking note of which actions gave us a higher score depending on the state of the game. This act of repeated play and accumulation of information is referred to as *experience*, and it replaces the role of data in supervised and unsupervised learning. The core appeal of Reinforcement Learning is exactly this ability to learn from past errors and successes, solving a task by experience alone.

In this work, we discuss the applications of reinforcement learning to the problem of maximization over large discrete spaces, in the context of solving conjectures. This approach was pioneered by Adam Zsolt Wagner in his 2021 paper [11], who introduced the framework we will now describe.

Definition. A *conjecture in normal form* over the elements of a space $\mathbb{X}$ is a statement of the form $f(x) \leq 0 \ \forall \ x \in \mathbb{X}$, where f is a real-valued function $f : \mathbb{X} \to \mathbb{R}$.

A conjecture in normal form can be proven or disproved by finding a maximum point x_* and checking whether or not $f(x_*) \leq 0$. However, finding x_* is not always trivial; namely, when

$\mathbb{X}$ is a large discrete space we cannot maximize f analytically, even if f is differentiable, and brute-force methods become infeasible. A very relevant example of this phenomenon is that of conjectures over graphs. The space of graphs with a fixed amount of nodes is a discrete space that can grow extremely large as the number of node grows, and many conjectures on graphs can be written in normal form; for example, f may be a function of the graph's adjacency matrix. To disprove such conjectures one wants to find a counterexample, which is a point $\bar{x}$ (not necessarily the maximum) such that $f(\bar{x}) > 0$. To tackle this problem Wagner devised a RL-based algorithm, the deep cross-entropy method, and used it to successfully find counterexamples to several open conjectures.

One key advantage of using reinforcement learning is that, even if the method does not successfully find a counterexample, the experience it accumulates and the evolution of its best attempts during training can be used to extrapolate the characteristics that such a point should have. This can give precious insight on the original problem, and potentially allows us to build on the uncovered pattern and find a counterexample. This novel approach has since been implemented and adapted by many with great success, for conjectures in fields such as graph theory and extremal combinatorics, and sparked a newfound interest in RL for problems of this form. One of the most relevant additions to Wagner's framework is the use of modern, ad-hoc deep learning techniques; for example, in [2] Angileri et al. discuss the use of Graph Neural Network architectures for graph-based conjectures, based on graph-building games they developed in [1].

Because of the experimental nature of this approach and of its ultimate goal of finding a counterexample, many works following in Wagner's tracks focus on the numeric results and on the implementation of deep learning architectures, rather than the RL component. In our work we instead take a first-principles approach to the reinforcement learning element, with the aim of understanding which properties a RL model should have to successfully accomplish the task of discrete maximization. This work will be organized as follows:

- In Chapter 1 we formalize the core concepts of environment, agent and reward using the framework of Markov Decision Processes (MDPs). The agent's decision are recorded by a certain *policy*; a policy is called optimal if it maximizes the long-term reward, and solving a MDP (and with it the original task) corresponds to determining its optimal policy. The main result of Chapter 1 is Theorem 1.1, which characterizes optimal policies and shows that they always exist.

- In Chapter 2 we use the theoretical background of Chapter 1 to characterize which MDPs can successfully be used for the task of discrete maximization. To do so we build several example MDPs on the space of binary strings, and for each one we show whether or not the optimal policy successfully leads us to the maximum point.

- In Chapter 3 we touch on classical RL methods for optimization: Dynamic Programming and Tabular methods. We then introduce Approximated methods, focusing on linear approximation, and study the convergence of the Sarsa algorithm when paired with linear approximation.

- Finally, in Chapter 4 we perform numerical experiments on the MDPs we built in Chapter 2 using the Sarsa algorithm with linear approximation, aiming to quantify the effect of our modeling choices on the performance of the models.

Chapter 1

Theory of Markov Decision Processes

The main theoretical framework of Reinforcement Learning learning is that of Markov Decision Processes. MDPs are structures used to formalize the relationships between agent, environment, and reward, and allow us to make formal claims on the properties of RL-based methods. In the following chapter we will rigorously define MDPs and progressively lay down the theoretical background we will need to formulate and prove results on discrete maximization. The order of topics is loosely based on chapter 3 of [8], but we will go in much deeper detail on the necessary proofs, which Sutton and Barto often omit favoring intuition over rigor. The main result of this chapter is Theorem 1.1, which characterizes optimal policies. Computing optimal policies is the main objective of RL methods, and this theorem guarantees that such methods are well founded. The second main result is a characterization of optimal policies for the special class of Finite Time Horizon MDPs, for which Theorem 1.1 does not always hold. This is especially relevant to prove that the MDPs used for experimentation, in our work and others, are well-founded.

1.1 Markov Decision Processes

Markov Decision Processes are a simple yet powerful framework to represent how a reinforcement learning model operates. The protagonist of a RL model is our agent, an intelligent entity that is tasked with exploring a dynamic environment which changes by reacting to the agent's actions. The agent has access to information on the configuration of the environment at any given time, and this information changes as the environment changes. Each possible unique set of information that the agent can access is called a *state*. The agent is able to distinguish between different states; the main role of the agent in any given state is to interact with the environment by making it transition to a new state, which it does by choosing an *action*. Each state comes with its own predetermined set of actions that the agent can choose, which may not be the same for all states. The main factor based on which the agent chooses an action is the *reward*, a numeric value that is awarded to the agent when the environment transitions to a new state as a result of the chosen action. States, actions, and rewards are the basic building blocks of every MDP; we will now formalize these concepts. For each time $t = 0, 1, 2, 3 \dots$ we have:

- A **state** S_t, which represents the information our agent can access from the environment.
- An **action** A_t which our agent chooses based on the state S_t. As a result of the agent's action the environment will move to the next state, S_{t+1}.

- A **reward** R_{t+1} which the system grants after choosing action A_t in state S_t.

This cycle of interactions between the agent and the environment, which is called *Agent-Environment interface*, can be summarized as follows:

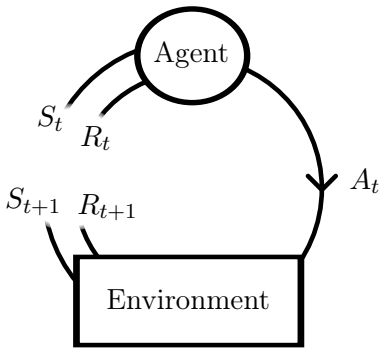


Figure 1: A visualization of the Agent-Environment interface scheme.

The system starts at time $t = 0$ in some initial state S_0. The agent chooses an action A_0 and the environment reacts to this choice, transitioning to another state S_1 and transmitting the reward signal R_1. The cycle then starts over, creating a sequence of states, actions, and rewards of the form $(S_0, A_0, R_1, S_1, A_1, R_2, S_2 \ldots)$, which represents the experience generated by our agent as it interacts with the environment.

Definition 1.1. A **Discrete Markov Decision Process** is given by $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p)$ where

- $\mathcal{S}$ is a countable set of **states**, the environment configurations in which our agent may find itself. For every time t, S_t is a random variable with values in $\mathcal{S}$.
- $\mathcal{A}$ is a countable set of all the possible **actions** our agent can choose, with $\mathcal{A}(s)$ denoting the set of actions available to the agent in state s. For every time t, A_t is a random variable with values in $\mathcal{A}$ that depends on S_t; if conditioned on the event $S_t = s$, A_t assumes values in $\mathcal{A}(s)$.
- $\mathcal{R}$ is a countable subset of $\mathbb{R}$ which contains all the possible **rewards** for picking an action $a \in \mathcal{A}(s)$ in some state $s \in \mathcal{S}$. For every time t, R_{t+1} is a real valued random variable which depends on (S_t, A_t), and $\mathcal{R}$ is its range.
- The function $p : (\mathcal{S} \times \mathcal{R}) \times (\mathcal{S} \times \mathcal{A}) \to [0, 1]$ represents the **dynamics** of the system, which are given by

$$p(s', r|s, a) = \mathbb{P}\left(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a\right) \tag{1.1}$$

  for $r \in \mathcal{R}$, $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. This means that when an agent is in state s and chooses action a, it has probability $p(s', r|s, a)$ of getting reward r and moving to state s'. The dynamics codify the way the environment reacts to the agent's actions.

The presence of s on the left side of the dynamics formula corresponds to the fact that, for any fixed action-state couple $S_t = s, A_t = a$, the next state $S_{t+1} = s'$ is not uniquely determined by s and a but is instead a random choice dictated by the system's dynamics,

that depends on (s, a) through (1.1). The same is true for the reward r, which is determined jointly with the next state; this is also the reason why the reward at time t is denoted as R_{t+1} using a shifted index, as it is determined at the same time as the next state S_{t+1}. Because of this there is no natural notion of "path" or "next step" in MDPs. This possibility of non-deterministic evolution of the system greatly expands the expressiveness of the class of MDPs. Still, when dealing with real word problems it is often very useful to instead focus on *deterministic* MDPs, where every action leads to transitioning to a predetermined state with probability 1.

Definition 1.2. A MDP is called *deterministic* if for each state $s \in \mathcal{S}$, for any action $a \in \mathcal{A}(s)$ there exists a unique couple of state $\bar{s} \in \mathcal{S}$ and reward $\bar{r} \in \mathcal{R}$ such that choosing a in state s causes the environment to transition to $\bar{s}$ with reward $\bar{r}$ with probability 1:

$$\forall\ s \in \mathcal{S}, a \in \mathcal{A}(s)\ \exists\ \bar{s} \in \mathcal{S}, \bar{r} \in \mathcal{R} : p(\bar{s}, \bar{r}|s, a) = 1.$$

Expression (1.1) summarizes the three main properties that the dynamics of a MDP must satisfy:

1. For each choice of $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, $p(\cdot, \cdot|s, a)$ is a discrete probability distribution over $\mathcal{S} \times \mathcal{R}$:
$$\sum_{s' \in S} \sum_{r \in \mathcal{R}} p(s', r|s, a) = 1\ \forall\ s \in \mathcal{S}, a \in \mathcal{A}(s).$$
This is emphasized by the use of the | symbol to separate the first two and the second two inputs, mirroring the standard notation for conditional probability.

2. p is **stationary**, meaning that (1.1) does not depend on the choice of time t.

3. p satisfies the **Markov Property**: the system's dynamics at time t are completely determined by the state at time $t-1$ and the chosen action at that time, with no dependence on decisions made further in the past.

When $\mathcal{S}, \mathcal{A}$ and $\mathcal{R}$ are all countable, the random variables S_t, A_t and R_{t+1} admit a discrete probability distribution. Stationarity means that these distributions may not depend on the time t but only on the past history of the MDP (i.e. the temporal sequence of states, actions and rewards). Lastly, the Markov property ensures that this dependence on past history can only go as far as one step back in time. These three properties ensure that expression (1.1) is well defined.

Remark 1.1. The Markov property can be interpreted as a property of the states rather than a property of the dynamics. It corresponds to the assumption that, at each time t, the state S_{t-1} contains exhaustive information on all the possible future evolutions of the system from t onwards. Because of this, S_t does not need to take into account the whole history of states, as the information contained in S_{t-1} is sufficient. Practically speaking, since we described states as the set of all possible information we can have on the environment at any given time, we are saying that for every time t the state S_t gives us absolute knowledge of the environment at time t.

Remark 1.2. Because of stationarity, the reward $R_{t+1}(s, a)$ cannot depend on the time t but only on the state action couple (s, a). Because of this the immediate reward is often denoted as a single discrete random variable $r(s, a)$, whose distribution is determined by the dynamics of the MDP. When the MDP is deterministic the reward is a deterministic function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, which is usually referred to as *reward function*.

The concept of MDP can be generalized to non-discrete spaces (for example when $\mathcal{S} = \mathbb{R}^n$, $\mathcal{R} = \mathbb{R}$) by using probability kernels in place of a single distribution p. We will not delve deeper into this, as our aim is specifically to use RL for exploring large finite spaces; we will instead focus on the class of finite MDPs.

Definition 1.3. A MDP $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p)$ is called **finite** if the sets $\mathcal{S}, \mathcal{A}$ and $\mathcal{R}$ are all finite.

1.2 Return, Episodes, and Continuing Tasks

The objective of our agent is to maximize reward in the long run: at each time t we do not only care about maximizing the next reward R_{t+1}, but also future rewards. To formalize this concept we introduce the notion of **return** G_t, which is generally defined as a function of all future rewards $(R_{t+1}, R_{t+2}, R_{t+3} \ldots)$ starting from time t. The agent can then choose the next action A_t by maximizing G_t instead of R_{t+1}, avoiding overly greedy behavior that might lead to less total reward in the long run. We will now introduce three categories of MDP, each with their own definition of return, and then introduce a common notation for it.

Finite Time Horizon: A very simple scenario is one where our agent is free to move for a fixed amount of time, say T, and is then stopped. At each time $t = 0 \ldots T-1$ they receive some reward R_{t+1}, and the return at time t is simply the sum of all rewards starting from t:

$$G_t = R_{t+1} + R_{t+3} + R_{t+3} + \ldots + R_T. \tag{1.2}$$

A task of this form is said to have a *Finite Time Horizon* (FTH), and T is its *termination time*. An alternative way to say this is that the finite time horizon of the MDP is T. A finite time horizon MDP with termination time T will be denoted as $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p, T)$. FTH tasks are among the most straightforward to model and solve, and because of this they are often used in numerical experimentation.

Episodic: Another option is for our task to be divided into multiple *episodes*, giving rise to an *episodic* task. Each episode is a smaller independent task where our agent moves around until it ends up in some special states, which are called *terminal*. After reaching a terminal state the current episode ends and a new episode may start, usually from a fixed starting state or a uniformly random sample of starting states. During each episode the agent is guaranteed to reach a terminal state in a finite amount of moves, meaning each episode is finite, but its duration is not known in advance. Each episode is not affected by previous episodes and cannot influence future episodes.

Episodic tasks differ from finite time horizon tasks in that, while the agent is guaranteed to reach a terminal state in a finite amount of time (i.e.: the task cannot go on forever), this amount is not known ahead of time. Because of this episodic tasks are sometimes referred to as *Indefinite Horizon Tasks*. An example of episodic task is the game of tic tac toe: each match starts from the same blank state, is guaranteed to end between five and nine moves, each state with three Xs or Os in a row is terminal and the result (win/loss/draw) has no effect on future matches. Notice that we know that each episode will end in a finite amount of moves, but do not know in advance what this amount is, meaning it is not a FTH task. If instead we wait for the board to be full before checking if a player won, the termination time is always 9, which makes each episode of tic tac toe a FTH task.

For episodic tasks, return takes on the form

$$G_t = R_{t+1} + R_{t+3} + R_{t+3} + \ldots + R_{\mathcal{T}} \tag{1.3}$$

where $\mathcal{T}$ is the *termination time*, the first time after t when a terminal state is entered. $\mathcal{T}$ is not a constant but instead a discrete random variable that varies from episode to episode. While this formula is almost identical to (1.2), in a finite time horizon task the return tallies all rewards until the task is over, whereas in an episodic task we only consider rewards within a single episode. In terms of return, a FTH task can be interpreted as a single episode of an episodic task with constant termination time $\mathcal{T} = T$; with this understanding, expression (1.3) is a generalization of (1.2), and in this sense we can think of episodic tasks as a generalization of FTH tasks. Because of this, throughout our work we will often use the term episodic to indicate both finite and indefinite horizon tasks.

Episodic tasks often excel at characterizing real life situations because of the presence of terminal states, which adds a layer of semantic depth to the set of states. For example, most common card and board games can be codified as episodic tasks, where each game is an episode and terminal states correspond to a player being able to win or force a draw. However, they are often harder to fully characterize compared to FTH and continuing tasks. This is partly because of the nature of the episodes, which introduce the additional random elements of termination time and reset to an initial state. Luckily, when dealing with them on a theoretical basis we usually need to focus on a single episode at a time, allowing us to lean on the simpler theory of finite time horizon tasks.

Continuing: Defining the return as the sum of immediate rewards after t is not possible for tasks that do not have well defined termination conditions, because the sum may diverge as $\mathcal{T} \to \infty$. This happens for tasks that can (at least theoretically) go on forever, which are called *continuing* tasks. An example of continuing task is a robot learning how to walk: we want the robot to be able to keep walking indefinitely and to get up by itself if it falls until it learns to walk without falling, making it so that there is no clear termination condition. Another example is a thermostat system managing a room's temperature.

To solve the problem of the sum diverging we introduce *discounting*: we pick a parameter $\gamma \in [0, 1]$, which we call *discount rate*, and consider the *discounted return*

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{1.4}$$

Every reward from time t onwards is taken into account and weighed by a factor γ^k, which makes each term exponentially less relevant the further back in time it is. The main advantage of this is that, if we assume that $\mathcal{R}$ is bounded, for every $\gamma < 1$ we can ensure that the series converges and the return is well defined. This is always true for finite MDPs, where $\mathcal{R}$ is finite and thus bounded in $\mathbb{R}$. Notice that for $\gamma = 0$ we have $G_t = R_{t+1}$, which corresponds to immediate reward maximization (i.e. the greedy algorithm with respect to the reward).

This notation can also be used for FTH and episodic tasks if we assume that, respectively, for $t \geq T$ and after reaching a terminal state (i.e. for $t \geq \mathcal{T}$) the agent can only cycle back to the last state with zero reward. Doing so turns them into continuing tasks while keeping their original dynamics and, most importantly, their value function. The series (1.4) simplifies to

a finite sum, and when $\gamma = 1$ we once again find the sum of all future rewards like in (1.3). With this assumption, (1.3) and (1.4) can be rewritten with the same unified notation:

Definition 1.4. Let M be a MDP with discount factor $\gamma \in [0, 1]$ where at least one of the following holds:

$$i)\ \gamma < 1 \qquad ii)\ \mathcal{T} < +\infty \qquad iii)\ \exists\, T \in \mathbb{N} \text{ such that } R_t = 0\ \forall\ t > T.$$

Then, the **return** G_t for M at time t is the series of discounted rewards:

$$G_t = \sum_{k=t+1}^{\mathcal{T}} \gamma^{k-t-1} R_k.$$

A rather useful property of discounted return is the recursive relationship

$$G_t = R_{t+1} + \gamma G_{t+1}$$

which has the computational advantage of not having to record the full sequence of rewards.

While theoretically sound, maximizing the discounted return is generally not feasible for multiple reasons:

- It is typically impossible to express in a closed form, meaning we cannot maximize it with analytical methods;
- Is is often very costly to compute and memorize;
- It is not always a deterministic quantity, but instead a random variable which depends on the dynamics of the MDP and other characteristics of the problem. In episodic tasks we have additional sources of randomness such as the termination time $\mathcal{T}$ and the random choice of starting state after termination.

The standard approach for tackling the problem of randomness is to instead maximize the *expected return* $\mathbb{E}[G_t]$. However, with no additional considerations, computing this expected value is still quite complicated. To properly express and characterize expected return we will need to introduce the fundamental concepts of *policy* and *value function*.

1.3 Policies, Value and Optimality

A *policy* represents the choices an agent makes as it explores the task, the actions it takes when finding itself in a given state. Maximizing expected return corresponds to determining which actions lead to better long term rewards: it can be thus be approached as the problem of determining what is the best policy that an agent can follow. With this understanding, "solving a MDP" means determining and outputting an optimal policy, which maximizes expected return among all policies. Our role as makers of the MDP is endow it with a structure that leads the optimal policy to accomplish the task our MDP was built for.

The most basic form of policy is the following:

Definition 1.5. A **deterministic policy** is of a mapping of states to actions $\pi : \mathcal{S} \to \mathcal{A}$, which takes as input a state s and outputs an action $a \in \mathcal{A}(s)$.

A classic example of deterministic policy is one that implements a *greedy algorithm*, choosing actions based on maximizing a certain function. When they exist, we expect optimal policies (for proper notions of optimality) to be of this form, with each action taken being the optimal action for a certain state. We will later see that greedy policies have a pivotal role in characterizing optimality.

Definition 1.6. Let $\mathcal{P}(\mathcal{A})$ be the set of probability distributions over the action space $\mathcal{A}$. A **stochastic policy** is a function $\pi : \mathcal{S} \to \mathcal{P}(\mathcal{A})$ which maps each state $s \in \mathcal{S}$ to a probability distribution $\pi(\cdot|s)$ over $\mathcal{A}$. For every $(s, a) \in \mathcal{S} \times \mathcal{A}$, $\pi(a|s)$ is the probability of selecting action a in state s.

This definition includes deterministic policies: they correspond to assigning probability 1 to a single action, with all remaining probabilities going to 0. With this notation, for every $s \in \mathcal{S}$ we have $\{a \in \mathcal{A} : \pi(a|s) > 0\} \subseteq \mathcal{A}(s)$, meaning that the distribution $\pi(\cdot|s)$ is supported on $\mathcal{A}(s)$.

With our definitions, policies do not depend on the time t but only on the state itself; policies of this kind are called *stationary*. Throughout this work all policies will be assumed to be stochastic and stationary unless otherwise specified. We will denote with Π the set of all stochastic stationary policies.

We now turn to our objective of maximizing expected return in a RL task. Given a policy π, we can use it to compute the expected return when starting the task and following π. This allows us to estimate how good it is for an agent to be in a state s (or how good it is to choose action a in state s) based on this expected value.

Definition 1.7. The **value function** $v_\pi : \mathcal{S} \to \mathbb{R}$ of a policy $\pi \in \Pi$ is the expected return when starting in state s and following π from then onwards:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t | S_t = s \right].$$

We use $\mathbb{E}_\pi$ to express that the expectation is computed based on moving according to π after starting from s. This means that when computing the expectation each action a will be weighted by a factor $\pi(a|s)$, as we will see shortly in calculations. An important consequence of the stationarity of both the policies and the system's dynamics is that the value function does not depend on the choice of time t inside the expected value, making the value function well defined as a function of only the state s. The value function acts as a measure of how good it is to be in a certain state under a given policy, a measure we aptly call *value*.

An important property of the value of a given state is the ability to be expressed in terms of the value of the states that can be reached after visiting s:

$$\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi \left[G_t | S_t = s \right] = \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} | S_t = s \right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} | R_{t+1} = r, S_{t+1} = s', S_t = s \right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma \mathbb{E}_\pi \left[G_{t+1} | S_{t+1} = s' \right] \right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_\pi(s') \right]
\end{aligned}$$

where the dependence on S_t and R_{t+1} cancels out because of the Markov property, and conditioning on $R_{t+1} = r$ allows us to take r out of the expected value.

The final expression is called *Bellman Equation* for v_π in s:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_\pi(s')\right]. \tag{1.5}$$

The Bellman Equation establishes a relationship between the value of a state and the values of the states that immediately follow it, according to the dynamics of the system. It takes on the classical form of an expected value, a weighted sum where the weights are determined by both the dynamics of the MDP and the specific policy. Through this equation, the value function can be interpreted as a breadth-first search-based method for estimating return, looking into a single step in the future and averaging all (weighted) possibilities.

We can use the same approach to define the value of a state-action couple (s, a), which is a measure of how good it is to choose action a in a state s.

Definition 1.8. The **action-value function** $q_\pi : \mathcal{S}\times\mathcal{A} \to \mathbb{R}$ of a policy $\pi \in \Pi$ is the expected return when starting in state s, choosing action a and following π from then onwards:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t | S_t = s, A_t = a\right].$$

Like v_π, the action-value function q_π can be expressed as an expectation on the values of the states visited after s:

$$\begin{aligned}
q_\pi(s, a) &= \mathbb{E}_\pi \left[G_t | S_t = s, A_t = a\right] = \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a\right] \\
&= \sum_{s',r} p(s', r|s, a) \left[r + \gamma \mathbb{E}_\pi \left[G_{t+1} | S_{t+1} = s', A_t = a\right]\right] \\
&= \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_\pi(s')\right] \qquad (1.6) \\
&= \mathbb{E} \left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a\right]. \qquad (1.7)
\end{aligned}$$

Notice that the dependence on A_t cancels out because of the Markov property, and that in the last equation the expectation depends on π only through v_π. This allows us to express q_π in terms of v_π: comparing (1.5) and (1.6) gives rise to the relation

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a) = \mathbb{E}_\pi \left[q_\pi(s, A_t)\right]. \tag{1.8}$$

In the case of deterministic policies we have $\pi(a|s) = 1 \iff a = \pi(s)$ and the weighted sums in (1.5) and (1.8) simplify to

$$\begin{aligned}
v_\pi(s) &= \sum_{s',r} p(s', r|s, \pi(s)) \left[r + \gamma v_\pi(s')\right], \\
v_\pi(s) &= q_\pi(s, \pi(s)).
\end{aligned}$$

As we will see, v_π is the unique solution of the Bellman Equation (1.5), and it can be computed explicitly by solving a system of linear equations assuming perfect knowledge of the

system's dynamics. This is not ideal, as the computational cost will rise quickly for tasks with a large number of states. Furthermore, even once the value function has been determined, its computation can itself be costly, especially when it has to be performed many times.

Despite these issues, the value function is a useful theoretical tool which we will now use to define exactly what it means for a MDP to solve its task. Given two policies $\pi, \pi' \in \Pi$ we say that π is a better policy than π' if and only if it achieves a better value in each state, i.e. $v_\pi(s) \geq v_{\pi'}(s)\ \forall\ s \in \mathcal{S}$. Our aim is to find a policy π_* that is better than all other policies, meaning $v_{\pi_*}(s) \geq v_\pi(s)\ \forall\ s \in \mathcal{S},\ \forall\ \pi \in \Pi$.

Definition 1.9. The **optimal value function** is $v_*(s) := \sup_{\pi \in \Pi} v_\pi(s)$.

Definition 1.10. A policy π_* is **optimal** if its value function is v_*.

Definition 1.11. The **optimal action-value function** is $q_*(s,a) := \sup_{\pi \in \Pi} q_\pi(s,a)$.

Because the optimal policy is characterized by realizing the optimal value function, it heavily depends on the discount rate γ through the definition of value function. This is particularly evident when considering the extremes $\gamma = 0$, which corresponds to immediate reward maximization, and $\gamma = 1$, where each action is equally important no matter how far in the future. It is easy to show that, even in very simple MDPs, the optimal policies for these two regimes can be very different.

The optimal value function is an upper bound to the value that can be achieved by any stationary policy π. In principle, it is not obvious that this upper bound can be reached simultaneously in all states by a single policy π_*, i.e. that an optimal policy exists. We will devote the following sections to prove the existence of optimal policies for two very relevant categories of MDPs.

1.4 Optimal Policies for Discounted MDPs

Definition 1.12. A MDP is called ***discounted*** if its discount factor is $\gamma < 1$, and we will denote it as $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma)$. If instead $\gamma = 1$, it is called *undiscounted*.

As we discussed, undiscounted MDPs have to be FTH or episodic in order for the return to be well defined, while continuing tasks are forced to introduce a discounting factor (unless their rewards are null after a finite time). Of course, both FTH and episodic tasks are allowed to introduce a discounting factor if so desired, for example in tasks where real life constraints make short and medium term decisions more relevant. We can also artificially extend FTH and episodic MDPs to the continuing case by having the system loop with reward zero after termination time. Because of this, while discounting is most relevant for continuing tasks, the results of this section extend to every type of MDP we introduced.

This section will be devoted to showing that all finite discounted MDPs admit an optimal policy. To do this we will use a simple yet insightful fixed point argument, which we adapted with [9] as the main reference. The same argument also allows us to characterize optimal policies as greedy with respect to the optimal action-value function, culminating in the following theorem:

Theorem 1.1. (Characterization of optimal policies for discounted finite MDPs): Every finite MDP with discount factor $\gamma < 1$ admits at least one optimal deterministic policy. Furthermore, a policy is optimal for its MDP if and only if it is greedy with respect to q_*.

We start by recalling the following classical result:

Theorem 1.2. (Banach Fixed-Point Theorem): Let $(V, \|\cdot\|)$ be a Banach space and $T : V \to V$ a γ-contraction, i.e. a function such that $\|Tu - Tv\| \leq \gamma \|u - v\|$ for some $\gamma < 1$ for any $u, v \in V$. Then T admits a unique fixed point x^* and, for every $x_0 \in V$, if $x_{n+1} = T(x_n)$ then $x_n \to x$ in the norm $\|\cdot\|$ and $\|x_n - x^*\| \leq \gamma^n \|x_0 - x^*\|$.

Let $\mathcal{B}$ denote the space $\mathcal{B}(\mathcal{S}) = \{v : \mathcal{S} \to \mathbb{R} : \|v\|_\infty < +\infty\}$ of bounded functions over the set of states $\mathcal{S}$. $\mathcal{B}$ is a Banach space with the infinity norm $\|v\|_\infty = \max_{s \in \mathcal{S}} |v(s)|$.

Given a policy $\pi \in \Pi$ and a discount factor $\gamma < 1$ consider the *Bellman operator* $T_\pi : \mathcal{B} \to \mathcal{B}$,

$$T_\pi v(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v(s') \right].$$

Assume that the set of rewards $\mathcal{R}$ is bounded, i.e. there exists some $M > 0$ such that $\mathcal{R} \subseteq [-M, M]$. This is always true for finite MDPs, since $\mathcal{R}$ is finite. We have

$$\begin{aligned}
|T_\pi v(s)| &= \left| \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v(s') \right] \right| \\
&\leq \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left| r + \gamma v(s') \right| \\
&\leq \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[M + \gamma \|v\|_\infty \right] \\
&= \sum_a \pi(a|s) \left[M + \gamma \|v\|_\infty \right] \\
&= M + \gamma \|v\|_\infty
\end{aligned}$$

where in the last two steps we use that both $p(\cdot, \cdot|s, a)$ and $\pi(\cdot|s)$ sum up to one once s and a are fixed. This proves that T_π is is a well-defined operator on $\mathcal{B}$. Furthermore, T_π is a γ-contraction with respect to the infinity norm: for every state $s \in \mathcal{S}$ we have

$$\begin{aligned}
&|T_\pi u(s) - T_\pi v(s)| = \\
&\left| \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma u(s') \right] - \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v(s') \right] \right| \\
&= \gamma \left| \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[u(s') - v(s') \right] \right| \\
&\leq \gamma \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left| u(s') - v(s') \right| \\
&\leq \gamma \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \|u - v\|_\infty
\end{aligned}$$

$$= \gamma \sum_a \pi(a|s) \|u - v\|_\infty$$
$$= \gamma \|u - v\|_\infty$$

and thus $\|T_\pi u - T_\pi v\|_\infty = \max_{s\in\mathcal{S}} |T_\pi u(s) - T_\pi v(s)| \leq \gamma \|u - v\|_\infty$.

Because T_π is a contraction, by Banach's fixed-point theorem it admits a unique fixed point. Recall now the Bellman Equation (1.5): it states exactly that v_π is a fixed point of T_π, meaning that v_π is exactly the unique fixed point of the operator T_π. On a computational level, this also gives us a way to approximate v_π: for any starting point $v_0 \in \mathcal{B}$ we can iterate the operator T_π and the succession of iterates will converge geometrically to v_π.

Consider now the *Bellman Optimality Operator* $T_* : \mathcal{B} \to \mathcal{B}$,

$$T_* v(s) = \max_{a\in\mathcal{A}} \left\{ \sum_{s',r} p(s', r|s, a) \left[r + \gamma v(s') \right] \right\}.$$

Because $\mathcal{A}$ is finite, the above maximum always exists. Like before, when $\mathcal{R}$ is bounded by some $M < +\infty$, the operator T_* is well defined:

$$\begin{aligned} |T_* v(s)| &= \left| \max_{a\in\mathcal{A}} \left\{ \sum_{s',r} p(s', r|s, a) \left[r + \gamma v(s') \right] \right\} \right| \\ &\leq \max_{a\in\mathcal{A}} \left\{ \sum_{s',r} p(s', r|s, a) \left| r + \gamma v(s') \right| \right\} \\ &\leq \max_{a\in\mathcal{A}} \left\{ \sum_{s',r} p(s', r|s, a) \left[M + \gamma \|v\|_\infty \right] \right\} \\ &= M + \gamma \|v\|_\infty . \end{aligned}$$

Furthermore, T_* is also a γ-contraction. To show this we start from the general inequality

$$\left| \max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a) \right| \leq \max_{a\in\mathcal{A}} |f(a) - g(a)| .$$

This is because, if $\max_{a\in\mathcal{A}} f(a) \geq \max_{a\in\mathcal{A}} g(a)$ then

$$\begin{aligned} \left| \max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a) \right| &= \max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a) \\ &= \max_{a\in\mathcal{A}} \left\{ f(a) - \max_{a'\in\mathcal{A}} g(a') \right\} \\ &\leq \max_{a\in\mathcal{A}} \{ f(a) - g(a) \} \\ &\leq \max_{a\in\mathcal{A}} |f(a) - g(a)| \end{aligned}$$

as $-\max_{a'\in\mathcal{A}} g(a') \leq -g(a)$ for every $a \in \mathcal{A}$.

Likewise, if $\max_{a\in\mathcal{A}} f(a) \leq \max_{a\in\mathcal{A}} g(a)$ then

$$\begin{aligned}\left|\max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a)\right| &= \max_{a\in\mathcal{A}} g(a) - \max_{a\in\mathcal{A}} f(a) \\ &\leq \max_{a\in\mathcal{A}} \{g(a) - f(a)\} \\ &\leq \max_{a\in\mathcal{A}} |g(a) - f(a)| \\ &= \max_{a\in\mathcal{A}} |f(a) - g(a)| .\end{aligned}$$

Going back to the optimality operator, we have

$$\begin{aligned}&\|T_*u - T_*v\|_\infty = \\ &\max_{s\in\mathcal{S}} \left| \max_{a\in\mathcal{A}} \left\{ \sum_{s',r} p(s',r|s,a) \left[r + \gamma u(s')\right] \right\} - \max_{a\in\mathcal{A}} \left\{ \sum_{s',r} p(s',r|s,a) \left[r + \gamma v(s')\right] \right\} \right| \\ &\leq \gamma \max_{s\in\mathcal{S}} \max_{a\in\mathcal{A}} \sum_{s',r} p(s',r|s,a) \left|u(s') - v(s')\right| \\ &\leq \gamma \max_{s\in\mathcal{S}} \max_{a\in\mathcal{A}} \sum_{s',r} p(s',r|s,a) \|u - v\|_\infty \\ &= \gamma \|u - v\|_\infty .\end{aligned}$$

Again, since $\gamma < 1$, T_* is a contraction and by Banach's fixed-point theorem it admits a unique point $\bar{v} \in \mathcal{B}$. We will now show that, in our setting, this fixed point is none other than the optimal value function.

Definition 1.13. Let $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ be a function such that $\arg\max_{a\in\mathcal{A}} f(s,a) \neq \varnothing$ for every $s \in \mathcal{S}$. A policy $\pi \in \Pi$ is *greedy* in a state $s \in \mathcal{S}$ with respect to f if

$$\pi(\bar{a}|s) > 0 \implies \bar{a} \in \arg\max_{a\in\mathcal{A}} f(s,a).$$

A policy is called greedy if it is greedy in s for every $s \in \mathcal{S}$.

Greedy policies assign positive probability only to actions that maximize f. In the case of finite MDPs, $\mathcal{A}$ is finite and the maximum is well defined for every $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Notice that if π is deterministic, greediness in a state s is equivalent to asking

$$\pi(s) \in \arg\max_{a\in\mathcal{A}} f(s,a).$$

This definition of greediness is based on a given state-action function, but it can be easily extended to make sense for state functions. Like we saw in the case of v_π and q_π, given any state function $v : \mathcal{S} \to \mathbb{R}$ we can define its associated state-action function $q_v : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$,

$$q_v(s,a) := \sum_{s',r} p(s',r|s,a) \left[r + \gamma v(s')\right] .$$

This is a generalized form of equation (1.6), which we can now express as $q_\pi = q_{v_\pi}$.

Definition 1.14. A policy $\pi \in \Pi$ is said to be *greedy* with respect to $v : \mathcal{S} \to \mathbb{R}$ if it is greedy with respect to the state-action function q_v.

When $v = v_\pi$ is the value function of a policy π, being greedy with respect to v_π corresponds to being greedy with respect to the action value function q_π.

Lemma 1.1. A policy $\pi \in \Pi$ is greedy with respect to $v \in \mathcal{B}$ if and only if

$$\sum_{a \in \mathcal{A}} \pi(a|s) q_v(s,a) = \max_{a \in \mathcal{A}} q_v(s,a).$$

Proof: Let $M = \max_{a \in \mathcal{A}} q_v(s,a)$. If π is greedy, on the left-hand side the only actions $\bar{a}$ with nonzero probability verify $q_v(s,\bar{a}) = M$ and the sum of $\pi(\bar{a}|s)$ over all such $\bar{a}$ simplifies to 1 since all mass is concentrated on them. If instead π is not greedy there exists some $\tilde{a}$ such that $\pi(\tilde{a}|s) = \varepsilon > 0$ but $q_v(s,\tilde{a}) < M$ and we have

$$\sum_{a \in \mathcal{A}} \pi(a|s) q_v(s,a) = \varepsilon q_v(s,\tilde{a}) + \sum_{a \neq \tilde{a}} \pi(a|s) q_v(s,a) \leq \varepsilon q_v(s,\tilde{a}) + (1-\varepsilon) M < M.$$

□

Lemma 1.2. A policy $\pi \in \Pi$ is greedy with respect to $v \in \mathcal{B}$ if and only if $T_\pi v = T_* v$.

Proof: Because $v \in \mathcal{B}$, $T_\pi v$ is well defined and

$$T_\pi v(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v(s') \right] = \sum_{a \in \mathcal{A}} \pi(a|s) q_v(s,a).$$

Furthermore, by rewriting the definition of $T_* v$ we have

$$T_* v(s) = \max_{a \in \mathcal{A}} \sum_{s',r} p(s',r|s,a) \left[r + \gamma v(s') \right] = \max_{a \in \mathcal{A}} q_v(s,a).$$

By Lemma 1.1 we know that π is greedy if and only if the right-hand sides are equal, and chaining with the above equalities concludes the proof. □

Theorem 1.3. Let $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma)$ be a discounted finite MDP, let $\bar{v}$ be the unique fixed point of T_* and suppose there exists a policy $\bar{\pi}$ that is greedy with respect to $\bar{v}$. Then $\bar{v} = v_*$ and $\bar{\pi}$ is an optimal policy.

Proof: Because the MDP is finite and discounted the operator T_π and T_* are well defined and both admit a unique fixed point by Banach's theorem. For any policy π, we have $T_\pi \leq T_*$ as operators. This is because

$$T_\pi v(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_v(s,a) \leq \sum_{a \in \mathcal{A}} \pi(a|s) \left(\max_{a \in \mathcal{A}} q_v(s,a) \right) = \max_{a \in \mathcal{A}} q_v(s,a) = T_* v(s).$$

This means that $v_\pi = T_\pi v_\pi \leq T_* v_\pi$. Furthermore, given $u \leq v$, it is obvious that $T_* u \leq T_* v$. Applying this to $v_\pi \leq T_* v_\pi$ yields $T_* v_\pi \leq (T_*)^2 v_\pi$, and chaining the inequalities we obtain $v_\pi \leq (T_*)^2 v_\pi$. By iterating this procedure we have that $v_\pi \leq (T_*)^n v_\pi$ for every n. Because T_* is a contraction, the right side converges to the unique fixed point of T_*, which is $\bar{v}$, and taking the limit we have $v_\pi \leq \bar{v}$. Because this holds for every π, we also have $v_* \leq \bar{v}$.

Let now $\bar{\pi}$ be a $\bar{v}$-greedy policy. By Lemma 1.2 we have

$$T_{\bar{\pi}} \bar{v} = T_* \bar{v} = \bar{v}$$

since $\bar{v}$ is a fixed point for T_*. This in turn means that $\bar{v}$ is a fixed point for $T_{\bar{\pi}}$, and we know that the unique fixed point for $T_{\bar{\pi}}$ is $v_{\bar{\pi}}$. This means $v_{\bar{\pi}} = \bar{v}$ and thus $v_* \geq v_{\bar{\pi}} = \bar{v}$; we conclude that $v_* = \bar{v}$ and that $\bar{\pi}$ is optimal since $v_{\bar{\pi}} = \bar{v} = v_*$. □

Corollary 1.1. Every discounted finite MDP admits a deterministic optimal policy.

Proof: In a discounted finite MDP, for any $v \in \mathcal{B}$ the maxima of the function q_v are well defined and by Banach's theorem there exists a unique fixed point $\bar{v} \in \mathcal{B}$ of T_*. We can construct a deterministic, $\bar{v}$-greedy policy in he following way: for every state $s \in \mathcal{S}$ we pick an action $\bar{a} \in \arg\max_{a \in \mathcal{A}} q_{\bar{v}}(s, a)$ and assign probability $\pi(\bar{a}|s) = 1$ to it. This deterministic policy is $\bar{v}$-greedy and thus optimal by Theorem 1.3. □

Remark 1.3. If the set $\arg\max_{a \in \mathcal{A}} q_{\bar{v}}(s, a)$ consists of a single action, an optimal policy has no choice but to be deterministic in state s as all the probability mass has to be concentrated on the one available maximal action. However, if multiple actions maximize $q_{\bar{v}}(s, a)$ any policy that distributes the probability mass between them will be optimal, meaning strictly stochastic optimal policies can exist.

Corollary 1.2. A policy π_* is optimal if and only if it is v_*-greedy.

Proof: By Theorem 1.3 we have that a v_*-greedy policy is optimal. Given an optimal policy π_*, we have $T_{\pi_*} v_* = T_{\pi_*} v_{\pi_*} = v_{\pi_*} = v_* = T_* v_*$ and by Lemma 1.2 this is true if and only if π_* is v_*-greedy. □

To conclude the proof of our main theorem we now turn to the optimal action-value function. Just like v_*, it is defined as the upper bound on all policies. Given an optimal policy π_*, which we now know always exists for finite MDPs, we have $v_{\pi_*} = v_*$ and

$$
\begin{aligned}
q_*(s, a) &= \sup_{\pi \in \Pi} q_\pi(s, a) \\
&\overset{(1.6)}{=} \sup_{\pi \in \Pi} \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_\pi(s') \right] \\
&\leq \sum_{s', r} p(s', r|s, a) \left[r + \gamma \left(\sup_{\pi \in \Pi} v_\pi(s') \right) \right] \\
&= \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_*(s') \right] \\
&= \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_{\pi_*}(s') \right] \overset{(1.6)}{=} q_{\pi_*}(s, a). \qquad (1.9)
\end{aligned}
$$

Because by definition $q_* \geq q_{\pi_*}$ we have $q_* = q_{\pi_*}$ and all the above are equalities, which shows that q_* is the state-action function associated to v_*. This means that Corollary 1.2 can equivalently be stated as "a policy π_* is optimal if and only if it is q_*-greedy", which together with Corollary 1.1, concludes the proof of Theorem 1.1.

Remark 1.4. Because the optimal value function is the fixed point of a contraction, Theorem 1.1 not only shows the existence of an optimal policy, but it gives us an iterative algorithm to find it. This is because, as stated in the Banach fixed-point theorem, the sequence of iterates obtained by repeatedly applying T^* converges geometrically to its fixed point, which is the optimal value function v_*. From there, any policy that is greedy with respect with v_* will be optimal. This procedure is at the basis of Dynamic Programming algorithms and Policy Iteration, which we will introduce and discuss in Chapter 3.

Remark 1.5. The first part of the proof can be easily extended to discrete MDPs by taking the supremum instead of the maximum in our definitions. In order for the operators to be well defined we must assume $\mathcal{R}$ is bounded, which is not a given if it is not finite. However, from Definition 1.13 onwards we need maxima to exist in order to properly characterize greediness and to ensure that the greedy policy in Theorem 1.3 always exists. This can be achieved by introducing assumptions such as compactness and continuity.

One advantage of the optimality conditions we just proved is that we can now express v_* without referring to any specific policy.

Lemma 1.3. In a discounted finite MDP, for every $s \in \mathcal{S}$

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s,a).$$

Proof: We showed that in discounted finite MDPs v_* is the unique fixed point of T_*, i.e. $v_* = T_*[v_*]$. This means that for every $s \in \mathcal{S}$ we have

$$v_*(s) = T_* v_*(s) = \max_{a \in \mathcal{A}} \left\{ \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_*(s') \right] \right\} = \max_{a \in \mathcal{A}} q_*(s,a).$$

□

Using the above Lemma, we can derive a version of the Bellman Equation (1.5) for the optimal value function:

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_*(s,a) \\ &= \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_*(s') \right]. \end{aligned} \tag{1.10}$$

The expressions (1.10) for all $s \in \mathcal{S}$ are called *Bellman Optimality Equations.* If the dynamics p of the MDP are completely known, the optimal value function can be computed by solving the system of equations given by (1.10) without ever using policies. Of course, in practice this is not efficient: we would have to solve a system of $|\mathcal{S}|$ nonlinear equations in $|\mathcal{S}|$ unknowns, which becomes numerically intractable as the number of states grows. Nonetheless this is remarkable because, if v_* is known, we can immediately build an optimal policy by assigning positive probability only to actions $a \in \mathcal{A}(s)$ that realize the maximum in (1.10) for every $s \in \mathcal{S}$.

1.5 Optimal Policies for Finite Time Horizon MDPs

Now that we characterized optimality for discounted MDPs we will focus on undiscounted MDPs, and more specifically on finite time horizon tasks. Compared to the continuous case, the presence of a termination time makes finding the optimal policy fundamentally different. On an intuitive level, knowing we only have time until T to accumulate reward, we might expect some choices to be worse or better depending on how much time we have left. Namely, at time $t = T-1$ the optimal choice should always be the action that maximizes the immediate reward; the same action may, however, be worse in the long run and thus not the optimal choice at time $t = 0$. To visualize this, consider the finite time horizon MDP in figure 2.

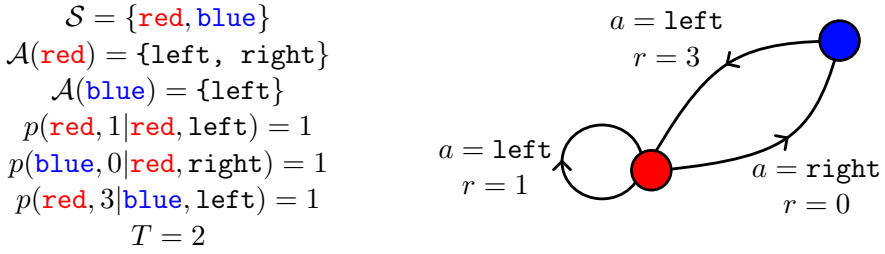


Figure 2: Example of a finite time horizon MDP for which the optimal policy is not stationary.

The dynamics on the left indicate that this MDP is deterministic: the next state S_{t+1} is uniquely determined by the previous state S_t and the chosen action A_t, and so is the reward R_{t+1}. Because of this, the value function of a deterministic policy π (which makes A_t uniquely determined by S_t) simplifies to the sum of rewards obtained starting from a state s and moving according to π until the termination time $T = 2$. In figure 2, this MDP is represented using with states as nodes and actions as arcs.

Assume we start from `red` at time 0. The optimal sequence of actions is to go `right` towards `blue` and then `left` back to `red`, for a total reward of $0 + 3 = 3$, rather than looping back to `red` two times, for a total reward of $1 + 1 = 2$. However, if we find ourself in `red` at time $t = 1$, it is no longer optimal to go `right` with reward 0, as the termination time of $T = 2$ means we have no time to go back and receive a reward of 3; it is instead better to loop back with reward 1. It is easy to see that the policy $\tilde{\pi}$ that goes `right` if $t = 0$ and `left` if $t = 1$ is optimal for this MDP.

Notice however that $\tilde{\pi}$ is not a viable policy according to our definitions. This is because in the beginning we assumed to only work with stationary policies: these policies only depend on the state s, and do not depend on time. In this case, a stationary policy is forced to make the same choice regardless of t, and will thus always be strictly worse than $\tilde{\pi}$ in at least one state. This is very easy to show for deterministic policies: if $\pi(\texttt{red}) = \texttt{left}$ then $v_\pi(\texttt{red}) = 1 + 1 = 2 < 3$, and if $\pi(\texttt{red}) = \texttt{right}$ then $v_\pi(\texttt{blue}) = 3 + 0 = 3 < 3 + 1 = 4$, which is what $\tilde{\pi}$ achieves when starting from `blue` at $t = 0$ and choosing `left` at $t = 1$. This generalizes to any stochastic $\pi \in \Pi$ by taking convex combinations, meaning that $\tilde{\pi}$ is strictly better than any stationary policy.

Our problematic policy $\tilde{\pi}$ belongs to the larger class $\Pi^{HD} = \{\pi(\cdot|h_t)\}$ of *History Dependent* stochastic policies, which do not depend only on the current state but instead on the entire *history* $h_t := (S_0, A_0, S_1, A_1 \dots S_{t-1}, A_{t-1}, S_t)$, which is the sequence of state and actions up to t. Of particular interest within Π^{HD} is the class $\Pi^M = \{\pi(\cdot|S_t)\}$ of *Markovian* stochastic policies, which are policies that depend on the history only through S_t. Note that this is different than depending only on the current state (stationarity) because there is an implicit dependence on the time t. The distinction between these three classes is especially relevant when discussing optimality: because $\Pi \subsetneq \Pi^M \subsetneq \Pi^{HD}$, it is possible that there exists a Markovian or history dependent policy which achieves better results than every stationary policy for the same task. This would mean that defining the optimal policy as a supremum only within Π and not Π^M or Π^{HD} is flawed, and keeps us from finding the actual best possible policy for our task. The optimal policy $\tilde{\pi}$ that we have built is an example of this, as $\tilde{\pi}$ is Markovian but not stationary and it performs strictly better than all stationary policies.

With this new knowledge, we must now ask ourselves whether or not our choice of only considering stationary policies was reductive from the beginning. We start from the discounted case, which we thoroughly analyzed in the previous section. The main idea of our fixed point argument is to act on value functions for stationary policies, acting on the space $\mathcal{B}$ of bounded functions from $\mathcal{S}$ to $\mathbb{R}$. While defining value functions for history dependent policies is not as straightforward, because of the dependence on time, they can still be represented as bounded functions. It is thus reasonable to assume that a similar fixed point argument can successfully capture optimality even in a much larger class of policies. For finite MDPs this is exactly the case, as stated by the following ([7], Theorem 6.2.10):

Theorem 1.4. Let M be a discounted MDP and assume that $\mathcal{S}$ is discrete and $\mathcal{A}(s)$ is finite for every $s \in \mathcal{S}$. Then there exists a deterministic stationary policy $\pi_* \in \Pi$ that is optimal among all history dependent policies.

The proof hinges on two main elements: first of all, in the case of continuing MDPs it is superfluous to consider all history dependent policies as it can be shown that for every $\pi \in \Pi^{HR}$ there exists a Markovian policy $\pi' \in \Pi^M$ which achieves the same value. From then on we can focus on just Markovian policies, which are much easier to represent in the space $\mathcal{B}$, and conclude with a slight variation of our fixed point argument. This result ensures that taking the supremum over the class Π of stationary stochastic policies is not restrictive in the case of discounted MDPs, confirming the significance of the results we proved in the last section.

On the opposite, finite time horizon MDPs tend to have strictly history dependent optimal solutions, as we have just witnessed. An optimal policy can be computed explicitly by generalizing the procedure with which we constructed $\tilde{\pi}$. This algorithm is known in the literature as **Backward Induction** ([7], Chapter 4.5) and it outputs an optimal history dependent policy; if at each step only one maximal action is chosen, the outputted policy is also deterministic. The induction starts at $t = T - 1$, where the optimal values at time $T - 1$ are obtained by choosing the actions which maximizes the immediate reward R_T. For the inductive step $t \to t - 1$ we find the actions that maximize the sum of the previous reward R_{t-1} and the expectation of the optimal values at time t, which are given inductively by the previous step; the algorithm proceeds until $t = 0$. Each step outputs a tail sequence of optimal values from t onwards, and at termination time we obtain a complete sequence of optimal values for our MDP. Furthermore, because these values are computed by finding which actions maximize a certain target at each step, any policy which distributes its probability mass between these maximal actions is optimal. This policy will be Markovian by construction as actions are decided by going exactly one step back in time, and deterministic if at each step we assign probability 1 to a single action.

While not overly difficult to formally state or prove, properly outlining the proof of the Backward Induction algorithm requires introducing some relatively heavy notation for history dependent policies and value functions. We thus decided to omit it and instead refer to the following result ([7], Theorem 4.5.1):

Theorem 1.5. For every finite MDP with finite time horizon there exists a deterministic Markovian policy $\pi_* \in \Pi^M$ which is optimal among all history dependent policies.

This means that we do not need to consider the entirety of history dependent policies, as the supremum is realized in the much friendlier class of Markovian policies. Starting from Markovian policies gives us a chance to characterize the optimal policies of undiscounted

finite time horizon MDPs relying only on the theory of stationary policies, by adopting a clever framing of the problem that bypasses the difficulties we encountered.

We now know that in the case of undiscounted, finite time horizon MDPs we cannot always find a stationary policy that is truly optimal. What we can do to get around it is construct an auxiliary MDP that does admit a stationary optimal policy, and use that policy to reproduce the Markovian optimal policy of the original MDP. The construction we devised is as follows: let $M = (\mathcal{S}, \mathcal{A}, \mathcal{R}, p, T)$ be a finite time horizon MDP with termination time T and consider the MDP $M_T = (\mathcal{S}_T, \mathcal{A}_T, \mathcal{R} \cup \{0\}, p_T)$, where

- $\mathcal{S}_T = \mathcal{S} \times \{0 \dots T\}$;
- $\mathcal{A}_T(s,t) = \mathcal{A}(s) \ \forall \ s \in \mathcal{S}, \ \forall \ t \in 0 \dots T-1$;
- $p_T((s', t+1), r|(s,t), a) = p(s', r|s, a) \ \forall \ t = 0 \dots T-1, \ \forall \ s, s' \in \mathcal{S}, a \in \mathcal{A}(s)$.

We start by taking the cartesian product $\mathcal{S} \times \{0, \dots T\}$ as our new state space. We then define our dynamics so that they reproduce the original dynamics of M on the first coordinate, while the second coordinate records the passing of time from $t = 0$ until T. Notice that because the original dynamics $p(s', r|s, a)$ already sum up to one, from the first equation it follows that $p_T((s', t'), r|(s,t), a) = 0$ if $t' \neq t+1$, as the states $(s, t+1)$ already absorb all of the available probability mass. Essentially, we are taking $T+1$ separate copies of $\mathcal{S}$ and allowing transitions only from states of the form (s,t) to states of the form $(s', t+1)$. This MDP has a finite time horizon T; it can also be interpreted as an episodic MDP, with starting states $(s, 1)$ and terminal states (s, T).

The structure we bestowed upon M_T induces a natural correspondence between the Markovian policies for M and the stationary policies for M_T. Given a Markovian policy $\pi(\cdot|S_t = s)$ for M, which is a probability distribution over $\mathcal{A}(s)$ that depends on both the state s and the time t, we can naturally define the policy π_T for M_T given by $\pi_T(\cdot|(s,t)) = \pi(\cdot|S_t = s)$. The new policy π_T only depends only the state $(s,t) \in S_T$, and is thus stationary with respect to M_T. Conversely, given a stationary policy $\pi'_T(\cdot|(s,t))$ for M_T we can define the Markovian policy $\pi'(\cdot|S_t = s) = \pi'_T(\cdot|(s,t))$ on M. This correspondence is not merely formal but it actually captures the dynamics of the MDP: being in state (s,t) in M_T corresponds exactly to being in state s at time t in M, or simply $S_t = s$. Because our dynamics preserve the same rewards, value is preserved when going from markovian policies of M to stationary policies for M_T. This means that the optimal Markovian policy for M, which exists because of Theorem 1.5 and performs better than all other Markovian policies for M, corresponds to a stationary policy for M_T that performs better than all other stationary policies for M_T and is thus an optimal policy. The existence of an optimal policy immediately allows us to extend some of the results of Section 1.4 to the FTH case, namely equation (1.9), which does not rely on having $\gamma < 1$ but only on the existence of an optimal policy.

Let us now assume we have a method to determine the optimal stationary policy of a given MDP. As we saw, if we applied it directly to a finite time horizon MDP M the optimal stationary policy (if it exists) could perform strictly worse than the actually optimal Markovian policy for M. To avoid this we can instead apply our method to M_T to determine an optimal stationary policy π_*, and use π_* and the structure of M_T to derive a Markovian optimal policy for the original M. This realizes our goal of finding an optimal policy for M, which is however non stationary. In practice, as we will see in the next chapter, in the case of FTH

tasks we will work directly on M_T, or more generally on versions of the original MDP where the set of states is endowed with temporal information, forcing stationarity.

Remark 1.6. In the literature, it is already common practice to include the time t in the set of states, as is the case for example in [1] and [2]. With the added knowledge that FTH MDPs admit Markovian optimal policies, this modeling choice (which is intuitively motivated by wanting to let our agent know the current time) is actually pivotal to properly capture the optimal policy. Of course, our construction for M_T is not the only option to implement this. For example, in [11] Wagner operates on the space of undirected graphs $\mathcal{G} \approx \{0,1\}^N$ by representing them as binary strings, where each bit is 1 if a certain edge is present in the current graph and 0 otherwise. At each time step, the model considers whether to add that edge to the graph or not according to some predetermined ordering of the edges. The inputs of his method are made up of two vectors: one which represents the graph and another one of the same size which is 1 on the index of the edge that the model is considering and 0 everywhere else. Because all edges are considered once in a fixed order, this essentially embeds the passing of time on the second vector.

This concludes our overview of the theory of MDPs. The main takeaways from this chapter are that optimal stationary policies always exist for discounted MDPs, and they also exist for undiscounted, finite time horizon problems as long as the set of states is endowed with temporal information. Furthermore, optimal policies are greedy with respect to the optimal value function, which in turn is characterized by the Bellman Optimality equations. These facts give us a theoretical background which we will build upon in the next chapter, where we will introduce the problem of modeling MDPs for maximization tasks.

Chapter 2

RL Models for Maximization

The process of solving a task through Reinforcement Learning by using a MDP can be split in two key steps: building a MDP and solving it. As previously discussed, solving a MDP means determining and outputting an optimal stationary policy. In the previous chapter we mostly focused on this second step by laying the theoretical background needed to work with MDPs, defining optimality and characterizing the optimal policy. In this chapter we will instead focus on the first step: starting from a given task, we want to build a MDP such that its optimal policy can be used to complete the original task. We will refer to this part of the RL process as building or *modeling* a MDP.

2.1 Modeling a MDP

Modeling a MDP starts with choosing and describing each of its components. The first step is defining the space of states $\mathcal{S}$, which has to be crafted so that each state can contain all the current information on the environment that can be relevant to our agent. As discussed in Chapter 1, this characterization of states can be interpreted as equivalent to the Markov property. After that, we have to define what actions are available to our agent in every state s, which corresponds to $\mathcal{A}(s)$, and with them the dynamics p, which determine the effect each action has on the environment. Very often these three components emerge naturally from the problem itself, as we have seen in some of our examples in the last chapters. The resulting intuitive model might however need to be tweaked first in order to fall within our definitions and allow us to solve them with the methods we know. We have already seen some examples of this in the last section of Chapter 2, such as artificially extending the game to be continuing and introducing a time coordinate in $\mathcal{S}$ for the FTH case. Using again tic tac toe as an example, the intuitive model is an episodic game that can be turned into a FTH task, which then needs to be further modified by introducing time information in order to find a stationary optimal solution of the game. Such modifications make the problem tractable with the tools we have while preserving the original structure of the problem.

Of all the elements of a MDP perhaps the most important, and the one where its creator can have the most input, is the choice of reward. While generally speaking the reward is a random variable, it is useful to think of the immediate reward as a function $r = r(s, a)$ of the state-action pair. This is exactly what happens for deterministic MDPs, which will be our main focus in the following sections. Usually we want the reward function to be high for "good" actions and low (often zero or negative) for "bad" actions, so that the agent is steered towards good actions and away from bad ones while trying to maximize return.

These quantitative decisions embed our external knowledge of the task in the structure of the MDP itself. The objective is for the numeric optimality of our output policy, which depends only on reward values we assigned, to translate into learning the best possible way to solve the real life task we modeled.

Just like for the other elements, there are situations where the reward function emerges quite naturally from the problem itself. Very often in the case of board games, the winner is the player who maximizes a certain point score; for games of this form, the score itself can be used as the reward function. However, at times, an already existing score function may not be a suitable reward function as is, and may need some modifications. For example, the point system in chess based on the material value of the player's pieces point does not capture their position on the board and their importance in the current phase (early/mid/late) of the game, both of which are fundamental in chess. To circumvent this, there are many modern point systems that add and subtract fractions of points based on position and game phase. Such modifications are different from the transformations of states and actions of the last chapter, as they do not preserve the original structure of the system; changing the reward function can result in MDPs with very different optimal solutions. In that case the objective is to engineer a new reward function that characterizes winning strategies better than the original score, and use it to solve the original task.

In this chapter we will focus on the problem of discrete maximization of a known function, which we first mentioned in the introduction. We will discuss general modeling guidelines, pointing out common mistakes and some subtleties in the definitions of states and actions. We will then explicitly build several MDPs that share a similar standard structure for states and actions, explaining our options for each of their four components. We will delineate how different choices of states, actions, rewards and dynamics generate different MDPs and for each one we will show whether the resulting MDP can achieve its goal or not. More specifically, for each fixed MDP we will either prove that it can successfully maximize any function f or show that there exists some function $\bar{f}$ that cannot be maximized by it.

2.2 Modeling discrete maximization

Let us recall the concept of conjecture, which we first mentioned in the introduction.

Definition 2.1. A *conjecture in normal form* over the elements of a space $\mathbb{X}$ is a statement of the form $f(x) \leq 0 \ \forall \ x \in \mathbb{X}$ for some function $f : \mathbb{X} \to \mathbb{R}$.

Disproving a conjecture in normal form corresponds to finding some $\bar{x} \in \mathbb{X}$ such that $f(\bar{x}) > 0$; if such an $\bar{x}$ does not exist, the conjecture is true. A parallel task to this is maximizing the function f, which means finding a point $x_* \in \mathbb{X}$ such that $f(x_*) = \max_{x \in \mathbb{X}} f(x)$. Once we determine a maximum point x_*, if $f(x_*) \leq 0$ we have proved the conjecture to be true; if instead $f(x_*) > 0$ we have disproved our conjecture. Notice that in the second scenario, in order to disprove the conjecture we do not strictly need to find a maximum point as long as somewhere along the process of maximization we find a suitable $\bar{x}$ such that $f(\bar{x}) > 0$.

We are specifically interested in the scenario where $\mathbb{X}$ is a finite but very large space, where the problem of maximization is not easily solved. It is impossible to apply analytical methods, both because of the discrete nature of $\mathbb{X}$ and because in many conjectures the function

f is not continuous or differentiable. In theory, because $\mathbb{X}$ is finite we can always find a maximum through exhaustive each; unfortunately this becomes intractable when $\mathbb{X}$ is very large, especially if computing f is not numerically trivial. This scenario is very relevant because many interesting finite classes of objects have sizes that grow extremely fast. For example, the space of undirected graphs with n nodes has $2^{\binom{n}{2}} = 2^{\frac{n(n-1)}{2}}$ elements, meaning its size grows very quickly even for relatively small n.

These difficulties are the reason we turn to reinforcement learning to try and build a maximizer for f that is of degree 0, meaning it relies on computing only f and not any of its derivatives, and that does not resort to brute force methods. This maximizer will be a MDP modeled after the properties of the conjecture; our aim is to build it in a way such that its optimal policy allows us to find a maximum x_*, so that solving the MDP corresponds to maximizing f and thus proving o disproving the conjecture. We will start by discussing the problem of modeling a MDP in order to solve a generic conjecture $f(x) \leq 0 \; \forall \; x \in \mathbb{X}$ without assuming anything about the function f or the space $\mathbb{X}$ themselves.

2.2.1 Structure of the MDP

We start by looking at each of the four components $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p)$ of a MDP.

$\mathcal{S}$: The first step in building our MDP to solve a conjecture is choosing the set of states. Because our objective is to explore the space $\mathbb{X}$ in order to find a maximum, the most natural choice is to have $\mathcal{S} = \mathbb{X}$. In this setting policies, which are functions of the states, can be directly interpreted as ways to explore the space $\mathbb{X}$. In general, we can expect the state space to be based on $\mathbb{X}$; some modifications may be needed depending on the specific task. We will sometimes refer to MDPs with $\mathcal{S} = \mathbb{X}$ as *intuitive* models.

$\mathcal{A}$: Determining the set of actions $\mathcal{A}$ is less straightforward: because actions are characterized by their effect on the environment, aka the way they act on states, they heavily depend on the choice of $\mathcal{S}$. In this case, choosing $\mathcal{S} = \mathbb{X}$ means that we cannot know in advance what actions look like without information on the structure of the space $\mathbb{X}$.

$\mathcal{R}$: Because our aim is to maximize f, it is natural to use f to define the reward function. The choice of $\mathcal{S} = \mathbb{X}$ allows us to do exactly that, as each state $s \sim x$ carries the information given by $f(s) \sim f(x)$.

p : As in the case of actions, there is not a lot we can say about the dynamics without taking into account the structure of the space $\mathbb{X}$. We can however make the operative choice to consider MDPs with deterministic dynamics. This is preferable because deterministic MDPs are usually much easier to work with, in terms of both computation and interpretability. Of course, if the space $\mathbb{X}$ has some intrinsic form of randomness it would need to be reflected in the dynamics in order for the MDP to accurately capture the problem. In our setting of discrete maximization this does not tend to be the case, and all of our examples in the next section will have deterministic dynamics. This will be especially relevant when defining what it means for an optimal policy to converge to a maximum.

The notion of computing f directly on the states can be extended beyond the specific scenario $\mathcal{S} = \mathbb{X}$ if we have access to a way to associate a unique point $x_s \in \mathbb{X}$ to every state $s \in \mathcal{S}$, by setting $f(s) := f(x_s)$. The states s associated to maximum points, by which we mean

that $x_s = x_* \in \arg\max_{x\in\mathbb{X}} f(x)$, will be referred to as *optimal states*. Assuming that we can compute f on states and that the dynamics are deterministic we can now explicitly define two natural choices of reward function based on f. For every state-action pair (s, a), because our MDP is deterministic, there exists a unique state s' such that if we choose the action a in the state s the environment transitions to s' with probability 1. Let us denote this state as $s \oplus a$. We will consider the two following types of reward function:

- The *full reward* $r(s, a) = f(s \oplus a)$ is given by computing f in the arrival state $s \oplus a$. This incentivizes the agent to pick actions which lead to states with higher values of f, thus maximizing f. When a MDP uses the full reward we will denote it with $r = f$.
- The *incremental reward* $r(s, a) = f(s \oplus a) - f(s)$ gives a reward based on the increment over the last state. This means that the immediate reward is positive when moving to a state with a higher value of f and negative when moving to a state with a lower value of f. This steers our agent into choosing actions that increase the value of f, which again leads to maximizing f. When a MDP uses the incremental reward we will denote it with $r = \Delta f$.

Each approach has its own pros and cons, depending on the structure of $\mathbb{X}$ and the function f; when building the example MDPs in the next section we will consider both types of reward each time and compare the results.

2.2.2 Convergence to a maximum

We must now define what it means for a policy to find a maximum. We will need distinct definitions for the two scenarios we will focus on: finite time horizon and continuing MDPs.

Definition 2.2. For a MDP with finite time horizon T we say that a policy π *converges* to a maximum of f if, for every choice of $S_0 = s \in \mathcal{S}$, starting from s and following π we have that S_T is an optimal state with probability 1:

$$\forall\ s \in \mathcal{S}\ \mathbb{P}_\pi\left(f(S_T) = \max_{x\in\mathbb{X}} f(x)\ \middle|\ S_0 = s\right) = 1.$$

Definition 2.3. For a continuing MDP we say that a policy π *converges* to a maximum of f if there exists some $N \in \mathbb{N}$ such that, for every choice of $S_0 = s \in \mathcal{S}$, starting from s and following π the system reaches an optimal state at time N at the latest and can only loop to the same state or transition to other optimal states afterwards:

$$\exists\ N \in \mathbb{N} :\ \forall\ s \in \mathcal{S},\ \forall\ t \geq N\ \mathbb{P}_\pi\left(f(S_t) = \max_{x\in\mathbb{X}} f(x)\ \middle|\ S_0 = x\right) = 1.$$

In both cases we want to make sure that following a policy for a fixed amount of steps (T or N) is guaranteed to lead us to a maximum point. In the continuing case there is the added possibility of oscillating between two or more optimal states if we check at different times after N; this is not a problem, as every maximum point is equivalent if all we want is to maximize f. The reason we want N to be fixed in advance is to make sure we don't have to wait for an indefinite amount of time for the policy to converge, which is of course impossible in a real life setting. We settled on these definitions of convergence to strike a balance between practicality and theoretical soundness, so that they are able to be proved or disproved using the theory we developed in Chapter 1.

When π is a deterministic policy for a deterministic MDP we have an even simpler way to characterize convergence. Given a state s, a deterministic policy π determines a unique action $a = \pi(s)$, which depends only on s and π. Because the MDP is deterministic a causes the environment to transition to the unique state $s' = s \oplus a$, which depends only on (s, a); but because a is uniquely determined by s through π, s' then depends only on s and π. With some slight abuse of notation we can write this as $s' = \pi(s)$, by which we mean that s' is the only state that is reached by starting in s and following π for one step. While using $\pi(s)$ to denote a state could be ambiguous, as $\pi(s)$ is also the action $a = \pi(s)$ as an output of a policy $\pi : \mathcal{S} \to \mathcal{A}$, this is not too problematic as both are uniquely determined by s and π. This process of action and state determination can be iterated: $\pi^2(s) = \pi(\pi(s))$ is the only state that is reached by starting in s and following π for two steps, and so on. This generates a sequence of states $(s, \pi(s), \pi^2(s) \dots \pi^n(s) \dots)$, where $\pi^0(s) = s$, that depends only on s and π. With this notation we have that a deterministic policy converges to a maximum in the FTH case if for every starting state $s \in \mathcal{S}$ the state $\pi^T(s)$ is optimal; in the continuing case we have convergence if $\pi^t(s)$ is optimal for every $t \geq N$. This notation greatly simplifies our computations compared to the previous chapter, where we dealt with stochastic policies and non-deterministic MDPs. Furthermore, focusing on deterministic policies is not reductive because, as we showed in the previous chapter, we know there always exists an optimal policy that is deterministic, and the optimal policy is the one that we want to converge to a maximum.

Another advantage of deterministic policies for deterministic MDPs is that they allow us to derive a very compact form for the value function. The value function of a policy π in a state s is the expectation of the return when starting from s and following a policy π, where the expectation is taken with respect to the dynamics of the system and the stochasticity of the policy π. When the policy and the MDP are deterministic this expected value is trivial and it coincides with the return, which is the (discounted) series of rewards. Because the MDP is deterministic these rewards can be represented by a reward function $r(s, a)$ and because π is deterministic we have $a = \pi(s)$. Like before, with a slight abuse of notation we can refer to this reward simply as $r(\pi(s))$. Using the notation we just developed, given a reward function r all rewards will be of the form $r(\pi^k(s))$ for some $k \in \mathbb{N}$ which gives us the formula

$$v_\pi(s) = \sum_{k=1}^{\infty} \gamma^{k-1} r(\pi^k(s)). \tag{2.1}$$

We will use this formula extensively to characterize optimal policies and the optimal value function.

2.2.3 Additional considerations

Depending on the structure of the task, in order for these convergence conditions to be realizable we may have to make some additional assumptions. First of all, we will have to assume that for every $s \in \mathcal{S}$ there exists an optimal state that can be reached after starting from s, so that the success of our task does not depend on the choice of initial state. While this is a desirable property in general, one might think it is sufficient to show there exists some starting state $s_0 \in \mathcal{S}$ that always leads to an optimal state. However, simply requesting existence is not enough because in many cases such a state exists trivially, for example if s_0 is itself optimal. This would mean that a good starting state always exists but it cannot be known in advance, making this useless for the purpose of finding the maximum. A sufficient

condition would be to show that such a state exists regardless of the choice of $\mathbb{X}$, and that it can be known in advance.

Another problem one may run into is that of oscillation between optimal and suboptimal states. Suppose that the function f has a unique maximizer x_* and that x_* is reached at a time t. As we will see in our examples, usually the best course of action is to then never move from that state, as every state x that is reachable from x_* tends to perform worse. However, if all the actions in $\mathcal{A}(s)$ result in the system transitioning to another state, even the optimal policy has no choice but to move away from the maximum towards some second-best state x'. It might then move back to x_* on the next move, only to be forced to leave again on the very next step. This creates an oscillation effect between optimal and suboptimal states, preventing the optimal policy from converging to a maximum point according to our definition. This phenomenon can also happen when there are multiple maxima, especially when they are far from each other action-wise. A possible solution is requesting that for every state $s \in \mathcal{S}$ the set of available actions $\mathcal{A}(s)$ includes a special `skip` action that loops the state to itself. However, this may significantly alter the original dynamics depending on the structure of the space $\mathbb{X}$, and introduce some ambiguities in undiscounted MDPs. When discussing the example MDPs in the next section we will take care to underline how and when such `skip` actions are necessary to make sure the MDP can function properly.

With this we have reached the end of what one can say about modeling a discrete maximization task without taking into account any knowledge of the space $\mathbb{X}$. In the next section we will instead focus on practical examples of MDPs on a fixed space, in order to analyze which choices for states, actions, rewards and dynamics allow us to achieve our objective of maximizing a function. To do this we chose a very simple yet extremely adaptable space, that can be used to model many different tasks: the space of binary strings of length n, $\mathbb{X} = \{0,1\}^n$.

2.3 Maximization over binary strings

We chose to use $\mathbb{X} = \{0,1\}^n$ for our MDP examples for many reasons. First of all, because its size grows exponentially with n it is an example of a finite but very large space, which makes brute force methods infeasible and justifies our desire to approach this problem using reinforcement learning. Furthermore, a lot of interesting spaces and objects can be represented using binary strings. For example, an undirected graph with n nodes (and no self loops) can be written down as a binary string of length $\frac{n(n-1)}{2}$ by ordering the arcs and setting each bit to be 1 if an arc is in the graph and 0 otherwise. Characterizing MDPs with this choice of $\mathbb{X}$ allows us to derive results on tasks that can be codified using binary strings, which makes $\mathbb{X}$ especially relevant in the case of proving or disproving conjectures.

Now that we have fixed our space $\mathbb{X}$ we can start to fill in the gaps by determining the set of actions $\mathcal{A}$. As a reminder, actions represent our agent interacting with the environment in ways that lead the system to transition to a new state. In our setting, actions will be operations that can modify binary strings. The most basic operation on binary strings is flipping a bit: given a string $x = (x_i)_{i=1}^n \in \mathbb{X}$ and an index $k = 1, 2, \dots, n$, flipping the k-th bit of x outputs the string $x' \in \mathbb{X}$ where $x'_k = 1 - x_k$ and $x'_h = x_h$ if $h \neq k$. This identifies n distinct actions $\texttt{flip}_1 \dots \texttt{flip}_n$, which act deterministically on strings. As discussed in the last section, it is often advisable to also have a special `skip` action to avoid oscillating

behaviors in the optimal policy. The resulting space of actions is

$$\mathcal{A} = \{\texttt{flip}_1, \dots, \texttt{flip}_n, \texttt{skip}\}\,.$$

Using bit flips as the basis of our action space makes interpreting policies very simple while giving us a lot of freedom in the way we explore $\mathbb{X}$, since any two points in $\mathbb{X}$ can be reached from each other by flipping at most n bits.

Remark 2.1. Of course, using `flip` actions is not the only possible choice for the state of actions. An arguably even simpler action than flipping a bit is simply setting it to 0 or 1 regardless of its previous value. Indeed, using $\mathcal{A}' = (\texttt{set}_1^0, \texttt{set}_1^1, \texttt{set}_2^0, \texttt{set}_2^1, \dots, \texttt{set}_n^0, \texttt{set}_n^1)$ as our set of actions would lead to almost identical results in the following examples. There is also no need of a special `skip` action to ensure that we can stay in the same state indefinitely, as we can just keep setting the same bit to the value it already has. Other options for actions include taking the opposite string (which is 1 where the previous string is 0 and vice versa) and shifting all bits to the left or to the right. Choosing what kind of actions are best for the MDP is an important part of modeling, and it can heavily depend on the task at hand and on what each bit represents.

We begin our experimentation with MDPs starting with the intuitive model $\mathcal{S} = \mathbb{X}$, using the space of actions $\mathcal{A}$ that we just defined. Because flipping bits is well defined for every element of $\mathbb{X}$, to further simplify our setting we will assume $\mathcal{A}(s) = \mathcal{A}$ for every s. Furthermore, the dynamics are determined by the nature of the flip bits themselves: starting from a state $s = x$ and choosing an action $a = \texttt{flip}_k$, the system transitions with probability 1 to the only string x' whose bits are all equal to the bits in x except for the bit in position k. Using our notation for the dynamics this can be written as

$$p\left((x_1, \dots, 1 - x_k, \dots, x_n), r_k \mid (x_1, \dots, x_k, \dots, x_n), \texttt{flip}_k\right) = 1.$$

The reward $r_k \in \mathcal{R}$ depends on the index k and on the choice of reward function, which is the last element we need to fully define our MDPs now that we have determined the set of actions and the resulting dynamics. We have already introduced two possible choices that can be used for any function f: the full reward $r = f$ and the incremental reward $r = \Delta f$.

We will start our analysis by considering MDPs with $\mathcal{S} = \mathbb{X}$, which we presented as a very natural initial choice of states. This forces us to only consider continuing tasks: to solve FTH tasks we need to introduce time information in the space of states, meaning that $\mathcal{S} = \mathbb{X}$ no longer holds. For each MDP we will show that either its optimal policies converge to a maximum for every function f or there exists some function $\bar{f} : \mathbb{X} \to \mathbb{R}$ such that optimal policies do not converge to any of its maximum points in at least one starting state.

2.3.1 Continuous, Full reward

Let $f : \mathbb{X} \to \mathbb{R}$ be a function on $\mathbb{X} = \{0, 1\}^n$ and let $M(f)$ be the continuous MDP defined by $\mathcal{S} = \mathbb{X}$, $\mathcal{A}(s) = \mathcal{A} = \{\texttt{flip}_1, \dots, \texttt{flip}_n, \texttt{skip}\}\ \forall\ s \in \mathcal{S}$ and $r = f$. Because $M(f)$ is continuous, in order for return to be well defined we need to either introduce a discount factor $\gamma < 1$ or make sure that rewards are definitely zero. Because we chose $r = f$, the second condition does not hold for a generic function f. We must then choose a discount factor $\gamma < 1$; we denote the corresponding continuous MDP as $M(f, \gamma)$.

An interesting property of the optimal policies for $M(f,\gamma)$ is that if they reach the global maximum point for f they are then forced to never move away from it. Because $\mathbb{X}$ is finite there always exists at least one global maximum point $x_* \in \mathbb{X}$; for simplicity we will assume that it is unique. Let π_* be an optimal deterministic policy for $M(f,\gamma)$, which always exists because it is a discounted finite MDP. Let $s \in \mathcal{S}$ and assume that after starting from s the policy π_* transitions to x_* at some time t_s, which (using our notation for deterministic policies) can be written as $\pi_*^{t_s}(s) = x_*$.

Lemma 2.1. Let π_* be an optimal policy for $M(f,\gamma)$ and let x_* be the unique global maximum of f. Given a starting state $s \in \mathcal{S}$, if there exists a time t_s such that $\pi_*^{t_s}(s) = x_*$ then π_* always skips for $t \geq t_s$, never moving away from x_*.

Proof: We want to show that $\pi_*(\pi_*^t(s)) = \texttt{skip}\ \forall\ t \geq t_s$. Because $\pi_*^{t_s}(s) = x_*$, this is equivalent to showing that $\pi_*^t(s) = x_*\ \forall\ t \geq t_s$. Let π_s be a policy that (starting from s) coincides with π_* until time t_s, where it reaches x_*, and then always skips. Using the formula (2.1) we can express the value of the state s as

$$v_{\pi_s}(s) = \sum_{k=1}^{\infty} \gamma^{k-1} f(\pi_s^k(s)) = \sum_{k=1}^{t_s} \gamma^{k-1} f(\pi_*^k(s)) + \sum_{k=t_s+1}^{\infty} \gamma^{k-1} f(x_*).$$

Compare this expression with the optimal value function, which can be computed using π_*:

$$v_*(s) = \sum_{k=1}^{\infty} \gamma^{k-1} f(\pi_*^k(s)) = \sum_{k=1}^{t_s} \gamma^{k-1} f(\pi_*^k(s)) + \sum_{k=t_s+1}^{\infty} \gamma^{k-1} f(\pi_*^k(s)).$$

Subtracting the above expressions we find

$$v_{\pi_s}(s) - v_*(s) = \sum_{k=t_s+1}^{\infty} \gamma^{k-1} \Big(f(x_*) - f(\pi_*^k(s)) \Big) \geq 0. \tag{2.2}$$

The above quantity is greater than zero because x_* is a global maximum for f, which means $f(x_*) - f(\pi_*^k(s)) \geq 0$ for every k. Because by definition the optimal function is an upper bound on all possible value functions we also have $v_*(s) \geq v_{\pi_s}(s)$ which together with (2.2) implies $v_{\pi_s}(s) - v_*(s) = 0$. This is in turn implies $f(x_*) - f(\pi_*^k(s)) = 0$ for every k, and because x_* is the unique global maximum this is possible if an only if π_* always skips after reaching x_*, which concludes our proof. □

Remark 2.2. The above proof can be generalized when f has more than one global maximum. Instead of always skipping after reaching a maximum, there is also the possibly to move to other optimal points if they are one action away. This is not a problem according to our definition of optimality, which we formulated in a way that accommodates this kind of behavior. However if a global maximum point is isolated, meaning we cannot move from it to another maximum point in one action, an optimal policy is still forced to stop there and skip indefinitely. This is because as soon as one non optimal action is chosen by v_* the sum in (2.2) has at least one term that is strictly greater than zero, contradicting optimality.

Theorem 2.1. If $n > 2$, for every $\gamma < 1$ there exists a function $\bar{f}$ such that the optimal policies of $M(\bar{f},\gamma)$ do not converge to a maximum of $\bar{f}$.

Proof: Let us assume by contradiction that, for any given $f : \mathbb{X} \to \mathbb{R}$, every optimal policy of $M(f, \gamma)$ converges to a maximum of f. The main idea of the proof is to build a function $\bar{f}$ with a local maximum point that is distant from the global maximum, where by distance we mean the minimum amount of actions needed to go from one to the other. Because $r = \bar{f}$, if we start from a state that is close to the local maximum and far from the global maximum it might be better return-wise for a policy to converge to the local maximum. This behavior is enabled by the discounting effect induced by γ, which gives more weight to earlier actions and less weight to later ones. If this is true the optimal policy cannot converge to the global maximum, contradicting our assumption and thus proving our thesis.

Let $\mathbf{0}, \mathbf{1} \in \mathbb{X}$ denote respectively the string of all zeros $\mathbf{0} = \{0, \dots, 0\}$ and the string of all ones $\mathbf{1} = \{1, \dots, 1\}$. For $i \in \{1, \dots, n\}$ let e_i be the string $(e_i)_j = \delta_{ij}$ that is 1 only in the i-th coordinate. In our setting, distance between strings is given by how many bits they have in common: if two strings x_1, x_2 have k bit in common, we can move from one to the other in exactly $n-k$ bit flips. The maximum distance between strings in $\{0,1\}^n$ is n; this is achieved by the strings $\mathbf{0}$ and $\mathbf{1}$, which will be our candidates for local and global maximum point respectively. To simplify our calculations we will assume that $\bar{f}(x) = 0 \; \forall \; x \in \mathbb{X} \setminus \{\mathbf{0}, \mathbf{1}\}$, so that we only have to determine two values: $\bar{f}(\mathbf{0})$ and $\bar{f}(\mathbf{1})$. Asking for $\bar{f}$ to have a local maximum in $\mathbf{0}$ and a global maximum in $\mathbf{1}$ gives us the condition $0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1})$.

Consider the starting state $e_1 = \{1, 0, \dots, 0\}$ and let $\pi_{\mathbf{0}}$ be a deterministic policy such that $\pi_{\mathbf{0}}(e_1) = \texttt{flip}_1, \pi_{\mathbf{0}}(\mathbf{0}) = \texttt{skip}$. This means that if we start in state e_1 and follow $\pi_{\mathbf{0}}$ we transition to $\mathbf{0}$ on the first step and then keep looping to it indefinitely. Because $\pi_{\mathbf{0}}$ is deterministic, using expression (2.1) with $r = \bar{f}$ we have that the value of state e_1 is given by

$$v_{\pi_{\mathbf{0}}}(e_1) = \sum_{k=1}^{\infty} \gamma^{k-1} r(\pi_{\mathbf{0}}^k(e_1)) = \sum_{k=1}^{\infty} \gamma^{k-1} \bar{f}(\mathbf{0}) = \bar{f}(\mathbf{0}) \sum_{k=0}^{\infty} \gamma^k = \frac{\bar{f}(\mathbf{0})}{1-\gamma}. \tag{2.3}$$

Let us now consider the behavior of the optimal policy. Because the MDP $M(f, \gamma)$ is finite and discounted which means it admits a deterministic optimal policy π_*, as we showed in the first chapter. If π_* does not converge to the maximum point $\mathbf{1}$ we found an optimal policy that does not converge, proving our claim. Let us then assume that π_* converges to $\mathbf{1}$. By our characterization of convergence for deterministic policies of continuing MDPs there exists some N such that, for every starting state $s \in \mathcal{S}$, $\pi_*^t(s) = \mathbf{1} \; \forall \; t \geq N$. This means that the deterministic policy π_* has the system transition to $\mathbf{1}$ in at most N steps no matter the starting state. This implies that $\pi_*(\pi^t(s)) = \texttt{skip}$ for $t \geq N$, meaning that after at most N steps the system loops to $\mathbf{1}$ indefinitely.

We want to characterize the behavior of the optimal policy in order to find a contradiction using the value of the state e_1. We already computed the value of e_1 according to the policy $\pi_{\mathbf{0}}$, which is a quantity that depends only on γ and $\bar{f}(\mathbf{0})$. Under our assumption that optimal policies converge to a maximum the optimal value $v_*(e_1)$ will depend on $\bar{f}(\mathbf{1})$, as the policy is forced to eventually loop to $\mathbf{1}$. Our objective is to find an upper bound $V_* \geq v_*(e_1)$ that depends only on $\bar{f}(\mathbf{0})$ and $\bar{f}(\mathbf{1})$ and show that there exists some choice of $0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1})$ such that $V_* < v_{\pi_{\mathbf{0}}}(e_1)$; this results in a contradiction because, by definition of optimal policy, we must have $v_*(e_1) \geq v_{\pi_{\mathbf{0}}}(e_1)$. This means that a policy that converges to the maximum of our function $\bar{f}$ cannot be optimal, which concludes our proof.

Because of Lemma 2.1 we know that if an optimal policy reaches the global maximum it can then only skip; this means that if N_s is the first time we reach $\mathbf{1}$ starting from a state s, we have $\pi_*^t(s) = \mathbf{1} \iff t \geq N_s$. Since we assume that the optimal policy π_* converges to the maximum $\mathbf{1}$ for every state, N_s is well defined for every s and $0 \leq N_s \leq N$. Applying this to $s = e_1$ there exists some number $0 \leq N_1 \leq N$ such that $\pi_*^t(e_1) = \mathbf{1} \iff t \geq N_1$. This means that the value of e_1 is of the form

$$v_*(e_1) = \sum_{k=1}^{\infty} \gamma^{k-1} \bar{f}(\pi_*^k(e_1)) = \sum_{k=1}^{N_1-1} \gamma^{k-1} \bar{f}(\pi_*^k(e_1)) + \sum_{k=N_1}^{\infty} \gamma^{k-1} \bar{f}(\mathbf{1})$$

where by definition of N_1 we have $\pi_s^k(e_1) \neq \mathbf{1}$ for each element of the first sum. The above expression characterizes the tail of the series of rewards for an optimal policy. As for the first $N_1 - 1$ terms, recall that we defined $\bar{f}$ so that $0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1})$ and that $\bar{f} = 0$ elsewhere; this means that the first sum is non zero if and only if somewhere along the first $N_1 - 1$ moves we passed through the state $\mathbf{0}$. We have two possibilities:

- If π_* does not ever transition to $\mathbf{0}$ all of the rewards in the first sum are zero and the value function simplifies to

  $$v_*(e_1) = \sum_{k=N_1}^{\infty} \gamma^{k-1} \bar{f}(\mathbf{1}) = \bar{f}(\mathbf{1}) \frac{\gamma^{N_1-1}}{1-\gamma}.$$

  To find a contradiction we must then solve the system

  $$\begin{cases} 0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1}), \\ \frac{\bar{f}(\mathbf{0})}{1-\gamma} > \bar{f}(\mathbf{1}) \frac{\gamma^{N_1-1}}{1-\gamma}. \end{cases}$$

  Because the system of equations is linear in $\bar{f}(\mathbf{0})$ and $\bar{f}(\mathbf{1})$ we can simplify it by assuming $\bar{f}(\mathbf{1}) = 1$, so that we only have to determine $\bar{f}(\mathbf{0})$. The resulting equation is $\gamma^{N_1-1} < \bar{f}(\mathbf{0}) < 1$ which admits the middle point $\bar{f}(\mathbf{0}) = \frac{\gamma^{N_1-1}+1}{2}$ as a solution as long as $\gamma^{N_1-1} < 1$; this is true for every $\gamma < 1$ as long as $N_1 - 1 > 0$. Recall that N_1 is the first time where the optimal policy reaches $\mathbf{1}$ after starting from e_1. Because e_1 only has one bit in common with $\mathbf{1}$, to get from e_1 we need to flip $n - 1$ bits which means $N_1 \geq n - 1$. This means that the condition is met if $(n-1) - 1 = n - 2 > 0$.

- Suppose instead $\pi_*^k(e_1) = \mathbf{0}$ for some $k \in \mathbb{N}$. Let N_0 be the maximum k for which this happens, which is well defined because by definition of N_1 we have $0 \leq N_0 < N_1$. The optimal value function can be written as

  $$v_*(e_1) = \sum_{k=1}^{N_0} \gamma^{k-1} \bar{f}(\pi_s^k(e_1)) + \sum_{k=N_0+1}^{N_1-1} \gamma^{k-1} \bar{f}(\pi_s^k(e_1)) + \sum_{k=N_1}^{\infty} \gamma^{k-1} \bar{f}(\mathbf{1}).$$

  By definition of N_0 and N_1, none of the states visited between N_0 and N_1 are $\mathbf{0}$ or $\mathbf{1}$ and so each summand in the second term is zero. Furthermore, because the policy does not reach $\mathbf{1}$ before N_1 and $N_0 < N_1$ each reward in the first term is less or equal than $\bar{f}(\mathbf{0})$; this gives us the upper bound

  $$v_*(e_1) \leq \sum_{k=1}^{N_0} \gamma^{k-1} \bar{f}(\mathbf{0}) + \sum_{k=N_1}^{\infty} \gamma^{k-1} \bar{f}(\mathbf{1}) = \bar{f}(\mathbf{0}) \frac{1-\gamma^{N_0}}{1-\gamma} + \bar{f}(\mathbf{1}) \frac{\gamma^{N_1-1}}{1-\gamma}.$$

We must now solve the system

$$\begin{cases} 0 < \bar{f}(\mathbf{0}) < \bar{f}(1) \\ \frac{\bar{f}(\mathbf{0})}{1-\gamma} > \bar{f}(\mathbf{0})\frac{1-\gamma^{N_0}}{1-\gamma} + \bar{f}(\mathbf{1})\frac{\gamma^{N_1-1}}{1-\gamma}. \end{cases}$$

Assuming $\bar{f}(\mathbf{1}) = 1$ and rearranging the terms we arrive to the condition $\gamma^{N_1-N_0-1} < \bar{f}(\mathbf{0}) < 1$ which is verified by $\bar{f}(\mathbf{0}) = \frac{\gamma^{N_1-N_0-1}+1}{2}$ for every $\gamma < 1$ if $N_1 - N_0 - 1 > 0$. Because $\mathbf{0}$ and $\mathbf{1}$ need n steps to go from one another we have $N_1 - N_0 \geq n$, which means that $N_1 - N_0 - 1 > 0$ is verified as long as $n > 1$.

This means that if $n > 2$ we can always build a function $\bar{f}$ such that the optimal policies of $M(\bar{f}, \gamma)$ cannot converge to the global maximum, proving our theorem. □

Remark 2.3. As we can see in (2.3), in our setting value functions can generally be expressed as series of values of our function f. Because of this, in most cases the value function is scale invariant in relation to f: if we use λf in place of f for some $\lambda > 0$ the value function is the same with a λ factor, and because of this optimal policies remain the same. We also have a form of translation invariance: if we use $f + K$ in place of f for some constant K the value function is the same with an added term $\frac{K}{1-\gamma}$, which does not affect optimality as it cancels out when comparing value functions.

The condition $n > 2$ is not a problem for us, considering we are specifically interested in scenarios where n is very large. It is however quite notable mathematically, as it expresses a geometric requirement on the minimum amount of space we need. Consider the following diagrams, which represent $\mathbb{X} = \{0, 1\}^n$ for $n = 2$ and $n = 3$:

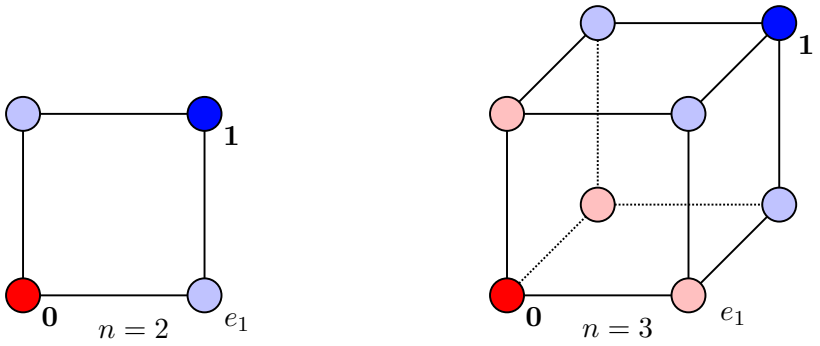


Figure 3: $\mathbb{X} = \{0, 1\}^n$ for $n = 2, 3$. Points are colored based on where an optimal policy converges after starting from them.

In both diagrams the local maximum $\mathbf{0}$ is in red, the global maximum $\mathbf{1}$ is in blue and the starting point e_1 is exactly one step to the right of $\mathbf{0}$. The other nodes are colored based on where an optimal policy converges after starting from them. Notice that for $n = 3$ the state e_1 is colored pink, which is exactly what we just proved: if $n > 2$ and we start from e_1 we achieve a higher return by converging to $\mathbf{0}$ in one step rather than converging to $\mathbf{1}$ in two. The same holds for the other pink points, which are all at distance 1 from $\mathbf{0}$; our proof holds for them as well, as the only thing we used about e_1 is exactly that we can move from it to $\mathbf{0}$ in one step. For the light blue points, who are at distance 1 from $\mathbf{1}$ and distance 2 from $\mathbf{0}$, it is easy to show that the more advantageous choice is to immediately converge to $\mathbf{1}$.

What emerges from this diagram is that the actual condition for policies to be captured by the local maximum is to start from states whose distance from $\mathbf{0}$ is strictly less than the distance from $\mathbf{1}$. This is exactly what characterizes the pink points for $n = 3$, and is also the reason why this is not true for $n = 2$. As we can see on the left, the only point whose distance from $\mathbf{0}$ is strictly lower than its distance from $\mathbf{1}$ is $\mathbf{0}$ itself; e_1 is equally distant from both, and because $f(\mathbf{1}) > f(\mathbf{0})$ it is always better for policies to move from e_1 to $\mathbf{1}$ rather than $\mathbf{0}$. The request $n > 2$ is exactly what is needed to make sure there are points in $\mathbb{X} \setminus \{\mathbf{0}, \mathbf{1}\}$ that are strictly closer to $\mathbf{0}$. The proof is the same as Theorem 2.1: if we choose an initial state x_k who has exactly k bits equal to 1, where $k \leq \lfloor \frac{n-1}{2} \rfloor$, they will be strictly closer to $\mathbf{0}$ than $\mathbf{1}$ because $k \leq n - 1 - k$ and thus $k < n - k$. In our setting k and $n - k$ are respectively the distance of x_k to $\mathbf{0}$ and $\mathbf{1}$; notice that for k to be at least 1 we need $n > 2$, which is exactly what we found. Compared to our previous calculations for e_1 the value $v_{\pi_\mathbf{0}}(x_k)$ will have a factor γ^{k-1}, and the resulting linear condition when the optimal policy avoids $\mathbf{0}$ (with $f(\mathbf{1}) = 1$) will be

$$1 > \bar{f}(\mathbf{0}) > \gamma^{N_1 - 1 - (k-1)} = \gamma^{N_1 - k}.$$

When starting from x_k the number of bits needed to flip to get to $\mathbf{1}$ is $n - k$, which means that $N_1 \geq n - k$ and the system admits a solution as long as $(n - k) - k > 0$ i.e. $k < n - k$, which is exactly our condition of being strictly closer to $\mathbf{0}$. The calculations are analogous when we assume that the optimal policy does pass through $\mathbf{0}$.

Notice that, in our proof of Theorem 2.1, when considering the possibility of passing through $\mathbf{0}$ before converging to $\mathbf{1}$ the minimum space needed goes down to $n > 1$. This happens because if we start from $\mathbf{0}$ instead of e_1 and consider the policy $\bar{\pi}$ that constantly skips, we obtain a contradiction with one less step required. Note that this does not contradict our conclusion that the condition for converging to $\mathbf{0}$ is being strictly closer to $\mathbf{0}$ than $\mathbf{1}$; this is because we measure distance not by the actual distance on the graph but by the number of actions needed to get to one state, and $\mathbf{0}$ has distance 1 from itself as we need one action (`skip`) to go from $\mathbf{0}$ to $\mathbf{0}$. This means that $\bar{\pi}$ converges to $\mathbf{0}$ as long as its distance from $\mathbf{1}$ is strictly greater than its distance from $\mathbf{0}$, which corresponds to asking that $n > 1$. Indeed, when $n = 1$ and thus $\mathbf{0}$ and $\mathbf{1}$ have distance 1 from each other, choosing `skip` gives out reward $f(\mathbf{0})$ which is strictly less than the reward $f(\mathbf{1})$ of moving to $\mathbf{1}$ on the first step, making it impossible for optimal policies to get stuck on $\mathbf{0}$.

With this understanding, if all we want was to show that there is at least one "bad" starting state $\mathbf{0}$, which is sufficient to confute convergence, the necessary condition becomes $n > 1$ (which is in any case required to ensure that $e_1 \neq \mathbf{1}$ in the first place). This would however be somewhat insightful as we are then only guaranteed to have one unsuitable state, which one could argue is negligible in the grand scheme of things; for example, if we decide to start from a random state we only have probability 2^{-n} to hit the one unlucky state, which is very low for high values of n. By characterizing this behavior using distance we find out that for every choice of n about half of our states run into this problem, at least when the global and local maximum are as distant at possible in $\mathbb{X}$. This further confirms that this MDP is not the right choice for discrete maximization.

2.3.2 Continuous, Incremental reward

Let $M(\Delta f)$ be the continuous MDP defined by $\mathcal{S} = \mathbb{X}$, $\mathcal{A}(s) = \mathcal{A} = \{\texttt{flip}_1, \ldots, \texttt{flip}_n, \texttt{skip}\}$ $\forall\ s \in \mathcal{S}$ and $r = \Delta f$. Again, because r depends on f we cannot ensure that rewards are

definitely zero, so we must introduce discounting. Let $\gamma < 1$ be the discount factor and let $M(\Delta f, \gamma)$ be the corresponding continuous MDP.

A peculiarity of the incremental reward is that for every $s \in \mathcal{S}$ choosing `skip` leads to a null reward, as

$$r(s, \texttt{skip}) = \Delta f(s, \texttt{skip}) = f(s \oplus \texttt{skip}) - f(s) = f(s) - f(s) = 0.$$

This means that if a policy π converges to any state s for $t \geq N$, and thus chooses to skip indefinitely, the series of rewards is actually a finite sum of at most N terms. This holds in particular for optimal policies if we assume that they converge to a maximum.

Lemma 2.2. Let π_* be an optimal policy for $M(\Delta f, \gamma)$ and let x_* be the unique global maximum of f. Given a starting state $s \in \mathcal{S}$, if there exists a time t_s such that $\pi_*^{t_s}(s) = x_*$ then π_* always skips for $t \geq t_s$, never moving away from x_*.

Proof: Let π_s be a policy that behaves like π_* until it reaches x_* and then always skips. Its value function is

$$v_{\pi_s}(s) = \sum_{k=1}^{\infty} \gamma^{k-1} r(\pi_{\pi_s}^k(s)) = \sum_{k=1}^{t_s} \gamma^{k-1} \left(f(\pi_*^k(s)) - f(\pi_*^{k-1}(s)) \right)$$

where each term for $k > t_s$ is null because the policy skips. The optimal value function will have the same initial segment as v_{π_s}, which means that subtracting the two leaves us with

$$v_*(s) - v_{\pi_s}(s) = \sum_{k=t_s+1}^{\infty} \gamma^{k-1} \left(f(\pi_*^k(s)) - f(\pi_*^{k-1}(s)) \right).$$

If we isolate a factor γ^{t_s} and write out the series we have

$$\begin{aligned} \frac{v_*(s) - v_{\pi_s}(s)}{\gamma^{t_s}} &= f(\pi_*^{t_s+1}(s)) - f(x_*) + \gamma(f(\pi_*^{t_s+2}(s)) - f(\pi_*^{t_s+1}(s))) \\ &\quad + \gamma^2(f(\pi_*^{t_s+3}(s)) - f(\pi_*^{t_s+2}(s))) + \gamma^3(f(\pi_*^{t_s+4}(s)) - f(\pi_*^{t_s+3}(s))) + \dots \end{aligned}$$

which can be rearranged by highlighting $f(x_*)$ and pairing summands with the same index as

$$\begin{aligned} &= -f(x_*) + (1-\gamma) f(\pi_*^{t_s+1}(s)) + (1-\gamma)\gamma f(\pi_*^{t_s+2}(s)) + (1-\gamma)\gamma^2 f(\pi_*^{t_s+3}(s)) + \dots \\ &= -f(x_*) + (1-\gamma) \sum_{i=1}^{\infty} \gamma^{i-1} f(\pi_*^{t_s+i}(s)). \end{aligned}$$

Because x_* is the global maximum each term of the series is less then or equal to $f(x_*)$, which gives us

$$\begin{aligned} \frac{v_*(s) - v_{\pi_s}(s)}{\gamma^{t_s}} &= -f(x_*) + (1-\gamma) \sum_{i=1}^{\infty} f(\pi_*^{t_s+i}(s)) \\ &\leq -f(x_*) + (1-\gamma) \sum_{i=1}^{\infty} \gamma^{i-1} f(x_*) \\ &= -f(x_*) + (1-\gamma) \frac{f(x_*)}{1-\gamma} = 0. \end{aligned}$$

This means that $v_*(s) - v_{\pi_s}(s) \leq 0$, and if any of the terms $f(\pi_*^{t_s+i}(s))$ is strictly less than $f(x_*)$ the inequality is strict. This would contradict the optimality of π_* as by definition of optimal value function $v_*(s) - v_{\pi_s}(s) \geq 0$, which means that $f(\pi_*^{t_s+i}(s)) = f(x_*)\ \forall\ i \geq 0$. Because x_* is the unique global maximum this can only happen if $\pi_*^t(s) = x_*\ \forall\ t \geq t_s$, which concludes our proof. □

Like in Lemma 2.1 the proof extends to functions with multiple global maximum points. If two or such points are consecutive, optimal policies are free to move among them indefinitely. If instead an optimal policy visits an isolated maximum point it will be forced to skip, as moving to any other state would mean that the first term in the above sum is strictly less than $f(x_*)$ making the inequality strict.

Theorem 2.2. For every $\gamma < 1$ there exists a function $\bar{f}$ such that the optimal policies of $M(\Delta\bar{f}, \gamma)$ do not converge to a maximum of $\bar{f}$.

Proof: The target function $\bar{f}$ is of the same form the same as in Theorem 2.1, in that $0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1})$ and $\bar{f} = 0$ elsewhere. This means that the reward term coincides with the one for $r = f$ for every movement that is not moving away from $\mathbf{0}$ or $\mathbf{1}$, as the second term in the increment is always zero. Let us again consider the state $e_1 = \{1, 0, \dots, 0\}$ and the deterministic policy $\pi_{\mathbf{0}}$ that moves from e_1 to $\mathbf{0}$ and then skips. Following this policy starting from e_1 gives us an initial reward of $\bar{f}(\mathbf{0}) - \bar{f}(e_1) = \bar{f}(\mathbf{0})$ and because $\pi_{\mathbf{0}}$ skips indefinitely after moving to $\mathbf{0}$ all subsequent reward terms are null, which gives us $v_{\pi_{\mathbf{0}}} = \bar{f}(\mathbf{0})$.

Let us now assume by contradiction that every optimal policy converges to the global maximum $\mathbf{1}$, and let π_* be an optimal deterministic policy that converges in at most N steps. By Lemma 2.2 we know that if an optimal policy moves to $\mathbf{1}$ it will be then forced to skip indefinitely. Our assumption of convergence means that for every state $s \in \mathcal{S}$ there exists some number $N_s \leq N$ such that $\pi_*^t(s) = \mathbf{1} \iff t \geq N_s$; let N_1 be this number for $s = e_1$. Again, we have two possibilities:

- If π_* does not transit through the state $\mathbf{0}$, its value function in the state e_1 is

$$v_*(e_1) = \sum_{k=1}^{\infty} \gamma^{k-1} r(\pi_*^k(e_1)) = \sum_{k=1}^{N_1} \gamma^{k-1} r(\pi_*^k(e_1)) = \gamma^{N_1-1} \bar{f}(\mathbf{1})$$

  because the only non-zero term $\bar{f}(x)$ is the one for $x = \mathbf{1}$. Because we need at least $n-1$ steps to go from e_1 to 1 we have that $N_1 \geq n-1$, which leads to the linear system

$$\begin{cases} 0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1}), \\ \bar{f}(\mathbf{0}) > \bar{f}(\mathbf{1})\gamma^{n-2}. \end{cases}$$

  This is solved by $\bar{f}(\mathbf{1}) = 1$, $\bar{f}(\mathbf{0}) = \frac{\gamma^{n-2}+1}{2}$ for any $\gamma < 1$ as long as $n > 2$.

- If π_* does move to $\mathbf{0}$ before converging to $\mathbf{1}$, let N_0 be the maximum time t such that $\pi_*^t(e_1) = \mathbf{0}$. The optimal value function is

$$\begin{aligned} v_*(e_1) &= \sum_{k=1}^{\infty} \gamma^{k-1} r(\pi_*^k(e_1)) \\ &= \sum_{k=1}^{N_0} \gamma^{k-1} r(\pi_*^k(e_1)) + \gamma^{N_0} \left(\bar{f}(\pi_*^{N_0+1}(e_1)) - \bar{f}(\mathbf{0}) \right) \end{aligned}$$

$$+ \sum_{k=N_0+2}^{N_1-1} \gamma^{k-1} r(\pi_*^k(e_1)) + \gamma^{N_1-1} \left(\bar{f}(\mathbf{1}) - \bar{f}(\pi_*^{N_1-1}(e_1)) \right).$$

Because (as already discussed in Theorem 2.1) $N_1 - N_0 \geq n$ and by hypothesis $n > 2$ we have $N_0 + 1 < N_1$; this means that $\bar{f}(\pi_*^{N_0+1}(e_1)) = 0$, as that state is not allowed to be $\mathbf{1}$ or $\mathbf{0}$ by definition of N_0 and N_1. For the same reason the only non zero term in the second part is $\gamma^{N_1-\mathbf{1}} \bar{f}(\mathbf{1})$ and we are left with

$$v_*(e_1) = \sum_{k=1}^{N_0} \gamma^{k-1} r(\pi_*^k(e_1)) - \gamma^{N_0} \bar{f}(\mathbf{0}) + \gamma^{N_1-1} \bar{f}(\mathbf{1}).$$

We can rearrange this sum like we did in in the proof of Lemma 2.2:

$$\begin{aligned} v_*(e_1) - \gamma^{N_1-1} \bar{f}(\mathbf{1}) &= \sum_{k=1}^{N_0} \gamma^{k-1} r(\pi_*^k(e_1)) - \gamma^{N_0} \bar{f}(\mathbf{0}) \\ &= \bar{f}(\pi_*(e_1)) - \bar{f}(e_1) + \gamma \left(\bar{f}(\pi_*^2(e_1)) - \bar{f}(\pi_*(e_1)) \right) \\ &\quad + \gamma^2 \left(\bar{f}(\pi_*^3(e_1)) - \bar{f}(\pi_*^2(e_1)) \right) + \dots \\ &\dots + \gamma^{N_0-1} \left(\bar{f}(\mathbf{0}) - \bar{f}(\pi_*^{N_0-1}(e_1)) \right) - \gamma^{N_0} \bar{f}(\mathbf{0}) \\ &= -\bar{f}(e_1) + (1-\gamma) \sum_{i=1}^{N_0} \gamma^{i-1} \bar{f}(\pi_*^i(e_1)). \end{aligned}$$

Because $N_0 < N_1$ each state $\pi_*^i(e_1)$ can be $\mathbf{0}$ but not $\mathbf{1}$, which means $\bar{f}(\pi_*^i(e_1)) \leq \bar{f}(\mathbf{0})$ for each member of the sum. Furthermore, $\bar{f}(e_1) = 0$. This gives us the upper bound

$$\begin{aligned} v_*(e_1) &= (1-\gamma) \sum_{i=1}^{N_0} \gamma^{i-1} \bar{f}(\pi_*^i(e_1)) + \gamma^{N_1-1} \bar{f}(\mathbf{1}) \\ &\leq (1-\gamma) \sum_{i=1}^{N_0} \gamma^{i-1} \bar{f}(\mathbf{0}) + \gamma^{N_1-1} \bar{f}(\mathbf{1}) \\ &= (1-\gamma) \frac{1-\gamma^{N_0}}{1-\gamma} \bar{f}(\mathbf{0}) + \gamma^{N_1-1} \bar{f}(\mathbf{1}). \end{aligned}$$

This leads to the linear system

$$\begin{cases} 0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1}), \\ \bar{f}(\mathbf{0}) > (1-\gamma^{N_0}) \bar{f}(\mathbf{0}) + \gamma^{N_1-1} \bar{f}(\mathbf{1}) \end{cases}$$

which admits the solution $\bar{f}(\mathbf{1}) = 1, \bar{f}(\mathbf{0}) > \gamma^{N_1-N_0-1}$ for any $\gamma < 1$ as long as $n > 1$ because $N_1 - N_0 \geq n$.

In both cases when $n > 2$ we can define a function $\bar{f}$ such that the optimal policies of $M(\Delta\bar{f}, \gamma)$ cannot converge to the global maximum, which concludes our proof. □

The proof is mostly similar to the proof of Theorem 2.1, as our choice of $\bar{f}$ makes Δf equal to f in the vast majority of cases. Once again it can be shown that the true condition for a policy to get stuck on the local maximum is for it to start on a state that is strictly closer to $\mathbf{0}$ than to $\mathbf{1}$.

This concludes our exploration of continuing MDPs on the space $\mathcal{S} = \mathbb{X}$. In both of the explored scenarios we found that the discounting effect, which we need to introduce to ensure convergence, can cause optimal policies to get stuck on a local maximum point if the starting point is close enough to it. While the counterexamples we built have a very specific structure for the sake of rigorously proving non-convergence, we can expect this to also be a problem for generic functions with sufficiently distant local maxima. We can conclude that continuous MDPs on $\mathcal{S} = \mathbb{X}$ are not a good choice for discrete maximization tasks.

Our next step is to look at what happens if we use Finite Time Horizon MDPs: they are not required to introduce discounting and may thus avoid the problems we encountered in continuous MDPs. For FTH tasks, in order to force stationarity of the optimal policies, we need to introduce the time dimension in the space of states. In our setting this means $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$ where T is our finite time horizon, which is a hyperparameter that we have to choose. Because we want to be able to reach a maximum point from any possible starting state we need to assume $T \geq n$, so that there is enough time to flip all n bits if needed (as is the case for $\mathbf{0}$ and $\mathbf{1}$).

Because of the introduction of a new time coordinate, we have to revise our definitions of convergence and optimality. Recall that, for MDPs with finite time horizon T, convergence to a maximum corresponds to asking that S_T is a maximum point with probability 1 regardless of the starting state $s \in \mathcal{S}$. In the finite horizon case the convergence condition assumes the form $S_T = (x_*, T)$ for $x_* \in \arg\max_{x \in \mathbb{X}} f(x)$, and because $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$ it is sufficient to ask that this condition holds for every state of the form $(x, 0)$. In the following we will use the convention $f((x,t)) = f(x)$ so that we can compute f directly on states, simplifying the reward terms.

2.3.3 FTH, Full reward

Let $T \geq n$ be our fixed time horizon, $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$, $\mathcal{A}(x,t) = \mathcal{A} \; \forall \; x \in \mathbb{X}, t = 0, \dots, T-1$, $r((x,t),a) = r(x,a)$ (according to the methods we discussed in chapter 1) and $r = f$. Let $M(f,T)$ be the resulting finite time horizon MDP.

Theorem 2.3. For every choice of $T \geq n > 1$ there exists a function $\bar{f}$ such that the optimal policies of $M(\bar{f}, T)$ do not converge to a maximum.

Proof: We will once again consider a function $\bar{f}$ such that $0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1})$ and $\bar{f}(x) = 0 \; \forall \; x \in \mathbb{X} \setminus \{\mathbf{0}, \mathbf{1}\}$. If we start from the state $(e_1, 0)$ and consider the deterministic policy $\pi_{\mathbf{0}}$ which moves from $(e_1, 0)$ to $(\mathbf{0}, 1)$ and then skips, which in the FTH scenario corresponds to moving to $(\mathbf{0}, 2)$, then $(\mathbf{0}, 3)$ and so on until $(\mathbf{0}, T)$, the value function is

$$v_{\pi_0}(e_1) = \sum_{k=1}^{T} r(\pi_{\mathbf{0}}^k((e_1, 0))) = \sum_{k=1}^{T} \bar{f}(\mathbf{0}) = T\bar{f}(\mathbf{0})$$

where we used the usual formula (2.1) with $\gamma = 1$ for the undiscounted case. Let now π_* be a deterministic optimal policy that converges to $\mathbf{1}$, which means that $\pi_*^T((x,0)) = \mathbf{1} \; \forall \; x \in \mathbb{X}$. In order to go from e_1 to $\mathbf{1}$ we need to pass through at least $n-1$ states that are neither $\mathbf{1}$ or $\mathbf{0}$. This means that in the sum for the optimal value function there are at least $n-1$ null terms, which gives us the upper bound

$$v_*(e_1) = \sum_{k=1}^{T} r(\pi_*^k((e_1, 0))) \leq (T - n + 1)\bar{f}(\mathbf{1})$$

as there are at most $T-(n-1)$ non-zero terms and each one is trivially less than the global maximum $\bar{f}(\mathbf{1})$. To find a contradiction we need to solve the system

$$\begin{cases} 0 < \bar{f}(\mathbf{0}) < \bar{f}(\mathbf{1}) \\ T\bar{f}(\mathbf{0}) > (T-n+1)\bar{f}(\mathbf{1}) \end{cases}$$

which admits the solution $\bar{f}(\mathbf{1}) = 1$, $\bar{f}(\mathbf{0}) = \frac{2T-n+1}{2T}$ for every $T \geq n$ as long as $n > 1$. □

Remark 2.4. Like we have seen for continuous MDPs, in the finite time horizon case optimal policies are scale and translation invariant in that value functions for λf are the same as the value functions for f multiplied by λ and that functions for $f + K$ are the same as the value functions for f with an additive factor TK. However, this is only true when the time horizon is fixed: if we consider an episodic MDP with non-constant termination time $\mathcal{T}$ we retain scale invariance but not translation invariance. This is because if we consider $f + K$ each value function will have an additive factor $\mathcal{T}K$ which depends on the episode length; when $K > 0$ this promotes longer episodes and discourages shorter episodes, and when $K < 0$ the opposite happens, making optimal policies differ greatly depending on K.

Compared to the previous two examples this proof is considerably more straightforward; this is because the lack of discounting means that we can ignore at the point in time an action is chosen, as the reward is the same. The fact that this MDP also cannot maximize every function is not particularly surprising; for its continuous counterpart we considered converging policies which skip indefinitely after reaching a certain state, and this is conceptually quite close to the MDP terminating after a finite time.

2.3.4 FTH, Incremental reward

Let $T \geq n$, $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$, $\mathcal{A}(x,t) = \mathcal{A}$, $r((x,t),a) = r(x,a)$, $r = \Delta f$ and let $M(\Delta f, T)$ be the resulting finite time horizon MDP.

Theorem 2.4. For every function $f : \mathbb{X} \to \mathbb{R}$, the optimal policies of $M(\Delta f, T)$ converge to a maximum of f.

Proof: Because the time horizon is finite and $\gamma = 1$, the value function of a deterministic policy π in a state $(x, 0)$ is given by a finite telescopic sum, which simplifies to the increment between the starting state and the final state at time T:

$$\begin{aligned} v_\pi((x,0)) &= \sum_{t=1}^{T} r(\pi^t((x,0))) \\ &= \sum_{t=1}^{T} f(\pi^t((x,0))) - f(\pi^{t-1}((x,0))) \\ &= f(\pi^T((x,0))) - f(\pi^0((x,0))) \\ &= f(\pi^T((x,0))) - f(x). \end{aligned}$$

Let $x_* \in \arg\max_{x \in \mathbb{X}} f(x)$ be a global maximum point for f and let π_{x_*} be the policy that, for every starting state x, sequentially flips only the bits of x that are different from the bits of x_* from position 1 to n and then skips. This policy always exists and is deterministic because it depends only on bits and thus only on the current state. Because $T \geq n$ the policy

π_{x_*} always has enough time to reach x_* from any starting state $(x, 0)$, which means it always converges to x_*. With this observation its value function is given by

$$v_{\pi_*}((x,0)) = f(\pi^T((x,0))) - f(x) = f(x_*) - f(x)$$

for every choice of $(x, 0)$. Consider now a deterministic optimal policy π_*. Because of optimality we have $v_*((x,0)) \geq v_{\pi_*}((x,0))$, which means $f(\pi_*^T((x,0))) - f(x) \geq f(\pi_{x_*}^T((x,0))) - f(x)$ and thus $f(\pi_*^T((x,0))) \geq f(x_*)$ for every $x \in \mathbb{X}$. This is only possible if $\pi_*^T((x,0)) \in \arg\max_{x\in\mathbb{X}} f(x)$ for every starting state $(x, 0)$, which tells us exactly that π_* converges to a maximum point. For non-deterministic optimal policies the term $\pi_*^T((x,0))$ corresponds to S_T and the optimality condition becomes $f(S_T) \geq f(x_*) = \max_{x\in\mathbb{X}} f(x)$; this confirms that all optimal stochastic policies converge to a maximum point, concluding our proof. □

We finally found our first successful MDP, whose optimal policies always converge to a maximum. This is enabled by the absence of discounting and by our choice incremental reward, which turns the value function into a telescopic sum and allows us to characterize value based only on the initial and final state, without needing to consider any of the intermediate stages. Because of this the optimal value function is not affected by local maxima, and the MDP is able to avoid the same fate as the ones we previously analyzed.

The success of this MDP also gives us a hint on how to modify our previous MDPs to potentially solve the problem of local maxima. The key point is choosing a reward function such that, for any fixed starting state, the value function depends directly on the final state. For continuing tasks on $\mathcal{S} = \mathbb{X}$ this is difficult to implement, as a discount factor $\gamma < 1$ is necessary for return to be well defined and discounting makes policies sensitive to the order of the intermediate actions by weighting them differently. Furthermore, because of the nature of continuing tasks a state can be final only in the sense that a policy converges to it and not intrinsically. FTH tasks are very different in this regard because each state of the form (x, T) can be properly interpreted as terminal, which allows us to define a third type of reward based on f:

- For a MDP with finite time horizon T with $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$, the *episodic reward* is

$$r((x,t),a) = f(x \oplus a)\mathbb{1}_{\{t=T-1\}} = \begin{cases} f(x \oplus a) & \text{if } t = T-1 \\ 0 & \text{otherwise} \end{cases}$$

  which corresponds to the full reward f only on the final state and is zero otherwise. When a MDP with finite time horizon T uses the episodic reward we will denote it with $r = f_T$.

As the name suggests the episodic reward is often used for episodic MDPs where it can represent the score of a single episode. This is consistent with our observation that FTH tasks can be seen as single episode of an episodic task. Let us now see if this new choice of reward is enough to make sure that our MDPs can maximize every f.

2.3.5 FTH, Episodic reward

Let $T \geq n$, $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$, $\mathcal{A}(x,t) = \mathcal{A}$, $r((x,t),a) = r(x,a)$, $r = f_T$ and let $M(f_T, T)$ be the resulting finite time horizon MDP. This MDP is a version of the *Global* game introduced by Angileri et al. in [1].

Theorem 2.5. For every function $f : \mathbb{X} \to \mathbb{R}$, the optimal policies of $M(f_T, T)$ converge to a maximum of f.

Proof: The proof is analogous to that of Theorem 2.4. Let $(x, 0)$ be a starting state and π a deterministic policy. With episodic reward, its value function in $(x, 0)$ is given by

$$v_\pi((x,0)) = \sum_{t=1}^{T} r(\pi^t((x,0))) = f(\pi^T((x,0)))$$

because, by definition of episodic reward, each reward is zero except the last one are zero which is given by f. Let now $x_* \in \arg\max_{x\in\mathbb{X}} f(x)$ be a maximum point for f and let π_{x_*} be the deterministic policy that flips bits from 1 to n so that they coincide with the bits of x_*. Because $T \geq n$, for every starting state $(x, 0)$ the policy π_{x_*} always has time to flip all necessary bits and reach x_* before time T, which means $\pi_{x_*}^T((x,0)) = x_*$ and thus $v_{\pi_*}((x,0)) = f(x_*)$ for every $x \in \mathbb{X}$. This means that every optimal policy has to satisfy

$$v_{\pi_*}((x,0)) = v_*((x,0)) \geq v_{\pi_{x_*}}((x,0)) = f(x_*).$$

This is only possible if π_* converges to a maximum point for every for every $x \in \mathbb{X}$, concluding our proof. □

Remark 2.5. In this MDP, the presence of the `skip` action is very important to allow convergence at time T. Suppose for a moment that $\mathcal{A} = \{\texttt{flip}_1, \ldots, \texttt{flip}_n\}$, so that that every action always changes one bit; the resulting MDP is a version of the *Flip* game in [1]. If we start from e_1, a policy needs to flip exactly $n-1$ bits to 1 in order to get to 1, which takes up $n-1$ out of T time steps. If $T = n$ we have one remaining action, which forces the policy to move away from $\mathbf{1}$ and prevents it from convergence to $\mathbf{1}$ at time T. More formally, because we cannot `skip`, every action changes the parity of the number of non-zero bits in the current state. We start from e_1 (which has parity 1) and want to get to $\mathbf{1}$ (which has parity $n \mod 2$) in exactly T actions; because each action changes parity this is only possible if $T \equiv n - 1 \mod 2$, i.e. T and $n-1$ have the same parity. Notably this is never true for $T = n$, which is a case of particular interest as we will soon discuss. This problem and its relation to parity stems from the fact that the only way for a policy to go back to a previous state is to choose the same $\texttt{flip}_k$ action an even number of times, whereas if a `skip` action is available we are never forced to move away from the maximum.

As it turns out, using episodic reward is indeed enough to ensure that optimal policies converge to a maximum, at least for finite time horizon tasks. This confirms our hypothesis that delaying reward so that the final return depends only on the final state is a successful strategy to ensure convergence.

While this method finally allowed us to achieve convergence, it is not without its issues, both in the case of incremental and episodic reward. Recall that a policy is optimal if and only if its value function is the optimal value function. Because we made it so that the value function depends only on the final state, every policy that gets to a maximum point will be optimal regardless on how it achieves that. This means that policies may at times choose `skip` before reaching the maximum or even go back and forth between states as long as they eventually converge. This behavior, while still optimal according to our definitions, is not what we would perceive as optimal in real life, as it feels like the policy is essentially wasting

time. Outside of theoretical worries, in a real life setting an overabundance of numerically optimal policies could also negatively affect the convergence of the algorithms used to compute or approximate the optimal policy.

This wandering behavior is possible because of the lack of discounting, which makes every reward equally important regardless of the time of its occurrence. When discounting is applied the agent is incentivized to choose the best actions as soon as possible, which means that optimal policies are not allowed to aimlessly wander around like we described. However, simply introducing a discount factor is not the solution: if $r = \Delta f$ we have already shown that the resulting MDP cannot maximize all functions, whereas when $r = f_T$ the only non zero term is the final one, making discounting irrelevant as maximizing $f(\pi^T((x, 0)))$ is the same as maximizing $\gamma^{T-1} f(\pi^T((x, 0)))$.

Whether the existence of wandering optimal policies is a problem or not depends on our objectives. If all we care about is maximizing a function our goal is met regardless of what the found policy does. In many cases however we are actually more interested in how the policy behaves rather than the actual numeric value. This is especially true in the case of conjectures, which are what prompted our analysis in the first place. Given a conjecture $f(x) \leq 0 \; \forall \; x \in \mathbb{X}$, analyzing the behavior of the optimal policy can help us characterize both the function f and the space $\mathbb{X}$ and allows us to possibly generalize our findings to similar functions and spaces. This is relevant even when the determined optimal policy π_* does not confute the conjecture, i.e. the optimal point x_* that π_* converges to does not satisfy $f(x_*) > 0$. An example of this is Conjecture 2.2 in [11]: the graph found by the reinforcement learning method ([11], figure 5) does not confute the conjecture by itself, but clearly suggests the kind of shape the desired graph should have, which allowed Wagner et al. to iterate on that structure and find the desired counterexample ([11], figure 6).

We will thus try to force more structure on our policies, hoping to make them more representative of how the initial problem is solved. With discounting off the table, we have a few options to make our policies more well behaved. The first problem we can try to fix is that of policies randomly choosing `skip` before reaching an optimal point. Assuming a starting point $(x, 0)$ is k bit flips away from the maximum point, an optimal policy can skip up to $T - k$ times at any point without losing its optimality. This means that choosing a lower time horizon T minimizes the amount of unnecessary `skip` actions. Because we need $T \geq n$ to ensure that policies can converge to a maximum point from every choice of starting state, the best we can do is $T = n$.

Another problem we can fix is policies going back and forth between states, which is what happens for example when the same bit is flipped consecutively. Choosing $T = n$ allows us to avoid this behavior by pairing each time step to a specific bit. More specifically we equip states $(x, k) \in \mathcal{S} = \mathbb{X} \times \{0, \dots, n\}$ with the restricted set of actions

$$\mathcal{A}((x, k)) = \mathcal{A}_{k+1} := \left\{\texttt{flip}_{k+1}, \texttt{skip}\right\} \; \forall \; x \in \mathbb{X}, k \in 0, \dots, T-1.$$

This means that each bit can only be flipped only once: the first bit can be flipped only when transitioning from $t = 0$ to $t = 1$, the second bit can be flipped only when transitioning from $t = 1$ to $t = 2$ and so on until $t = n$, when the MDP terminates. This operation introduces a form of monotonicity in our policies, as they cannot change whatever bit they want at any time but instead only have one opportunity to flip each bit. This means that no back and

forth motion is allowed and that optimal policies cannot choose to randomly `skip`, as they cannot go back to flip it if they miss their one shot.

Reducing the set of actions does not affect the convergence properties of our FTH MDPs, because no matter the starting point $(x, 0)$ we need at most n bit flips to converge to a maximum point. The proofs or Theorem 2.4 and Theorem 2.5 remain the same when $\mathcal{A}((x,k)) = \mathcal{A}_{k+1}$, as all we need for the proof to work is for optimal policies to be able to converge from every starting state and this is also true for MDPs with restricted action space as long as $T \geq n$. Furthermore, the added structure allows us to completely characterize deterministic optimal policies:

Proposition 2.1. Let M be a finite time horizon MDP with $\mathcal{S} = \mathbb{X} \times \{0, \dots, n\}$, $\mathcal{A}(x,k) = \mathcal{A}_{k+1}$, $r((x,t),a) = r(x,a)$ and either $r = \Delta f$ or $r = f_T$. Then, a deterministic policy π is optimal if and only if there is a maximum point $x_* \in \arg\max_{x \in \mathbb{X}} f(x)$ such that $\pi = \pi_{x_*}$, where for every $x \in \mathbb{X}$ and $k = 0, \dots, n-1$

$$\pi_{x_*}((x,k)) = \begin{cases} \texttt{flip}_{k+1} & \text{if } (x)_{k+1} \neq (x_*)_{k+1}, \\ \texttt{skip} & \text{otherwise.} \end{cases}$$

Proof: As we showed in Theorems 2.4 and 2.5, a deterministic policy π for M is optimal if and only if $\pi^n((x,0)) = (x_*, n)$ for some $x_* \in \arg\max_{x \in \mathbb{X}} f(x)$. Let $(x_k, k) = \pi^k((x,0))$ for some $k \leq n$, and from the optimality condition we have that $\pi^{n-k}((x_k,k)) = (x_*, n)$. Because of our restricted action sets the first k bits of x_k cannot be flipped anymore, which means that $(x_k)_h = (x_*)_h \; \forall \, h \leq k$ for every k: after k steps of π the first k bits of our current state must coincide with the first k bits of x_*. Two subsequent states x_k and x_{k+1} must both have the same first k bits as x_*, and by construction $(x_{k+1})_{k+1} = (x_*)_{k+1}$. Because $\pi((x_k,k)) = (x_{k+1}, k+1)$ we have that, respectively, if $(x_k)_{k+1} = (x_*)_{k+1}$ the policy π must choose `skip` and if $(x_k)_{k+1} \neq (x_*)_{k+1}$ the policy π must choose $\texttt{flip}_{k+1}$ to be able to reach x_{k+1}. Because this holds for every starting x and for every k we conclude that $\pi = \pi_*$. □

A consequence of this proposition is that MDPs with this structure have exactly one optimal deterministic policy for each maximum point, which means that if f has a unique maximum point the deterministic optimal policy is also unique. Compared to MDPs with unrestricted action space, the optimal policies are strongly correlated to maximizing f in the most efficient way possible, which achieves our goal of making policies more representative of the optimal solutions to the original problem.

Remark 2.6. The MDP with restricted action set and episodic reward is a version of the *Linear* game introduced by Angileri et al. in [1], which itself is a formalization of the graph construction method of Wagner et al. in [11]. Each bit corresponds to an arc in the graph according to some ordering, so that $\mathbf{0}$ corresponds to the empty graph and $\mathbf{1}$ corresponds to the complete graph. In [11] the algorithm starts from the empty graph and builds it by choosing arcs and flipping bits to 1, while in [1] the MDP starts from the complete graph and discards arcs by flipping bits to 0.

Remark 2.7. Restricting the time horizon and the set of actions is not the only way to introduce monotonicity in a MDP. Consider the graph building MDP from [11]: if we start from $x_0 = \mathbf{0}$ and choose the set of actions $\mathcal{A}^1 = \{\texttt{set}^1_1, \dots, \texttt{set}^1_k, \texttt{skip}\}$, each bit can only be flipped from 0 to 1 but never from 1 to 0, which means that once an arc is chosen to be

part of the graph it can never be discarded. Compared to the MDPs in Proposition 2.1 this ensures monotonicity regardless of the choice of T, and we have access to all bit indexes at any time instead of only one per action.

While Proposition 2.1 is an interesting result, so far we have been able to define successful maximizers only when considering MDPs with a finite time horizon. This was mainly a consequence of the need for discounting making continuous MDPs susceptible to getting stuck in local maxima. In the FTH case we solved this problem by choosing reward functions such that return depends only on the final state, using either incremental or episodic reward. In the basic setting $\mathcal{S} = \mathbb{X}$ we have already shown in Theorem 2.2 that continuous MDPs on with incremental reward cannot maximize every function, and episodic reward is not well defined as there is no natural notion of terminal state or termination time. We will thus try building a completely new structure that can accommodate episodic reward, with the objective of creating a continuing MDP whose optimal policies always converge to a maximum point.

2.3.6 Continuous, Episodic reward

Our new objective is to construct a continuous MDP with a natural definition of episodic reward. As discussed this is not possible if we restrict ourself to $\mathcal{S} = \mathbb{X}$, as there is no proper notion of terminal state or terminal time. Taking inspiration from the FTH setting, we will instead use the space $\mathcal{S} = \mathbb{X} \times \{1, \dots, n\}$ with deterministic dynamics such that states of the form (x, k) for $k < n$ can only move to states of the form $(x', k+1)$ with reward zero. The main difference, which turns the MDP from FTH to continuing, is that once we reach a state of the form (x, n) and pick an action a the system loops back to to the state $(x \oplus a, 1)$, with reward $f(x \oplus a)$. This looping effect induces a form of episodic reward, as the only time we receive a non-zero reward is every n time steps when the system loops back from n to 1.

Remark 2.8. The use of the term *episodic* for the choice of reward could suggest that this MDP could also be interpreted as episodic, with starting states $(x, 1)$ and a single terminal state reachable from states (x, n) so that each iteration is an episode. This is however not true: the looping effect means that the new starting state at time 1 depends on past history, which cannot happen for episodic games.

Remark 2.9. Compared to the finite time horizon case the indexes are shifted by one, because the looping effect eliminates the need for states with $t = 0$ to ensure we have enough time to take n actions. If instead one wants to preserve the same notation as the FTH case, one can instead consider $\mathcal{S} = \mathbb{X} \times \{0, \dots, n\}$ with reward 0 until reaching states (x, n) and looping back to $(x, 0)$ with episodic reward $f(x)$ every $n + 1$ time steps. This introduces an extra term γ in the returns, for a total factor of γ^{n+1} instead of γ^n.

Theorem 2.6. Let $f : \mathbb{X} \to \mathbb{R}$, $\gamma < 1$ and let $M(f_n, \gamma)$ be the MDP defined by

- $\mathcal{S} = \mathbb{X} \times \{1, \dots, n\}$
- $\mathcal{A}((x, k)) = \mathcal{A}_k = \{\texttt{flip}_k, \texttt{skip}\}$
- $\begin{cases} p\left((s \oplus a, k+1), 0 | (s, k), a\right) = 1 \ \forall \ k = 1, \dots, n-1 \\ p\left((s \oplus a, 1), f(s \oplus a) | (s, n), a\right) = 1 \end{cases}$

with discount factor γ. Then, if f has a unique maximum point x_* the only optimal policy for M is $\pi_* = \pi_{x_*}$.

Proof: Let $(x, 1)$ be a starting state and let π_{x_*} be the usual deterministic policy that flips bits only if they differ from the bits of x_*. In n time steps the policy π_{x_*} can always go from $(x, 1)$ to $(x_*, 1)$, and the value function assumes the form

$$\begin{aligned} v_{\pi_{x_*}}((x,1)) &= \sum_{i=1}^{\infty} \gamma^{i-1} r(\pi_{x_*}^i((x,1))) = \gamma^{n-1} \sum_{i=1}^{\infty} \gamma^{(i-1)n} f(\pi_{x_*}^{in}((x,1))) \\ &= \gamma^{n-1} \sum_{i=1}^{\infty} \gamma^{(i-1)n} f(x_*) = \gamma^{n-1} \sum_{i=1}^{\infty} (\gamma^n)^{(i-1)} = \frac{\gamma^{n-1}}{1-\gamma^n} f(x_*). \end{aligned}$$

This means that for any optimal policy π_* we must have $v_*((x,1)) \geq \frac{\gamma^{n-1}}{1-\gamma^n} f(x_*)$. Let now π_* be a deterministic optimal policy; the optimal value function can be expressed as

$$v_*((x,1)) = \sum_{i=1}^{\infty} \gamma^{i-1} r(\pi_*^i((x,1))) = \gamma^{n-1} \sum_{i=1}^{\infty} \gamma^{(i-1)n} f(\pi_*^{in}((x,1))).$$

Comparing the two value functions, the optimality condition $v_*((x,1)) \geq \frac{\gamma^{n-1}}{1-\gamma^n} f(x_*)$ can be achieved only if every single term $f(\pi_*^{in}((x,1)))$ of the series on the right is exactly $f(x_*)$. Because the set of actions is restricted, this can only happen if $\pi_* = \pi_{x_*}$. Indeed, if the two policies differ at some time k it means that the k-th bit of $\pi^n((z,1))$ will be different from the k-th bit x_*. This means that the next episodic reward is f computed on a state that is not x_*, and because x_* is the unique global maximum point it is strictly less than $f(x_*)$, contradicting optimality. Furthermore no strictly stochastic policy π can be optimal: the optimality condition requests that $f(S_{in}) = f(x_*)$ with probability 1 under π, which is only possible if $S_{in} = x_*$ because x_* is the unique maximum point. The only policy under which $S_{in} = x_*$ with probability 1 is the one that in each state (x, k) assigns probability 1 to the action a such that $(x \oplus a)_k = (x_*)_k$, which is by definition π_{x_*}. □

Because π_{x_*} always chooses `skip` after it reaches x_*, the above theorem also proves that optimal policies must converge to the maximum point x_* when starting from states of the form $(x, 1)$. This is similar to what we did for finite time MDPs on $\mathbb{X} \times \{0, \dots, T\}$, where to be able to state convergence we relaxed our definition by only considering starting states of the form $(x, 0)$. In the FTH case, this is a necessary restriction. Suppose we start from an intermediate state (x', k), where x' is such that its first k bits are not the same as the first bits of x_*. If we start from such a choice of (x', k) the optimal policy cannot ever converge to a maximum point; it will instead converge to some point $\bar{x}$ that maximizes f among the states with the same first k bits as x'. However, in the continuing setting we can actually achieve convergence regardless of the starting state. This is because, if we start from any state (x', k) with $k > 1$, in the first iteration we reach the aforementioned suboptimal state $\bar{x}$, ending up in $(\bar{x}, 1)$ after looping with reward $f(\bar{x})$. From then on the optimal policy has n time steps before the next episodic reward, which is enough to always reach the maximum point x_*; the optimality conditions then force the optimal policy to always `skip` and never leave x_*. This ensures convergence of optimal policies to the maximum point from every starting state in the sense of Definition 2.3 with $N = 2n$, because we are guaranteed to reach a maximum point in the second iteration at the latest.

The reason we added the unique maximum condition is that, if there are two distinct maximum points x_*^1 and x_*^2, there might exist a policy that oscillates between them every n steps.

If this policy exists it is optimal because all the episodic rewards are maximized, but it does not converge to a maximum according to Definition 2.3 if the intermediate points visited when going from one to the other are not optimal as well. For example this can happen if $x_*^1 = \mathbf{0}$, $x_*^2 = \mathbf{1}$ and π is the policy that always chooses to flip the current bit. To solve this issue we can once again relax our definitions of convergence: instead of asking for $\pi_*^t((x,1))$ to be optimal for every $t \geq N$, we ask that there exists some $I \in \mathbb{N}$ such that $\pi_*^{in}((x,1))$ is optimal for every $i \geq I$. This definition allows us to say that optimal policies converge to a maximum even if there are multiple maximum points, when starting from states of the form $(x,1)$. We can generalize this for all states (x,k) by asking that $\pi_*^{in+n-k+1}((x,k))$ is optimal for every $i \geq I$, so that the indexes properly line up with the time steps where episodic reward is received. Furthermore, the presence of multiple maximum points allows the existence of strictly stochastic optimal policies: for example, if two maximum points x_*^1 and x_*^2 differ in the k-th bit the optimal policy is allowed to be stochastic in states of the form (x,k).

Remark 2.10. The MDP in Theorem 2.6 can be seen as a continuous version of the *Linear* game from [1]. In practice, we are taking *Linear* episodes and using the looping effect to staple them together, building a continuing MDP. If instead we use the full action space $\mathcal{A}(x,k) = \mathcal{A}$, the resulting MDP can be seen as a continuous version of *Global*. Convergence to a maximum for this MDP (according to the aforementioned generalized definition of convergence) can be showed with the same proof as Theorem 2.6: the only thing we need to characterize optimality is to be able to reach any point in n time steps, which is of course still true for $\mathcal{A}$. The relaxed definition is necessary because the additional freedom of movement granted by the use of the unrestricted action space $\mathcal{A}$ means that we cannot properly characterize the behavior of optimal policies in the intermediate time steps. We can however ensure that the policy does reach an optimal state in each time step where looping occurs on the second iteration at the latest, because the discount factor penalizes policies that do not by a factor γ^n; this is enough to prove generalized convergence. Still, optimal policies for MDPs with unrestricted actions have a lot of freedom in the way they reach the maximum, and the uniqueness of the optimal policy in Theorem 2.6 does not hold even when there is only one maximum point. Namely, for every permutation $\sigma \in S(n)$ the policy $\pi_{x_*}^{\sigma}$ that flips bits based on x_* in the order $\sigma(1), \dots, \sigma(n)$ is optimal, with π_{x_*} corresponding to the trivial permutation $\sigma(n) = n$. Outside of this, the usual phenomena of random skipping and oscillating with repeated flipping of the same bits may occur, further inflating the set of optimal policies.

This concludes our study of MPDs for discrete maximization of generic functions $f : \mathbb{X} \to \mathbb{R}$. We started from an intuitive model using $\mathbb{X}$ as the space of states and f-based rewards, showed that these naive models are susceptible to local maxima and developed ways to circumvent this using episodic reward. In doing this we rediscovered and formalized finite time horizon MDPs equivalent to the *Linear* and *Global* games from [1] and [2]. We also went a step further and constructed a continuous MDP with the same convergence property by using a looping effect to string up multiple episodes, which is to our knowledge a novel approach. Table 1 summarizes our convergence results, based on running time (Continuing or Finite Time Horizon) and choice of reward function for our MDPs. These convergence results hold for both restricted and unrestricted action sets for each MDP using the same proof, because our choice to start from e_1 allows us to move to $\mathbf{0}$ immediately even with restricted actions.

	Full	Incremental	Episodic
Continuing	✗	✗	✓
FTH	✗	✓	✓

Table 1: Convergence of MDPs for maximization. A check mark means that the optimal policies of the considered MDP converge to a maximum point of f for every $f : \mathbb{X} \to \mathbb{R}$.

We have now exhibited examples of both continuing and finite time horizon MDPs that, if solved, allow us to maximize any function f. Now that we have a functioning model for discrete maximization in both settings, in the next chapter we move to the next step: finding algorithms that take a MDP and compute their optimal policy. To do this we will first introduce the core concepts of policy evaluation and policy improvement and the framework of generalized policy iteration. We will then focus on approximated methods that allow us to deal with the large size of the space $\mathbb{X}$, focusing specifically on linear approximation.

Chapter 3

Dynamic Programming, Tabular Methods and Approximation

In this chapter we will give a brief outlook on a few classical reinforcement learning paradigms, roughly following the same order as Sutton and Barto in [8]. These methods can be used to compute the optimal policy of a given MDP, a step that we took for granted so far but that is not at all trivial to accomplish, as we will see. We start with Dynamic Programming methods ([8], chapter 4), which are always able to compute the optimal policy if we have perfect information on the dynamics of the MDP, and introduce the core concepts of policy evaluation and improvement. We then consider Tabular methods, which work on the assumption that we can store our current estimate of the state or state-action values in a lookup table. Tabular methods, among which we present Monte Carlo and Temporal Difference methods ([8], chapters 5 and 6), are able to learn without perfect knowledge of the environment by using a policy π to generate experience for them to learn from. Among them, we introduce the Sarsa algorithm for on-policy control (optimization), which we chose to adopt for our experiments in Chapter 4. We will then introduce approximation methods ([8], chapter 9) with a particular focus on linear approximation, and study the convergence of the Sarsa algorithm when combined with linear approximation.

3.1 Dynamic Programming

The term Dynamic Programming refers to a wide class of algorithms that can be used to compute the optimal policy of a MDP assuming we have perfect knowledge of all of its components. In the general context of solving a task using Reinforcement Learning, in many cases we do not have access to a full model of the task, but instead we may lack some information such as the full dynamics of the system. In our case, in the last chapter we described MDPs that can successfully find a maximum point of any function as long as we are able to determine their optimal policies, and whose dynamics are completely known. This means that using DP algorithms does achieve our goal of discrete maximization. The main weakness of DP algorithms is that they are computationally very intensive, to the point where they may end up being worse than brute force search of the maximum. However, the theory of dynamic programming serves as a solid foundation to an abundance of RL optimization algorithms, which attempt to reach the same convergence results as DP but in a more numerically efficient way; we will analyze some of these algorithms in the later sections of this chapter.

The main objective of DP is to use the value function as a tool to progressively get closer to the optimal policy. This process can be separated into two components: going from a policy to its value function and then using it to find a new policy whose value function is better than the previous one. Iterating this process should eventually lead us to the optimal value function, since it is an upper bound on all value functions. These two fundamental procedures are called respectively Policy Evaluation and Policy Improvement.

3.1.1 Evaluation and Improvement

Given a stationary policy $\pi \in \Pi$, the problem of Policy Evaluation is the computation of its value function v_π. Computing the value function is not difficult per se, as it is characterized by the Bellman equations (1.5). However, it requires solving a system of $|\mathcal{S}|$ linear equations, which can be very costly or even unfeasible for very large state spaces. Recall however that during the proof of Theorem 1.1 we showed that in discounted MDPs the operator T_π over the space $\mathcal{B}$ of bounded functions over $\mathcal{S}$ is well defined, is a γ-contraction and its only fixed point is v_π. This means that, for every $v_0 \in \mathcal{B}$, iterative application of T_π guarantees convergence to v_π. This method is called *Iterative Policy Evaluation* and its update rule is

$$v_{k+1}(s) = T_\pi v_k(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_k(s') \right].$$

The above method can be implemented using two vectors of size $|\mathcal{S}|$, which store the values of $v_k(s)$ and $v_{k+1}(s)$ at every iteration. However, in most cases policy evaluation is done in-place, so that both old values and new estimates are used to compute the next step. This procedure is known as *bootstrapping*, and it often allows methods to learn much faster. Usually, the evaluation algorithm is made to terminate when updates become smaller than a fixed convergence rate θ, which means that the output is the first vector v_{k+1} such that $\max_{s \in \mathcal{S}} |v_{k+1}(s) - v_k(s)| < \theta$.

Suppose now we have a policy π and that we were able to compute (or approximate) its value function v_π. Policy Improvement is the process of finding a new policy π' who is at least as good as π in terms of value function, which means $v_{\pi'}(s) \geq v_\pi(s) \ \forall \ s \in \mathcal{S}$. Once again, the proof of Theorem 1.1 gives us a hint on how to do this. As we have seen, the optimal policy is the unique fixed point of the optimality operator T_*, which given a function $v \in \mathcal{B}$ and a state $s \in \mathcal{S}$ outputs the value of choosing an action $a \in \mathcal{A}(s)$ greedily with respect to v. If $v = v_\pi$, the optimality operator essentially takes us from v_π to the value of $\pi' \sim \text{greedy}(v_\pi)$. This suggests that a policy that is greedy with respect to v_π will do better than π, which is exactly the improvement we are looking for. As it turns out this is a consequence of a fundamental dynamic programming result, which is usually referred to as *Policy Improvement Theorem*. This theorem is presented in Section 4.2 of [8] with no formal proof, which we provide here, using a similar argument as Theorem 11 in [3].

Theorem 3.1. (Policy Improvement Theorem:) Let M be a finite MDP with discount factor $\gamma \in [0, 1]$ such that M has a finite time horizon T or $\gamma < 1$. For any given policy $\pi \in \Pi$, let π' be a deterministic policy that is greedy with respect to q_π. Then π' is at least as good as π: $v_{\pi'}(s) \geq v_\pi(s)$ for every $s \in \mathcal{S}$.

Proof: Because π' is greedy we have that $q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a)$ for every $s \in \mathcal{S}$.

This means that

$$\begin{aligned} v_\pi(s) &= \sum_{a\in\mathcal{A}} \pi(a|s) q_\pi(s,a) \\ &\leq \sum_{a\in\mathcal{A}} \pi(a|s) \left(\max_{a\in\mathcal{A}} q_\pi(s,a) \right) \\ &= \max_{a\in\mathcal{A}} q_\pi(s,a) \\ &= q_\pi(s,\pi'(s)). \end{aligned} \tag{3.1}$$

To prove $v_{\pi'}(s) \geq v_\pi(s)$ we want to use the value function in (3.1), which depends on π', to estimate $v_{\pi'}$. The first step is the following:

$$\begin{aligned} v_\pi(s) &\overset{(3.1)}{\leq} q_\pi(s,\pi'(s)) \\ &\overset{(1.7)}{=} \sum_{s',r} p(s',r|s,\pi'(s)) \left[r + \gamma v_\pi(s') \right] \\ &= \sum_a \pi'(a|s) \sum_{s',r} p(s',r|s,\pi'(s)) \left[r + \gamma v_\pi(s') \right] \\ &= \mathbb{E}_{\pi'} \left[R_1 + \gamma v_\pi(S_1) | S_0 = s \right]. \end{aligned} \tag{3.2}$$

The last equalities come from the fact that π' is deterministic, meaning that $\pi'(a|s) = 1 \iff a = \pi'(s)$. This means that we can freely introduce a sum over a as it has the same effect as multiplying by one; the resulting equation is exactly $\mathbb{E}_{\pi'} \left[R_{t+1} + \gamma v_\pi(S_t + 1) | S_t = s \right]$, where we chose $t = 0$ for simplicity. From there we can apply (3.2) inside the expectation:

$$\begin{aligned} v_\pi(s) &\leq \mathbb{E}_{\pi'} \left[R_1 + \gamma v_\pi(S_1) | S_0 = s \right] \\ &\overset{3.2}{=} \mathbb{E}_{\pi'} \left[R_1 + \gamma \mathbb{E}_{\pi'} \left[R_2 + \gamma v_\pi(S_2) | S_1 \right] | S_0 = s \right] \\ &= \mathbb{E}_{\pi'} \left[R_1 + \gamma R_2 + \gamma^2 v_\pi(S_2) | S_0 = s \right]. \end{aligned}$$

In the last step we use the tower rule of conditional expectation to cancel the dependence on S_1, which itself depends only on S_0 and π'. If we iterate this procedure N times we have

$$v_\pi(s) \leq \mathbb{E}_{\pi'} \left[R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots + \gamma^{N-1} R_N | S_0 = s \right] + \gamma^N \mathbb{E}_{\pi'} \left[v_\pi(S_N) | S_0 = s \right]. \tag{3.3}$$

We now have two possibilities. If M has a finite time horizon T, rewards and value are defined only before T, which means that for $N = T$ the second term in (3.3) disappears and

$$\begin{aligned} v_\pi(s) &\leq \mathbb{E}_{\pi'} \left[R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots + \gamma^{T-1} R_T | S_0 = s \right] \\ &= \mathbb{E}_{\pi'} \left[G_0 | S_0 = s \right] = v_{\pi'}(s). \end{aligned}$$

If instead $\gamma < 1$, from (3.3) we have

$$v_\pi(s) - \mathbb{E}_{\pi'} \left[R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots + \gamma^{N-1} R_N | S_0 = s \right] \leq \gamma^N \mathbb{E}_{\pi'} \left[v_\pi(S_N) | S_0 = s \right].$$

For $N \to \infty$ the second term converges to $v_\pi(s) - v_{\pi'}(s)$, as $\sum_{i=1}^N \gamma^{i-1} R_i \to G_0$ and $v_{\pi'}(s) = \mathbb{E}_{\pi'} \left[G_0 | S_0 = s \right]$, where the discount factor ensures that the limit holds under the expectation. Furthermore, because $\gamma < 1$ the right-hand side goes to 0 as $\gamma^N \to 0$ and v_π is bounded, and that taking the limit yields $v_\pi(s) - v_{\pi'}(s) \leq 0$ and thus $v_{\pi'}(s) \geq v_\pi(s)$. □

This result extends to non-deterministic greedy policies by introducing a term $\pi'(a|s)$ for every optimal action $a \in \arg\max_{a\in\mathcal{A}} q_\pi(s,a)$, showing that the inequalities hold for each a and recovering the same result as the sum over a simplifies to 1 after taking the expectation under π'. Furthermore, if there exists some state $s \in \mathcal{S}$ such that $v_\pi(s) < q_\pi(s, \pi'(s))$, in the end we will find $v_\pi(s) < v_{\pi'}(s)$ which means that π' is not only at least as good as π but a strict improvement. An interesting corollary of this is the following:

Corollary 3.1. If M is discounted or it has a finite time horizon, every optimal policy of M is greedy with respect to the optimal state action function q_*.

Proof: Let π_* be an optimal policy and assume by contradiction that π_* is not greedy with respect to q_*. This means that here exists a state-action couple $(\bar{s}, \bar{a})$ such that

$$\pi(\bar{a}|\bar{s}) > 0, \qquad\qquad q_*(\bar{s},\bar{a}) < \max_{a\in\mathcal{A}} q_*(\bar{s},a). \tag{3.4}$$

. Let now π' be a policy that is greedy with respect to q_*; we have

$$\begin{aligned} v_*(\bar{s}) &\overset{(1.9)}{=} \sum_{a\in\mathcal{A}(\bar{s})} \pi(a|\bar{s}) q_*(\bar{s},a) \\ &\overset{(3.4)}{<} \sum_{a\in\mathcal{A}(\bar{s})} \pi(a|\bar{s}) \left(\max_{a\in\mathcal{A}} q_*(\bar{s},a)\right) \\ &= \max_{a\in\mathcal{A}} q_*(\bar{s},a) \\ &= q_*(\bar{s},\pi'(\bar{s})) \end{aligned}$$

where the last equality is the definition of greediness for π'. From here, following the Policy Improvement Theorem with $\pi = \pi_*$ yields $v_*(\bar{s}) < v_{\pi'}(\bar{s})$, which contradicts the optimality of π_*. □

Going back to the problem of Policy Improvement, Theorem 3.1 ensures that the policy $\pi' \sim$ greedy(v_π) is better than π. We will refer to the process of improving the current policy π by taking $pi' \sim$ greedy(v_π) as *Greedy Improvement*. After each step of greedy improvement, there are two possible scenarios: either π' is strictly better than π in at least one state, or $v_{\pi'}(s) = v_\pi(s)\ \forall\ s \in \mathcal{S}$. In the latter case we also have $q_{\pi'}(s) = q_\pi(s)\ \forall\ s \in \mathcal{S}$, and so

$$\begin{aligned} v_{\pi'}(s) &= q_{\pi'}(s,\pi'(s)) \\ &= q_\pi(s,\pi'(s)) \\ &= \max_a q_\pi(s,a) \\ &= \max_a \sum_{s',r} p(s',r|s,a)\,[r + \gamma v_\pi(s)] \\ &= \max_a \sum_{s',r} p(s',r|s,a)\,[r + \gamma v_{\pi'}(s)] \end{aligned}$$

where the third equality derives from the fact that $\pi' \sim$ greedy(v_π). We thus have that $v_{\pi'}$ verifies the Bellman Optimality equations (1.10), which means that $v_{\pi'}$ is the optimal value function and π' is an optimal policy. We can thus say that at each step of greedy improvement either strictly improves the current policy or it converges to the optimal policy. Notice that for finite MDPs convergence is guaranteed in a finite amount of steps, as there is a finite number of deterministic (greedy) policies and the condition of improving the value function ensures we always choose a new policy until convergence.

3.1.2 Generalized Policy Iteration

Combining evaluation and improvement allows us to derive an algorithm to compute the optimal policy. Deriving such algorithms is usually referred to in the RL setting as the *control* (or *optimization*) problem. Given some starting policy π_0, we can first apply one step of evaluation to compute v_{π_0}. We can then apply one step of improvement and obtain a new policy $\pi_1 \sim \text{greedy}(v_{\pi_0})$, that is an improvement over π_0 due to the Policy Improvement Theorem. The process then starts again, generating a chain of deterministic policies and value functions that converges to (v_*, π_*) in a finite amount of time as long as the MDP is finite. This procedure of iterative combination of evaluation and improvement is referred to as *Policy Iteration*. In our case, using iterative policy evaluation and greedy improvement gives us an explicit algorithm that we can implement to numerically compute the optimal policy. That being said, the term Policy Iteration does not refer to a single algorithm, but rather a framework of algorithms that operate on the same logic of evaluating value functions and using them to improve the current policy, converging to (v_*, π_*).

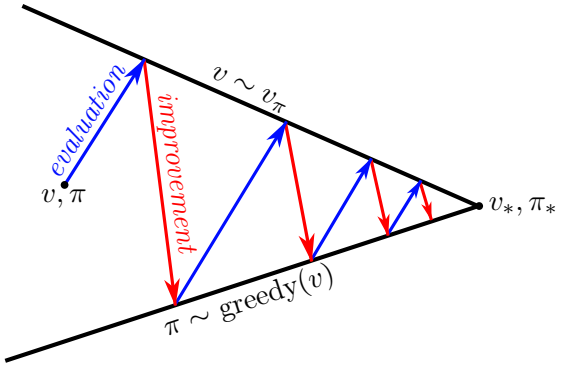


Figure 4: Convergence scheme of Policy Iteration methods.

The above figure visualizes the main idea of Policy Iteration: we have two different goals (computing the value function and improving the current policy) which are distinct but come together to ensure convergence. This is because both processes only come to a halt when the current policy is greedy with respect to its value function, which implies optimality through the Bellman optimality equations. In practice, Policy Iteration can be implemented in a number of ways:

- For the evaluation step, we can apply iterative policy improvement only for a certain amount of steps instead of waiting for convergence. We can also choose to apply the update rule only a random sample of states or a subset of currently available states instead of sweeping through the whole space $\mathcal{S}$, which can be computationally quite advantageous when $\mathcal{S}$ is very large. Methods that avoid repeated sweeps of the whole space $\mathcal{S}$ are sometimes referred to as *Asynchronous Dynamic Programming*.

- For the improvement step, using greedy policies restricts us to deterministic policies, which are not allowed to explore; in some circumstances, it can be very advantageous or even necessary to allow stochasticity in our policies. This can be done in various ways; one of the simplest is to use ε-greedy policies, which act greedily with probability $1 - \varepsilon$ and split the remaining ε probability mass uniformly within $\mathcal{A}(s)$. The set of ε-greedy policies is finite for every finite MDP, and as $\varepsilon \to 0$ all ε-greedy get progressively closer to greedy deterministic policies. Another option is to compute our policy using the

softmax function on the current estimate of the action values. This way every action has nonzero probability, ensuring stochasticity, and actions with higher value are given higher probability.

- Some methods combine the two steps instead of keeping them separate; for example, an alternative to Iterative Policy Evaluation is *Value Iteration*, where the update rule is

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_k(s) \right].$$

  This combines both evaluation (we express v_k using the Bellman equations) and improvement (we pick the greedy action at each step) at each step. Termination occurs for example when the magnitude of updates to v falls under some threshold θ, and the output is a policy $\pi \sim \text{greedy}(v) \approx \pi_*$.

To denote this class of algorithms, which are based on having the evaluation and improvement processes interact without specifying how often, in which order or to which extent, we use the term *Generalized Policy Iteration*. GPI is a flexible framework for a wide range of RL and DP algorithms, and most value-based RL methods can be interpreted through the lens of GPI.

Our last consideration before we conclude our overview of Dynamic Programming methods is efficiency. Generally speaking, DP algorithms converge to the optimal policies in a finite amount of time that is polynomial with respect to the number of states and actions. This by itself is a good result in a vacuum, as it is exponentially faster than inspecting all of the $|\mathcal{S}|^{|\mathcal{A}|}$ possible deterministic policies. However, polynomial performance with respect to $|\mathcal{S}|$ this can still be quite computationally intensive, since the number of states itself tends to grow exponentially depending on the problem (like we have seen in the case of graphs). In practice, DP methods usually achieve convergence much earlier than the worst-case scenario; they are widely employed and often very successful where other methods (such as Linear Programming) fail. Important factors in aiding their performance when the number of states is large are the use of asynchronous DP methods, to avoid costly sweeps over $\mathcal{S}$, and devising efficient heuristics to find good initial values and policies.

3.2 Tabular Methods

One of the key disadvantages of Dynamic Programming is that it requires complete knowledge of the system, meaning that we have a perfect model whose dynamics are known and completely describe our task. In many cases, this is not possible because we lack information on the dynamics of the system; notably, in such cases we cannot directly implement Policy Evaluation. To get around this we want to introduce methods that are able to learn through the *experience* that the agent gets by observing the succession of states, actions and rewards that is generated by interacting with the system. This allows the agent to learn without previous knowledge of the system and still attain optimality.

The main idea is to use the accumulated experience to approximate the value function v_π, or, equivalently, the action value function $q_\pi(s, a)$. This corresponds to the evaluation part of policy iteration, and can thus be used for control (finding the optimal policy) following the GPI framework. The state-action values are usually recorded as a lookup table, giving the moniker of *tabular methods*. Generally speaking, a RL method can be called tabular if it

relies on computing the state (or state-action) values for each state (or state-action couples) and storing them in multi-dimensional arrays so that they can be updated. With this understanding, DP methods are also elements of this class. We will now very briefly describe two important categories of tabular methods: Monte Carlo and Temporal Difference.

Monte Carlo methods are based on generating experience by repeatedly sampling a policy π in order to approximate the value function of an episodic task. Recall that v_π is defined as an expectation under π of the return G_t; this means that if we use π to generate multiple returns, by the law of large numbers the sequence of averages of these sampled returns will converge to the value function:

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t | S_t = s\right] \quad \overset{LLN}{\Longrightarrow} \quad \left(\frac{1}{n}\sum_{i=1}^{n} G_{t,n}\right) \overset{n\to\infty}{\longrightarrow} v_\pi(s).$$

Each return $G_{t,n}$ is obtained by choosing a starting state S_0 and repeatedly sampling π to generate the sequence $(S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T)$ until the episode ends. It is then appended to a list of returns, and our estimate of the value function is updated to the average of all returns. Using the LLN argument, Monte Carlo methods can be shown to always converge to the optimal value under ideal assumptions. Namely, we usually need to assume that each state (or state-action couple) is visited an infinite number of times as the number of episodes n goes to infinity. To ensure that this is the case, we can assume that each state-action couple has a positive probability of being picked as start; this assumption is known as *exploring starts*. Another option is to fix some small parameter $\varepsilon > 0$ and consider only ε-*soft* policies, which are policies $\pi \in \Pi$ such that $\pi(a|s) > \frac{\varepsilon}{|\mathcal{A}(s)|}$ for all $s \in \mathcal{S},\ \forall\ a \in \mathcal{A}(s)$. Notice that ε-greedy policies, which we introduced as a possible option for the improvement step in GPI, are also ε-soft. Assuming that π is ε-soft is enough to guarantee that (at least in the limit) all possible action values can be updated using episodes generated under π.

Despite their theoretical convergence properties, Monte Carlo methods can be rather inflexible in practice. First of all they can be applied exclusively to episodic tasks, as they need to compute the value of the return G_t. This also means that values can only be updated episode by episode and not *online* i.e. time step by time step, which is usually preferable in terms of implementation. Because of these practical limitations, for our numeric experiments we will instead focus on the more flexible paradigm of Temporal Difference methods.

Temporal Difference algorithms implement ideas from both DP and MC methods. Like Monte Carlo they do not need a full model of the problem but instead use experience, learned by sampling a policy π, to update the action values. However, unlike MC, experience is generated time step by time, which allows online value updates and makes TD applicable to continuous tasks as well as episodic ones. Online learning is particularly relevant in the context of implementation using iterative methods. Furthermore, like in the case of policy evaluation with dynamic programming, TD can learn faster by bootstrapping on the action value estimates.

The simplest form of temporal difference learning is $TD(0)$, also called *one-step TD*. Like in Monte Carlo we start with the definition of value function, $v_\pi(s) = \mathbb{E}_\pi\left[G_t | S_t = s\right]$. Recall from Chapter 1 the recursive definition of return, $G_t = R_t + \gamma G_{t+1}$; this means that

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t | S_t = s\right] = \mathbb{E}_\pi\left[R_t + \gamma G_{t+1} | S_t = s\right] \approx \mathbb{E}_\pi\left[R_t | S_t = s\right] + \gamma \mathbb{E}_\pi\left[G_{t+1} | S_{t+1}\right]. \quad (3.5)$$

Given a policy π and a starting state $S_t = s$, we can sample an action a from π, and the system transitions to a new state $S_{t+1} = s'$, with reward r. Notice that the last term in (3.5) corresponds to $v_\pi(s')$, as the dependence on S_t is canceled by the Markov property once S_{t+1} is introduced. This allows us to update our estimate of $v_\pi(s)$ by using r, the discount factor γ and the current estimate of $v_\pi(s')$ (which introduces bootstrapping). $TD(0)$ is the base case, where only one time step is applied; if n steps are taken, meaning that the recursive definition of G_t is applied n times, the corresponding TD method is called n-step TD or $TD(n)$.

TD-based policy evaluation can be employed for optimization using the GPI framework. Given a policy π, we want to sample experience from π to update our estimate of the state-action values q. This differs from (3.5) in that we update state-action values instead of state values, but the reasoning is the same. Updates can be done in an online fashion by fixing a step size parameter $\alpha \in (0, 1]$ and using the incremental form of the $TD(0)$ update for the action-value function:

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma q(S_{t+1}, A_{t+1}) - q(S_t, A_t)\right]. \tag{3.6}$$

For the improvement step we fix a small parameter $\varepsilon > 0$, sometimes called *exploration rate*, and take policies that are ε-greedy with respect to our current estimate. Such policies will be ε-soft, guaranteeing exploration of all space-action couples in the limit. Because at each step the update uses only the values $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$, this algorithm is known as *Sarsa*. Sarsa is a very simple and adaptable algorithm, which has been widely studied and used for control; because of this, we chose Sarsa to test the performance of our discrete maximization MDPs in Chapter 4. The pseudocode for Sarsa in the case of episodic tasks is as follows:

Input: Step size $\alpha \in (0, 1]$, exploration rate $\varepsilon > 0$
Output: Estimate of the optimal action-value function $q \approx q_*$

Initialize $q(s, a)$ for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$, with $q(s, \cdot) = 0$ when s is terminal

for $i = 1, 2, \ldots$ **do**
  Choose a starting state s
  Take a policy π that is ε-greedy with respect to q
  while *s is not terminal* **do**
    Sample an action $a \in A(s)$ from π
    Take action a, receive reward r and transition to state s'
    Sample an action $a' \in A(s')$ from π
    $q(s, a) \leftarrow q(s, a) + \alpha \left[r + \gamma q(s', a') - q(s, a)\right]$
    $s \leftarrow s', a \leftarrow a'$
  end
end

Algorithm 1: Sarsa for episodic tasks

Policy improvement is performed at the start of every episode by taking the ε-greedy policy with respect to our current estimate q. The policy π is then used to generate an episode, and the estimate is updated every step until we reach a terminal state and the episode ends. At the end of each episode we also performed a step of policy evaluation, as q was updated

towards q_π following the $TD(0)$ rule. This procedure can be repeated until convergence (for example when the updates fall under a certain magnitude threshold) or, in an experimental setting, a constant number of times. In the case of continuous tasks we have no need for the inner cycle as there are no terminal states, and policy evaluation can go on until convergence or for a fixed amount of steps.

Like dynamic programming, tabular methods tend to become very expensive in terms of memory and computation. For example, to implement the above pseudocode we need enough memory to record a table of size $|\mathcal{S}| \cdot |\mathcal{A}|$. Furthermore, each value state-action couple (s, a) needs to be visited multiple times in order to properly update $q(s, a)$; this means that, while convergence is theoretically guaranteed, it can in practice be quite slow. Because of these phenomena, when the spaces $\mathcal{A}$ and $\mathcal{A}$ are large we must abandon the idea of computing (or estimating) each possible state or state-action value; we instead turn our attention to approximation methods.

3.3 Approximation Methods

When the space of states is very large, tabular methods are held back by the necessity of computing $v_\pi(s)$ for every s (or, equivalently, $q_\pi(s, a)$ for every (s, a)), which compounds with the base difficulty of computing v_π. A simple way to solve both problems at once is to represent the value function as $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$ using a vector of weights $\mathbf{w} \in \mathbb{R}^d$, which eliminates the need to record all the possible values in a large table. At each time step, instead of updating the value of a state s we update the weight vector $\mathbf{w}$. Because the number of weights is usually much smaller than the number of states ($d \ll |\mathcal{S}|$), this change in weights affects the value of a large number of states in a single iteration. This effect is referred to as *generalization* and it can be a very powerful tool to make learning more efficient, at the cost of interpretability.

A key aspect of generalization is that, inevitably, improving our approximation for a state s can negatively affect the approximation in other states, as opposed to tabular methods where each action value is updated separately and has no immediate effect on the other values. This makes it advantageous to weigh each state s differently based on how much we care about being the approximation being accurate in s. If we introduce a distribution $\mu(s) \geq 0, \sum_{s \in \mathcal{S}} \mu(s) = 1$ over the states we can define the function $\overline{\text{VE}}$, which measures how close $\hat{v}(\cdot, \mathbf{w})$ approximates v_π in terms of mean squared error:

$$\overline{\text{VE}}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2 .$$

If we find a vector $\mathbf{w}_*$ that minimizes $\overline{\text{VE}}(\mathbf{w})$ we reasonably expect $\hat{v}(s, \mathbf{w})$ to be a good approximation for $v_\pi(s)$. One big advantage of using $\overline{\text{VE}}(\mathbf{w})$ to choose the weights of our approximation $\bar{v}(\cdot, \mathbf{w})$ is that the weights can be learned efficiently using stochastic gradient descent. To do so we must assume that states are visited with the same distribution as μ; to ensure this, in continuing tasks μ is the distribution induced by the policy π, whereas for episodic tasks μ is determined both by π and the choice of initial state. Under this assumption we can minimize $\overline{\text{VE}}(\mathbf{w})$ time step by time step by taking a small step of size α in the direction that most reduces the mean squared error, which is given by its gradient

with respect to the weights:

$$\begin{aligned}\mathbf{w}_{t+1} &= \mathbf{w}_t - \frac{1}{2}\alpha\nabla\left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)\right] \\ &= \mathbf{w}_t + \alpha\left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w})\right]\nabla\hat{v}(S_t, \mathbf{w}_t).\end{aligned} \tag{3.7}$$

Of course, minimizing $\overline{\mathrm{VE}}(\mathbf{w})$ with the above iteration is not feasible because we do not have access to the real value function $v_\pi(s)$. In practice, the term $v_\pi(s)$ is replaced with some noisy estimate U_t, usually referred to as the *update target*, that can be computed by interacting with the MDP under the policy π. In order for equation (3.7) to hold for U_t the target must not depend on the current state of the weights, so that it is unaffected by the gradient. Furthermore, if the target is an unbiased estimate of v_π, i.e. $\mathbb{E}_\pi\left[U_t | S_t = s\right] = v_\pi(S_t)$ for every time t, then $\mathbf{w}_t$ converges to a local minimum of $\overline{\mathrm{VE}}(\mathbf{w})$ for decreasing α due to well known results for stochastic approximation. The general form of the weight update is given by

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\left[U_t - \hat{v}(S_t, \mathbf{w})\right]\nabla\hat{v}(S_t, \mathbf{w}_t). \tag{3.8}$$

In most cases it is impossible to prove convergence to a global minimum in a general setting, especially if we choose complex approximators such as neural networks. Without additional knowledge on the task or the chosen method of approximation, showing convergence to a local minimum is usually the best one can hope for.

Different choices of U_t will lead to different iterative methods; for example if $U_t = G_t$ (which is by definition an unbiased noisy estimate of v_π) we obtain the Gradient Monte Carlo algorithm, which can be applied to episodic tasks. Namely, given a policy π we can use it to generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ and iteratively apply the weight update

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\left[G_t - \hat{v}(S_t, \mathbf{w})\right]\nabla\hat{v}(S_t, \mathbf{w}_t) \tag{3.9}$$

for every $t = 0, 1, \ldots, T-1$; this is then repeated for multiple episodes until convergence. Because G_t does not depend on the weights, the iteration (3.9) is a well defined gradient method, and we have guaranteed convergence to a local minimum for decreasing α. Namely, if we let α vary with time, the standard sufficient condition for convergence is

$$\sum_{t=0}^{\infty}\alpha_t = \infty, \quad \sum_{t=0}^{\infty}\alpha_t^2 < \infty \tag{3.10}$$

where α_t is the step size at time t. In the following formulas we will often write α for simplicity, with the understanding that to prove convergence we might have to assume that α does decrease with time.

If the update target U_t bootstraps on $\mathbf{w}_t$, meaning that it uses the current value of $\mathbf{w}_t$ to estimate $v_\pi(S_t)$, the above results no longer hold and we have no guarantee of convergence for the iteration (3.8). Such methods are called *semi-gradient* methods because they take into account the change of $\mathbf{w}_t$ only on the estimate $\bar{v}$ and not on the target U_t, meaning that only part of the gradient is included. While semi-gradient algorithms do not offer the same guarantee of convergence as proper gradient methods, they still converge reliably in a wide range of scenarios. As we mentioned when discussing policy evaluation through dynamic programming and TD, the use of bootstrapping often leads to much faster learning compared to pure gradient methods, making semi-gradient methods a valid choice for value function approximation. A very relevant case in which they are quite successful is the framework of linear approximation, which we will cover in the next section.

3.3.1 Linear Methods

In the context of updating our weights using gradient descent, a particularly convenient choice for our approximation $\hat{v}(\cdot, \mathbf{w})$ is to use a function that is linear with respect to the weight vector $\mathbf{w}$. To implement this, consider a function $\mathbf{x} = (\mathbf{x}_1, \ldots, \mathbf{x}_d) : \mathcal{S} \to \mathbb{R}^d$ that assigns a real valued vector to each state. Each function $\mathbf{x}_i$ is a *feature* of the state s, and $\mathbf{x}(s)$ is referred to as the *feature vector*. We can use the feature vector to approximate the value function using the inner product of the features and the weights:

$$\hat{v}(s, \mathbf{w}) = \mathbf{w} \cdot \mathbf{x}(s) = \sum_{i=1}^{d} \mathbf{w}_i \mathbf{x}_i(s).$$

This method of value function approximation makes gradient descent a natural choice, as the gradient with respect to the weights is simply

$$\nabla \hat{v}(s, \mathbf{w}) = \nabla \left(\mathbf{w} \cdot \mathbf{x}(s) \right) = \mathbf{x}(s),$$

which means that the update formula (3.8) simplifies to

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[U_t - \mathbf{w} \cdot \mathbf{x}(S_t) \right] \mathbf{x}(S_t). \tag{3.11}$$

The simplicity of the weight update is advantageous for both implementation and mathematical analysis. Furthermore, having a linear objective ensures that there there exists a unique $\mathbf{w}_*$ that minimizes $\overline{\text{VE}}(\mathbf{w})$; this ensures that methods that converge to a local minimum will converge to the global minimum. For example, if $U_t = G_t$ we have a version of the Gradient Monte Carlo iteration (3.9) that is guaranteed to converge to the global optimum of $\overline{\text{VE}}(\mathbf{w})$ under the usual GD conditions for α.

The same cannot be said for Temporal Difference methods, because of the presence of bootstrapping in the update; ad-hoc results are required to ensure convergence of the approximated methods. Furthermore, the weight vector they converge to is not the global optimum but instead a point close to a local optimum, the *TD fixed point* $\mathbf{w}_{TD}$, which is uniquely determined by the policy π and the choice of features. Namely, when we replace U_t with the TD update in (3.11) we have

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha \left(R_{t+1} + \gamma \mathbf{w}_t \cdot \mathbf{x}_{t+1} - \mathbf{w}_t \cdot \mathbf{x}_t \right) \mathbf{x}_t \\ &= \mathbf{w}_t + \alpha \left(R_{t+1} \mathbf{x}_t - \left((\mathbf{x}_t - \gamma \mathbf{x}_{t+1}) \cdot \mathbf{w}_t \right) \mathbf{x}_t \right) \end{aligned}$$

where $\mathbf{x}_t = \mathbf{x}(S_t)$. If we take the conditional expectation with respect to w_t on both sides, under a policy π, we have

$$\mathbb{E}_\pi \left[\mathbf{w}_{t+1} | \mathbf{w}_t \right] = \mathbf{w}_t + \alpha \left(\mathbf{b} - \mathbf{A} \mathbf{w}_t \right)$$

where

$$\mathbf{b} = \mathbb{E}_\pi \left[R_{t+1} \mathbf{x}_t \right] \in \mathbb{R}^d, \qquad \mathbf{A} = \mathbb{E}_\pi \left[\left(\mathbf{x}_t - \gamma \mathbf{x}_{t+1} \right) \mathbf{x}_t \right] \in \mathbb{R}^{d \times d}. \tag{3.12}$$

If the algorithm converges we have $\mathbb{E}_\pi \left[\mathbf{w}_{t+1} | \mathbf{w}_t \right] = \mathbf{w}_t$ and thus $0 = \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$; this means that, as long as the matrix $\mathbf{A}$ is invertible, if the weights $\mathbf{w}_t$ converge the only possible limit they can converge to is $\mathbf{w}_{TD} := \mathbf{A}^{-1}\mathbf{b}$. This was shown by Tsitsiklis and Van Roy in [10], where they also show that the approximation performance of $\mathbf{w}_{TD}$ is bounded by

$$\overline{\text{VE}}(\mathbf{w}_{TD}) \leq \frac{1}{1 - \gamma} \overline{\text{VE}}(\mathbf{w}_*)$$

where γ is the discount factor. When γ is close to 1, the expansion factor can be quite large, leading to worse asymptotic performance compared to Monte Carlo. In practice, TD methods have a significant speed advantage over MC; which method is preferable depends on the task and on whether speed or accuracy have the higher priority.

Among TD methods, the Sarsa algorithm has been studied extensively and proven to converge (at least under ideal assumptions) in both episodic and continuous settings. In the next section we will present the algorithm for Sarsa with linear approximation and study its convergence.

3.3.2 Convergence of Linear Sarsa

One of the first works on the convergence of Sarsa with linear approximation is [6] by Perkins & Precup in 2003, where they show the convergence for a version of Sarsa for continuing discounted tasks using a contraction argument. A similar result for continuing tasks was proven by Melo et al. in [4], this time with an ODE approximation argument. In [4] the authors put a bigger focus on the importance and feasibility of each hypothesis, discussing what may go wrong in practical or experimental settings. Lastly, in [5] we have a proof of convergence for episodic tasks, again using a form of contraction argument.

The three aforementioned papers use a similar notation, which we will now introduce. The action-value function $q_\pi(s,a)$ is approximated as $q_\pi(s,a) \approx \phi(s,a) \cdot \mathbf{w}$ using a feature vector $\phi(s,a) \in \mathbb{R}^d$, where d is the length of the weight vector. We denote by Φ the matrix whose rows are the features $\phi(s,a)$ for all possible state-action couples (s,a), so that for every weight vector $\mathbf{w}$ the current estimate of the value function is the matrix $\Phi\mathbf{w}$. In [4] and [5] the weights are denoted as θ_t instead of $\mathbf{w}_t$, but in the following we will keep using $\mathbf{w}$ for consistency. In [6] and [5] ε-soft policies are defined as policies such that $\pi(a|s) \geq \varepsilon \ \forall \ a \in \mathcal{A}(s)$, without the factor $\frac{1}{|\mathcal{A}(s)|}$ in our previous definition. The pseudocode of Algorithm 2 corresponds to the algorithm considered in [6].

In [6], the policy improvement step is performed by a *policy improvement operator* Γ, which takes in the current estimation of the action-value function and outputs a stochastic policy in a way that is continuous with respect to the weights. Let n be the number of states and let m the number of actions. Each estimate of the action values can be represented as a real-valued lookup table with at most $|\mathcal{S}| \cdot |\mathcal{A}|$ entries, i.e. a matrix $Q \in \mathbb{R}^{m \times n}$. Furthermore, policies can also be represented as matrices $P \in \mathbb{R}^{m \times n}$, where $P_{ij} = \pi(j|i)$. We can thus interpret Γ as an operator from $\mathbb{R}^{m \times n}$ to itself. We say that Γ is ε-soft if the policy $\Gamma(Q)$ is ε-soft for every $Q \in \mathbb{R}^{m \times n}$; we can also take Γ so that the outputs are greedy, or ε-greedy, which generalizes the standard improvement step of taking the greedy or ε-greedy policy.

The algorithm can be iterated until the weights converge, or until the updates become small enough. A common choice is to fix a *convergence rate* $\theta > 0$ and say that the weights converge at time t if $\|\mathbf{w}_{t+1} - \mathbf{w}_t\| < \theta$ for some norm; in our experiments, we used the infinity norm $\|x\|_\infty = \max_{i=1}^{d} |x_i|$. For the first convergence result we will need two main assumptions:

> **Assumption 1:** Under any policy π_i, the considered Markov decision process acts as an irreducible, aperiodic Markov chain over $\mathcal{S}$.
>
> **Assumption 2:** The columns of Φ are linearly independent.

Input: Initial policy π_0, step sizes $\alpha \in (0, 1]$, policy improvement operator Γ
Output: Estimate of the optimal policy $\pi \approx \pi_*$

for $i = 0, 1, 2, \ldots$ **do**
  Initialize weights $\mathbf{w}_i \in \mathbb{R}^d$
  Choose a starting state s_0
  Sample a_0 from $\pi_i(\cdot|s_0)$
  Take action a_0, receive reward r_1 and transition to s_1
  Policy Evaluation: Sarsa updates for q_{π_i} with linear approximation
  repeat for $t = 1, 2, \ldots$:
    Sample a_t from $\pi_i(\cdot|s_t)$
    $\Delta_t = r_t + \gamma\phi(s_t, a_t) \cdot \mathbf{w}_i - \phi(s_{t-1}, a_{t-1}) \cdot \mathbf{w}_i$
    $\mathbf{w}_i \leftarrow \mathbf{w}_i + \alpha\Delta_t\phi(s_{t-1}, a_{t-1})$
    Take action a_t, receive reward r_{t+1} and transition to s_{t+1}
  until $\mathbf{w}_i$ *converges*
  Policy Improvement:
  $\pi_{i+1} = \Gamma(\Phi\mathbf{w}_i)$.
end

Algorithm 2: Sarsa with linear approximation ([6], Figure 1)

Assumption 1 is often referred to as *ergodicity*, which is a property of Markov chains. In MDP terms, ergodicity corresponds to asking that each state $s \in \mathcal{S}$ is reachable from any other state while avoiding cyclic behavior; this depends on the dynamics of the MDP, and thus on the task and the way we chose to model it. If the MDP itself has the necessary freedom, Assumption 1 can be verified by taking ε-soft policies at every step, i.e. that Γ is ε-soft, so that we are always able to move from every state to every other state under π. Assumption 2 depends instead on the features chosen for approximation, which is under our control, meaning that when implementing our methods we have to make sure that the chosen features are linearly independent. Linearly independent features are fundamental to ensure that the equivalent of matrix $\mathbf{A}$ in (3.12) is invertible, which guarantees that the algorithm converges to the TD fixed-point.

To prove the convergence theorem we will also need the following Lemma (Lemma 6 in [6]). Given a policy π, let $\mathbf{w}^\pi$ be the TD fixed point that our Sarsa updates converge to.

Lemma 3.1. Let $\varepsilon > 0$ and let $\Pi_\varepsilon = \{\pi \in \Pi : \pi(a|s) \geq \varepsilon \ \forall \ s \in \mathcal{S}, a \in \mathcal{A}(s)\}$ denote the set of ε-soft stochastic policies. For any $\varepsilon > 0$ there exists some constant $c_\varepsilon > 0$ such that $\|\mathbf{w}^{\pi_1} - \mathbf{w}^{\pi_2}\| \leq c_\varepsilon \|\pi_1 - \pi_2\|$ for any $\pi_1, \pi_2 \in \Pi_\varepsilon$, where $\|\cdot\|$ is the euclidean norm on $\mathbb{R}^{n\times m}$.

The proof of the Lemma hinges on characterizing $\mathbf{w}^{\pi_1}$ and $\mathbf{w}^{\pi_2}$ as solutions of a linear system which depends on π and Φ, using the same reasoning as (3.12). Once again, Assumption 2 ensures that the system is invertible and has a unique solution; the remaining passages are simple estimates with constants that depend on the dynamics of the MDP. We are now ready to prove the convergence theorem (Theorem 1 in [6]):

Theorem 3.2. Let M be a continuing discounted MDP that satisfies Assumption 1, let Φ satisfy Assumption 2 and let step sizes $\{\alpha_t\}$ verify the condition (3.10). Then, for every $\varepsilon > 0$ there exists a constant $c > 0$ such that, if the policy improvement operator Γ is ε-soft

and c-Lipschitz continuous with respect to the weights, the sequence of policies generated by Algorithm 2 converges to a unique limiting policy $\pi \in \Pi_\varepsilon$ for M, regardless of the choice of starting policy π_0.

Proof: We want to show that the map $g : \pi \to \Gamma(\Phi \mathbf{w}^\pi)$, which corresponds to a single iteration of Algorithm 2, is a contraction of the space Π_ε. Assume that Γ is c-Lipschitz and let $\pi_1, \pi_2 \in \Pi_\varepsilon$. We have that

$$\begin{aligned}
\|\Gamma(\Phi \mathbf{w}^{\pi_1}) - \Gamma(\Phi \mathbf{w}^{\pi_2})\| &\leq c \, \|\Phi \mathbf{w}^{\pi_1} - \Phi \mathbf{w}^{\pi_2}\| \\
&= c \, \|\Phi(\mathbf{w}^{\pi_1} - \mathbf{w}^{\pi_2})\| \\
&\leq c \, \|\Phi\| \, \|\mathbf{w}^{\pi_1} - \mathbf{w}^{\pi_2}\| \\
&\leq c \, \|\Phi\| \, c_\varepsilon \, \|\pi_1 - \pi_2\|
\end{aligned}$$

where we used that Γ is c-Lipschitz in the first step and Lemma 3.1 for the last step. The map g is a contraction if $c \, \|\Phi\| \, c_\varepsilon < 1$, which gives us the condition $c < \|\Phi\|^{-1} \, c_\varepsilon^{-1}$ (where Assumption 2 guarantees that $\|\Phi\| > 0$, so we can invert it). When this condition holds, the Banach fixed-point theorem ensures that g has a unique fixed point; because g is a single step of our algorithm, we conclude that the sequence of policies generated by the algorithm converges to that fixed point. □

The value of c essentially dictates how much the improvement operator Γ can exploit the information contained in the action-values. For any given Q, the best possible policy for the improvement step would be the one that is greedy with respect to Q; however, taking the greedy policy is not a continuous operation, and we need Γ to be both Lipschitz continuous and ε-soft. Larger values of c will make this restriction not as impactful, allowing us to get policies closer to the optimal greedy ones, but may not guarantee convergence because of the contraction requirement.

In [5], Lina Palmorg provides a similar proof for undiscounted episodic MDPs, using a slightly different version of Linear Sarsa where the weights updates are accumulated and then applied at the end of each episode. Let $H_t = (S_0^t, A_0^t, R_1^t, S_1^t, \ldots, R_{\mathcal{T}(t)}^t, S_{\mathcal{T}(t)}^t)$ be the t-eth generated episode, with termination time $\mathcal{T}(t)$. The weight update rule can be written as

$$\begin{aligned}
&\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \mathcal{H}(\mathbf{w}_t, H_{t+1}), \\
&\mathcal{H}(\mathbf{w}_t, H_{t+1}) = \sum_{s=0}^{\mathcal{T}(t+1)-1} \left(R_{s+1}^{t+1} + \phi_{s+1}^{t+1} \cdot \mathbf{w}_t - \phi_s^{t+1} \cdot \mathbf{w}_t\right) \phi_s^{t+1}
\end{aligned}$$

where ϕ^t denotes the linear approximation features at time t. While the proof in the episodic case is more mathematically involved, partly because of the lack of discounting and the additional randomness in termination time and choice of initial state, the core ideas remain the same. Namely, we must assume that the policies generated by the algorithm are ε-soft and depend on the weights in a Lipschitz-continuous way. The resulting convergence theorem (Theorem 4.1 in [5]) has the same statement as Theorem 3.2.

In [4], Melo et al. study the convergence of TD methods when combined with linear approximation, in a more general setting with the possibility of a non-discrete (but compact) set of states. In the first part it studies the convergence of *Q-learning*, a TD method that

differs from Sarsa in that it tries to approximate the optimal action value function q_* at each step instead of q_π. The Q-learning update takes on the form

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) - q(S_t, A_t) \right].$$

This differs from the Sarsa update (3.6) in that at each step we consider the best possible action in terms of our current estimate of the action-value function, which allows to directly estimate q_*; the policy π is still used to sample the trajectories of the MDP. Methods that approximate the value function of the same policy π used to sample trajectories are called *on-policy*, whereas methods that sample a different value function from π are called *off-policy*. Sarsa is an on-policy temporal difference method, and Q-learning is an off-policy temporal difference method.

The fact that Q-learning directly estimates the optimal action-value function q_* can allow for quicker convergence, as we directly move towards q_* at each step instead of relying on the GPI framework. However, each update can be computationally much heavier compared to Sarsa, which means that the algorithm may not necessarily take less physical time to converge. Furthermore, we have to take into account the interaction with linear approximation, which is what Melo at al. do in Section 3.1. Under the usual assumptions on α_t, they show that the linear algorithm with the Q-learning update converges to a unique limiting weight vector, using an ODE argument. However, they also find that convergence significantly depends on the discount rate γ and the choice of initial policy π_0. Namely, when γ is close to 1, for the algorithm to converge to the optimal policy π_* we must assume that π_0 is already relatively close to π_*; in general, the algorithm will converge to a local optimum that is close to the starting policy. This makes Q-learning with linear approximation behave more like an on-policy algorithm, eliminating all the advantages of directly estimating q_*. This is one of the reasons for our choice of Sarsa among TD methods for our numerical experimentation.

In Section 3.2 Melo et al. derive a convergence result for Sarsa with linear approximation for continuing tasks, again with similar assumptions and statement as Theorem 3.2, using an ODE argument instead of the contraction argument of Perkins & Precup. They also comment on the difficulty of implementing a linear method with a decaying exploration rate $\varepsilon \to 0$; this eventually makes it so that the learned policy is greedy with respect to the current weights, which is not a continuous operation and thus does not verify the hypotheses of Theorem 3.2.

This concludes our overview of convergence results for the Sarsa algorithm. In the next chapter we will focus entirely on numerical experimentation on discrete maximization of functions over the space of binary strings, using Sarsa with linear approximation to test the MDPs we studied in Chapter 2.

Chapter 4

Computational results

We conclude our work by performing some numerical experiments on the MDPs we introduced in Chapter 2. Using the theory of Chapter 1 we discarded a few inadequate models and eventually devised three MDPs whose optimal policies always lead to a maximum point of the function f. What remains to be determined is whether or not we can use these MDPs to maximize a function in a real life setting. In this chapter we will do exactly this, testing our MDPs using the Sarsa algorithm with linear approximation. We will comment on our modeling choices as well as our experimental choices, and study how each component of the implemented algorithms affects performance. The code used for our experiments is available in our GitHub repository, with scripts and pre-run Python notebooks for all of our figures.

4.1 Theoretical setting

In this section we will explain the various theoretical and modeling reasons for our choices in the experimental phase. Recall that, in Chapter 2, we defined three functional MDPs for discrete maximization: two finite time horizon MDPs, respectively with incremental and episodic reward function, and one continuous MDP with episodic reward function. The two FTH models are both defined on the space of states $\mathcal{S} = \mathbb{X} \times \{0, \dots, T\}$, where T is the time horizon and $\mathbb{X} = \{0,1\}^n$ is the space of binary strings of length n. The second component records the passing of time, and is necessary to force stationarity, so that there exists an optimal stationary policy. The continuous model is instead defined on $\mathcal{S} = \mathbb{X} \times \{1, \dots, n\}$, again with the passing of time recorded in the second coordinate, and implements episodic reward by having reward 0 for transitions before n and reward f when looping back from n to 1.

In Chapter 2 we mentioned how a large time horizon T gives optimal policies a lot of freedom, which may result in suboptimal behaviors such as oscillation or reaching the maximum point late instead of doing so as soon as possible, which would be the ideal. To solve this issue we proposed two countermeasures. The first is to restrict the time horizon to $T = n$, so that policies do not have "free time" to do whatever they want. This also allows us to represent all tasks on the same space of states $\mathcal{S} = \mathbb{X} \times \{1, \dots, n\}$ with looping, stopping after n steps for the FTH versions (which happens exactly after the first loop). The second countermeasure is restricting the set of actions so that at every time $t = 1, 2, \dots, n$ we are only able to flip the bit in position k or skip it. This forces policies to choose carefully at each step, as they will not be able to go back and undo their errors before the end of the iteration. We will test our MDPs with and without restricted actions to see if the added rigor aids convergence when compared to the larger freedom provided by having access to every bit at every time.

The MDPs will be tested by their performance on discrete maximization by using the Sarsa algorithm with linear approximation. In order to do so, we must define what the features $\phi(s, a)$ are. In our case, the states are of the form $(x, k) \in \mathbb{X} \times \{1, \ldots, n\}$; we want to make sure that the linear features capture information from both the string x and the time t, so that the algorithm can properly take into account the temporal information. To do so, we implemented $\phi((x, k), a)$ as a vector of length $2n$:

$$\phi((x, k), a) := (x \oplus a, e_k) \in \mathbb{R}^{2n}.$$

The first n entries correspond to the string $x \oplus a$, and the remaining n entries represent the time k with one-hot encoding: the string e_k is 1 in position k and 0 everywhere else. The use of one-hot encoding to represent time is used by Wagner in [11] when defining the inputs of his neural network-based method. In the context of our findings on FTH tasks at the end of Chapter 1, this procedure is important to ensure that the algorithm can access time information in a way that induces stationarity. Furthermore, in a experimental setting, doubling the size of the feature vector compared to the string size n gives our methods more weights to work with, which can speed up and enhance their learning while still using a low amount of weights compared to the size 2^n of the space $\mathbb{X}$.

This choice of linear features interacts in a peculiar way with episodic MDPs with restricted actions. Recall that restricting our actions means that for every state $s = (x, k)$ the actions $a \in \mathcal{A}(s)$ depend on the time k, and the only two possibilities are $\texttt{flip}_k$ and `skip`. Episodic MDPs are implemented in our code by starting from $(x, 1)$, where x is a string that is randomly sampled from the uniform distribution over $\mathbb{X}$ at every iteration. During the first time step the feature vector will be $\phi((x, 1), a) = (x \oplus a, e_1)$ where $a \in \{\texttt{flip}_1, \texttt{skip}\}$. The string has exactly one bit, the first one, that is determined by a; the rest of them are the same bits from the randomly chosen string, because they cannot be changed from any of the available actions. In general, for every state (x, k) will have a feature vector with k bits that were able to be changed during the current iteration and $n - k$ bits of essentially random noise. Once the episode ends a new random string is generated, the iteration starts anew and once again random noise is inserted in our Sarsa updates. This random noise can potentially hurt the convergence of our methods; to avoid this we can slightly modify our choice of ϕ and instead use *masked* linear features

$$\psi((x, k), a) = ((x \oplus a)_1, (x \oplus a)_2, \ldots, (x \oplus a)_k, 0, \ldots, 0, e_k).$$

The masked features ψ keep the first k bits of $x \oplus a$ and set the remaining ones to zero, blocking out the random bits from episode initialization while keeping all of the bits that were able to be affected within the iteration. Of course, this reasoning only applies to MDPs with restricted actions; if actions are not restricted, every bit can be changed in any index regardless of the time component k. Because of this, we have three possible configurations for each of our MDPs: unmasked features with restricted actions (`Unmasked_Restricted` or `UM_R`), masked features with restricted actions (`Masked_Restricted` or `M_R`) and unrestricted actions (`Unrestricted` or `UR`). While we introduced masking specifically for episodic tasks, as part of our experimentation we also implemented the option of masking for continuous MDPs, to see if it can have a positive effect on methods beyond its intended use case. It is reasonable to expect that masking will lead to worse results for continuing methods, as from the second iteration onwards each bit has been eligible for flipping at least once and thus carries valuable information.

Lastly, as part of our experiments we will consider a third classical setting for continuing MDPs, that differs from both the undiscounted episodic and discounted continuing cases: the *Average Reward* (AR) setting. Up until now, given a policy π, the main objective of our methods was to maximize the value function $v_\pi = \mathbb{E}_\pi[G_t]$; the return G_t heavily depends on the discount factor γ, which is fundamental in the case of continuing tasks. This means that, in order to solve continuing tasks, we have to accept that rewards will be weighted differently based on time. The average reward formulation bypasses this problem by rewriting return in terms of a new quantity that depends on π, the *average reward*:

$$r(\pi) := \lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T} \mathbb{E}\left[R_t | S_0, (A_i)_{i=1}^{t-1} \sim \pi\right].$$

The main idea of the average reward criterion is to let a MDP go on for T steps by sampling trajectories with a policy π and observe the undiscounted sum of rewards, like we would do for a finite time horizon task, but then take the limit for $T \to \infty$, enforcing the continuing structure. This limit is guaranteed to exist under the same ergodicity assumption of section 3.3.2. In the average reward setting, the return G_t is defined as the sum of differences between rewards and the average reward:

$$G_t := (R_{t+1} - r(\pi)) + (R_{t+2} - r(\pi)) + (R_{t+3} - r(\pi)) + \dots$$

The above expression is known as *differential* return, which can be used to define the value and action-value functions in the average reward setting. Notice that this form of G_t satisfies the recursive definition $G_t = R_{t+1} - r(\pi) + G_{t+1}$; this allows us to derive a version of Sarsa for the average reward case like we did starting from expression (3.5). The full pseudocode for the tabular version of average reward Sarsa can be found in chapter 10.3 of [8]. For our experiments, the update for Sarsa with linear approximation in the average reward setting (using the same notation as Algorithm 2) takes on the form

$$\begin{aligned}
&\Delta_t = r_t - R + \phi(s_t, a_t) \cdot \mathbf{w}_i - \phi(s_{t-1}, a_{t-1}) \cdot \mathbf{w}_i \\
&R \leftarrow R + \beta \Delta_t \\
&\mathbf{w}_i \leftarrow \mathbf{w}_i + \alpha \Delta_t \phi(s_{t-1}, a_{t-1})
\end{aligned}$$

where R is the current estimate of the average reward $r(\pi)$. This estimate gets updated at each iteration by using a step size parameter $\beta > 0$, which is a hyperparameter of the algorithm. For our experiments we will approach continuous MDPs with both the classical and average reward settings.

4.2 Experimental setup

In this section we will explain our experimental choices and describe how our tests were conducted. Let us first summarize the main takeaways from section 4.1:

- Each of our MDPs has the same space of states $\mathcal{S} = \{0, 1\}^n \times \{1, \dots, n\}$. The MDP can only move from $(\cdot, k)$ to $(\cdot, k+1)$ for $k < n$, and for $k = n$ it loops back to $(\cdot, 1)$.
- We have four total MDP settings: FTH with $T = n$ and episodic reward (`Episodic`) or incremental reward (`Incremental`), continuing in the standard setting (`Continuing`) and continuing in the average reward setting (`Average_Reward`).

- The linear approximation features are of the form $\phi((x,k),a) = (x \oplus a, e_k)$ with the option of restricting the set of actions and, in that case, of using masked features, This creates three possible configurations: `Unmasked_Restricted`, `Masked_Restricted` and `Unrestricted`.

This gives us a total of twelve possible combinations. In terms of hyperparameters, the first one we have to fix is the string length n. Once n is fixed, the Sarsa algorithm with linear approximation needs three parameters: the step size α, the exploration rate ε and the convergence rate θ. Furthermore, in the classical setting we have the discount factor γ, and in the average reward setting we have the step size β. Finally we fix an iteration cap parameter `max_it`, which forcibly stops the algorithm after `max_it` iterations even if the weights did not converge.

The experiments will be conducted in the following way. Once the MDP is fixed, each run starts by generating an objective vector $\mathbf{u} \in \mathbb{R}^n$ by sampling a centered normal distribution with standard deviation $\frac{1}{2n}$. The objective function $f : \mathbb{X} \to \mathbb{R}$ is the dot product $f(x) = x \cdot \mathbf{u}$. The method then initializes the weight vector $\mathbf{w} \in \mathbb{R}^{2n}$, from the same distribution as u, and the starting state $(x, 1)$ where the string x is sampled uniformly. From there the Sarsa iteration starts; each iteration is made of n time steps, which is one episode for the FTH methods and one loop with episodic reward for the continuing methods. Actions are sampled using the ε-greedy policy with respect to the current weights, which changes after every time step, which is the same method as the one described by Melo et al. in [4]. After each full iteration a convergence check is made; the algorithm stops if $\|\mathbf{w} - \mathbf{w}_0\|_\infty < \theta$, where $\mathbf{w}_0$ is the weight vector from the last iteration, or if the iteration cap `max_it` reached. Because of this, the maximum number of weight updates is $n \cdot$ `max_it`. If convergence is not reached, in the FTH case the MDP is reset to a new random string; for continuing MDPs the starting string is only generated once at the beginning. In both cases, $\mathbf{w}_0$ gets assigned the new value $\mathbf{w}$ and a new iteration begins. After the method converges or reaches the iteration cap it output the weights $\bar{\mathbf{w}}$, and the final estimate of the optimal policy is the policy $\bar{\pi} \sim \text{greedy}(\bar{\mathbf{w}})$. As we have seen in chapter 2, an optimal policy leads to the maximum point after at most n steps when starting from any state $(\cdot, 1)$. To compute the maximum point found by $\bar{\pi}$ we sample a random string x, let the MDP start from $(x, 1)$ and follow $\bar{\pi}$ for n steps; this determines a string $\bar{x}$, which is our candidate maximum point.

In our algorithms policy improvement is performed at every time step by taking the ε-greedy policy with respect to the estimate of the action-value function given by the current weights. The corresponding operator Γ is ε-soft (up to a constant depending on the chosen definition of ε-soft) but not necessarily Lipschitz-continuous, even if ε is fixed and not decaying. This is a notorious problem in the literature, and was touched by Sutton and Barto, Melo et al. and many others. Taking ε-greedy policies is the main way to improve our policy, but because it is not continuous this prevents us from having guaranteed convergence. Conversely, if we choose Γ to be c-Lipschitz-continuous where c is small enough so that we are guaranteed to converge, the constant c limits how much Γ can improve on the current policy, weakening the improvement step. For our experiments we chose to have stronger improvement steps, with the belief that in our context learning an almost perfect policy quickly is better than having to wait for very slow convergence. This is especially true in the context of disproving a conjecture, as converging quickly to a counterexample has the same result as finding the actual maximum while taking much less time and resources.

Our choice of using a normal distribution with very low variance was based on avoiding numerical divergence of the iterative method, which is a very relevant problem as the number of updates is very high. Standard deviation $\frac{1}{2n}$ was chosen after some preliminary testing, and allowed us to avoid numerical divergence in all of the following experiments.

Our choice of using a linear objective function may raise some questions, as maximizing a linear function over $\{0,1\}^n$ whose coefficients are known is trivial. What we want to implement using RL is a degree 0 optimizer over the finite set $\{0,1\}^n$; because of the No Free Lunch Theorem, we know for certain that no algorithm can perform better than brute force search. What we can hope for in practice is that our choice of features can efficiently capture the underlying structure of the problem and thus enable faster optimization for the specific task. In our case, using a linear objective function allows our linear features to be well suited for our task by construction, putting us in a setting where we can hope to achieve much better performance than brute force methods. Furthermore, using a dot product as our objective function f has the important upside of allowing us to immediately determine the unique maximum point of f. Given our objective vector $\mathbf{u}$, the unique maximum point of f over $\mathbb{X}$ is $x_{\mathbf{u}}$, where

$$(x_{\mathbf{u}})_i = \begin{cases} 1 & \text{if } \mathbf{u}_i > 0, \\ 0 & \text{otherwise.} \end{cases}$$

This means that after our algorithms output a candidate $\bar{x}$ we can directly compute their performance by checking how close they got to the actual maximum point $x_{\mathbf{u}}$. To do so, during testing we kept track of the following four metrics:

- The absolute difference $\texttt{a_d} = f(x_{\mathbf{u}}) - f(\bar{x})$;
- The relative difference $\texttt{r_d} = \frac{f(x_{\mathbf{u}}) - f(\bar{x})}{f(x_{\mathbf{u}})}$;
- The number of wrong bits $\texttt{w_b} = |\{i = 1, \ldots, n | \bar{x}_i \neq (x_{\mathbf{u}})_i\}|$;
- The number of iterations `n_it` that the algorithm took to converge.

The absolute difference is always non-negative by definition of $x_{\mathbf{u}}$, and is zero if and only if the method successfully found the maximum point. Because of our initialization choices, the objective vector's components are very close to 0; this means that `a_d` is always a small number, making it hard to compare methods at a glance by looking solely at `a_d`. The relative difference `r_d` remedies this scaling issue dividing by the actual maximum value $f(x_{\mathbf{u}})$. This value is always greater than zero and is exactly zero only if all components of $\mathbf{u}$ are negative, which only happens with probability 2^{-n}; in that case, `r_d` is set as `NaN`. If $f(\bar{x}) \geq 0$, `r_d` is a percentage: for example, if $\texttt{r_d} \leq 0.02$ we will say that our method achieved performance within 2% of the actual maximum. This allows us to compare methods to each other in a fast and intuitive way. Another possible criterion is the number of wrong bits `w_b`, which is however is also not the best indicator in terms of the actual values of f. This is because the influence each bit has on the value is not constant among bits and instead depends on the magnitude of the associated weight. Because of these considerations, relative difference will be our main metric to measure performance. Notice that every one of the first three metrics is zero if an only if the true maximum point is reached. Finally, the number of iterations `n_it` was recorded to see which methods converge before the iteration cap, and so that among methods with similar performance (which is still the main priority) ones that converge faster can be favored.

4.3 Results

Hyperparameter Grid Search

Our first experiment was a simple grid search on the hyperparameters $\alpha, \varepsilon, \theta$ and γ or β. Each hyperparameter combination was tested on five different random seeds and all resulting four metrics were then averaged between seeds. The first objective of this experiment is to compare all of our twelve methods, so that we can reduce our scope and focus on the best performing among them. This also allows us to make broader considerations on the pros and cons of each MDP and action/feature setting. The second objective is to determine the best hyperparameter combination for each method, which will be used in our next experiments on the best methods. We used the following grids:

```
EIC_grid = [(theta, alpha, gamma, epsilon)
    for theta in [1e-6, 5e-6, 1e-5, 5e-5, 1e-4]
    for alpha in [0.0001, 0.0005, 0.001, 0.005, 0.01]
    for gamma in [0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99, 1]
    for epsilon in [0.01, 0.02, 0.05, 0.1, 0.2]]

AR_grid = [(theta, alpha, beta, epsilon)
    for theta in [1e-6, 5e-6, 1e-5, 5e-5, 1e-4]
    for alpha in [0.0001, 0.0005, 0.001, 0.005, 0.01]
    for beta in [0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5]
    for epsilon in [0.01, 0.02, 0.05, 0.1, 0.2]]
```

The first grid is used for `Episodic`, `Incremental` and `Continuing`, with the option of $\gamma = 1$ for episodic tasks (which were also tested with discounting), and the second one is used for `Average_Reward`. Both grids have a total of $5 \times 5 \times 8 \times 5 = 1000$ possible hyperparameter combinations. The iteration cap was set as `n_it` $= 50000$. As for the string length n we chose $n = 55$; this choice was inspired by the experiments of Angileri et al. in [2] on Brouwer's conjecture on the space of graphs with $k = 11$ nodes, which can be represented as strings of length $\frac{11(11-1)}{2} = 55$. The main results of our hyperparameter grid search are the following:

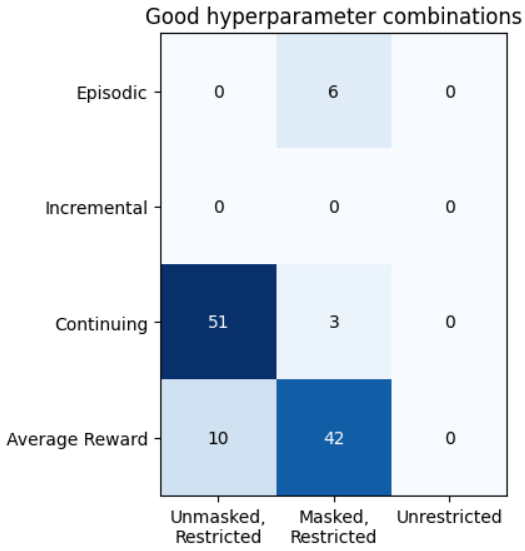


Figure 5: Number of hyperparameter combinations that performed within 1% of the maximum.

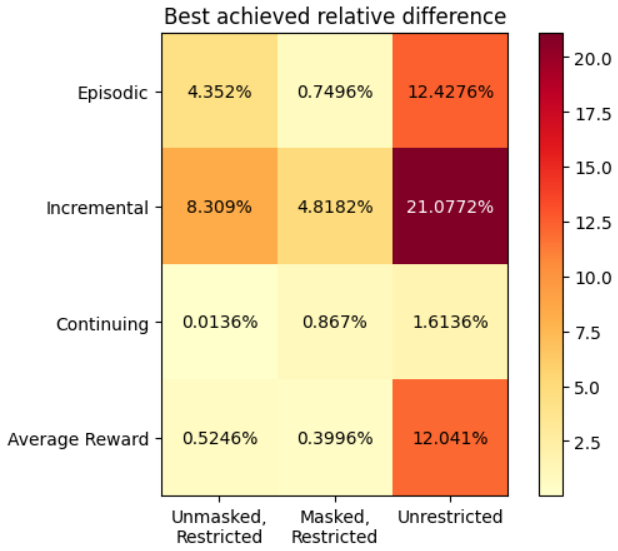


Figure 6: Relative difference from the maximum of the best performing hyperparameter combination found.

Each row in figures 5 and 6 corresponds to a MDP setting, and each column corresponds to an action/feature setting. In figure 5 we have the number of hyperparameter combinations for which the value of the point $\bar{x}$ located by the method is within 1% of the value of actual maximum. If a method has 0 such parameters it means that no hyperparameter combination was good enough to get within 1% of the optimal value; we expect methods for which many hyperparameter combinations achieved good performance to be better. Figure 6 complements figure 5 in that it reports the relative difference between the actual maximum and the point $\bar{x}$ located by the best performing hyperparameter combo for each method. A 0 in figure 5 corresponds to values larger than 1 in figure 6 and vice versa.

Figure 6 paints a rather clear picture of which MDP and action settings do better, and which options generally help or hinder performance. Namely, `Incremental` methods perform at least 4% worse than the `Episodic` methods for every action setting, and generally do much worse than the continuing methods. Among all MDP settings `Continuing` is overall the best, as in the `Unmasked_Restricted` setting it achieves by far the lowest relative difference of all methods. It is also the only MPD setting which does not achieve very poor results in the `Unrestricted` setting, which is generally a much worse option compared to the two restricted options for every MDP setting. As we expected, using masked features has a noticeable positive effect on the performance of both FTH methods, and allows `Episodic` to clear the 1% threshold. In the case of `Continuing` masking actually lowers performance, which is especially noticeable from the difference in of good parameters in figure 5 compared to the unmasked setting. `Average_Reward` MPDs have overall quite good results in settings with restricted actions, with masking leading to a slight improvement and noticeably increasing the amount of good hyperparameter combinations. Lastly, as one can see from figure 6 the 1% threshold for figure 5 is not arbitrary, as all but one of the methods are either comfortably under it or noticeably above it.

The effect of masking on `Episodic`, `Incremental` and `Continuing` matches with our expectations: FTH methods improve their learning by eliminating random noise, while in continuing methods masking may lead to loss of information from the second iteration onwards. Because of this, the fact that masking seems to have a positive effect on `Average_Reward` is unexpected and quite interesting. Based on the results of figures 5 and 6 we selected the four best performing methods for further testing, with their respective optimal hyperparameters:

```
Episodic_M_R : (1e-06, 0.01, 0.9, 0.2)
Continuing_UM_R : (1e-06, 0.0005, 0.8, 0.2)
Average_Reward_UM_R : (1e-06, 0.001, 0.0001, 0.2)
Average_Reward_M_R : (1e-06, 0.005, 0.0005, 0.2)
```

Each of the above hyperparameter sets achieves the best recorded relative difference of figure 6 for its own method. Two additional candidates we considered are the masked, restricted version of `Continuing`, whose performance is not too far off from `Episodic_M_R`, and the unrestricted version of `Continuing`, which notably manages to get within 2% of the actual maximum and is the only unrestricted option with decent results. However, this felt superfluous as `Continuing_UM_R` outperforms all the other methods by more than an order of magnitude. Furthermore, it is the only method that managed to converge before the iteration cap: the runs with the optimal hyperparameters `(1e-06, 0.0005, 0.8, 0.2)` had an average number of iterations of 33371.8, and the majority of its 51 good hyperparameter combinations from figure 5 converged in less than 40000 iterations.

As for the effect of hyperparameters on performance, all the good hyperparameter combinations of figure 5 had $\theta \leq$ `1e-5`, and most methods achieve better results with the lowest value $\theta =$ `1e-6`, which is to be expected. None of the methods except `Continuing_UM_R` managed to converge before `max_it` iterations, even with the smallest possible θ. In terms of the other hyperparameters $\alpha, \varepsilon, \gamma$ and β, all methods that achieved good results (mostly `Continuing_UM_R`, `Average_Reward_M_R` and less so `Average_Reward_M_R`) managed to do so with a wide range of options for each of them, with some interesting trends:

- There is a preference towards higher exploration rates: for all hyperparameter combinations that perform within 1% of the maximum we have $\varepsilon \geq 0.05$, and the average reward settings have an even stronger preference, with the majority of good results having the maximal value $\varepsilon = 0.2$.
- `Continuing_UM_R` has good results for most values of γ, with a slight preference for higher values ($\gamma = 0.9, 0.95$) but notably excluding the extremes $\gamma = 0.99, 1$. It also has a preference towards the higher options of the learning rate α, with good results for every value except the lowest, $\alpha = 0.0001$.
- Average reward methods share similar preferences for α, but are much more flexible in terms of β: we have good results for every available choice, from $\beta = 0.0001$ to $\beta = 0.5$, with comparable performance across the line. This suggests that average rewards methods are quite stable with respect to changes in β.

The full list of around one hundred good hyperparameter combinations from figure 5 is available on our GitHub repository.

Remark 4.1. Within the code and in the notebook there are results for one additional MDP setting, called Monotonic, which we briefly described in Remark 2.7. They are FTH tasks on $\mathbb{X} \times \{1, \dots, n\}$ which always start from the empty string $\mathbf{0}$ and where the only available actions are setting a bit to 1 or skipping; because there is no option of flipping bits from 1 to 0 this forces monotonicity in the policies. During testing we implemented a version with episodic and incremental reward functions, but the results were quite poor, with a best relative difference score of 2.9% and over 31% respectively. Because of their poor performance, the fact that monotonic actions cannot be implemented for continuing tasks and the fact that we did not study them in detail during Chapter 2 we chose to exclude them from our experiments.

Multi-seed validation

Our second experiment was to take the four best methods from the grid search and further study their performance by testing them on a large number of random seeds, in order to estimate the actual distribution of their relative difference. The tests were done on 1000 random seeds using the optimal hyperparameters found by the grid search. The performance metrics are the same as the first experiment, with the addition of the `exact` column which corresponds to the instances within the 1000 seeds for which each method successfully found the maximum point. We also added the `it_maxed` columns, which tells us for how many seeds each method had to reach the iteration cap. The main results of our experiments are summarized in Table 2.

	r_d_mean	w_b_mean	exact	it_maxed
Episodic_M_R	0.957%	2.404	91	1000
Continuing_UM_R	0.34%	1.351	285	3
Average_Reward_UM_R	0.682%	2.013	149	1000
Average_Reward_M_R	0.866%	2.211	123	1000

Table 2: Multi-seed validation results. The mean values `w_b_mean` and `w_b_mean` are computed over 1000 seeds.

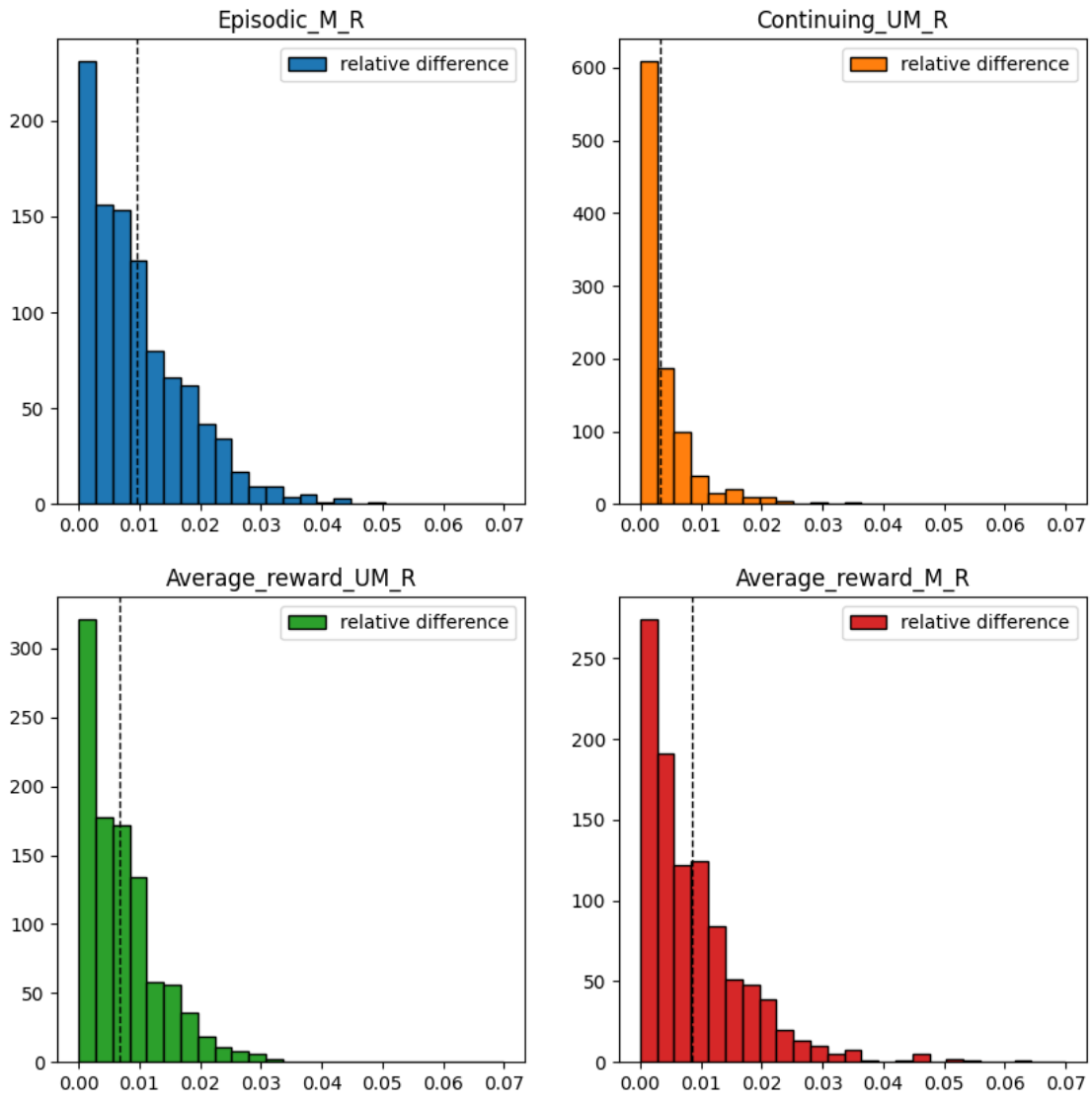


Figure 7: Histograms of the relative difference values for each method and seed. The dotted lines represent the mean.

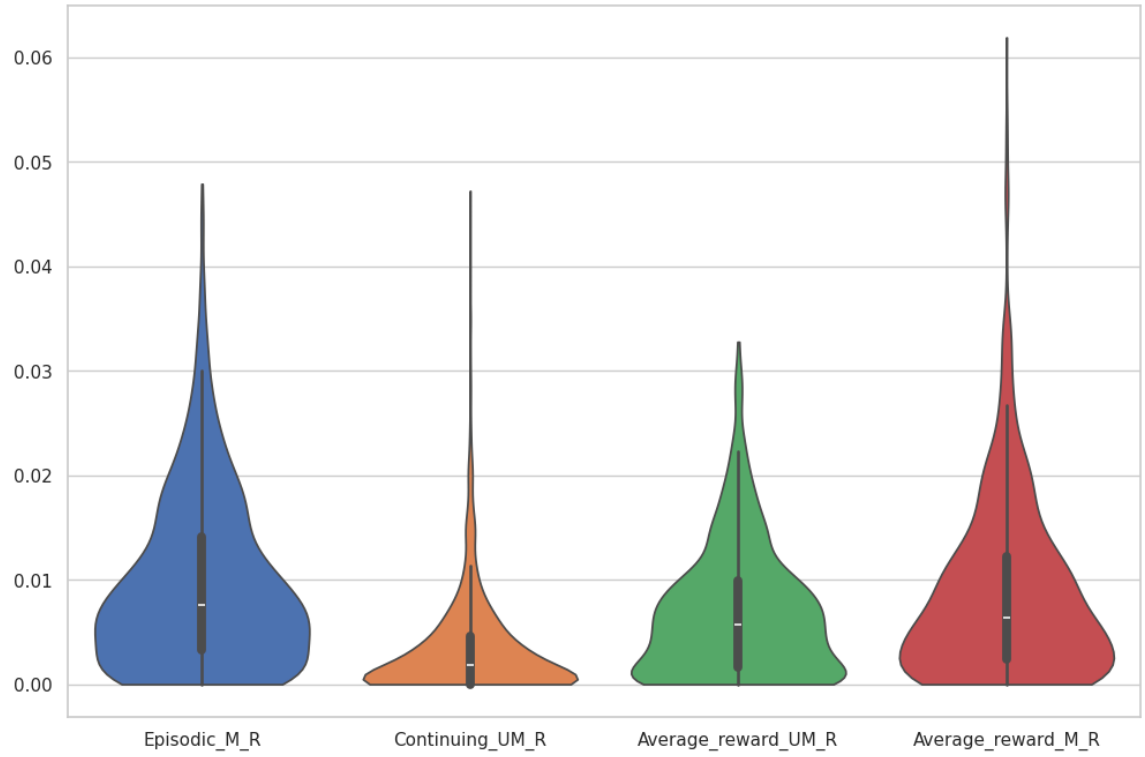


Figure 8: Violin plots of the relative difference.

The results more or less coincide with our first impressions based on the grid search experiment. `Continuing_UM_R` is by far the best method, getting twice as close to the actual maximum than all other methods in terms of relative difference. In general, continuing methods all perform noticeably better than the only FTH method, `Episodic`. Interestingly, the masked version of `Average_Reward` now does worse than the unmasked version, which is what we expected to happen in the first place when applying masking for a continuing task. The `w_b` and `exact` column tell a similar story, and for each metric we have the same order from best to worse method.

Of course, mean values do not tell the whole story; because of this, we also represent the distribution of the relative difference for each method, using histograms in figure 7 and violin plots in figure 8. As we can see in figure 7, the distribution for all methods is mostly concentrated around zero, very strongly so in the case of `Continuing_UM_R`. The violin plots paint a similar picture, with a very bottom-heavy shape for `Continuing_UM_R` and with values more evenly distributed around the mean for other methods. Notice that `Average_Reward_M_R` has some of the worst performances while `Average_Reward_UM_R` has the smallest range of values, suggesting that masking can hurt the stability of average reward methods.

One last observation is that `Continuing_UM_R` is not only quite comfortably the best method in terms of performance, but is also by far the one with that converges the fastest. As we can see in the `it_maxed` column of Table 2, while every other method reached the iteration cap for every single seed, `Continuing_UM_R` achieved early convergence in all but 3 over 1000 seeds. The distribution of the number of iterations before convergence is represented with a histogram in figure 9.

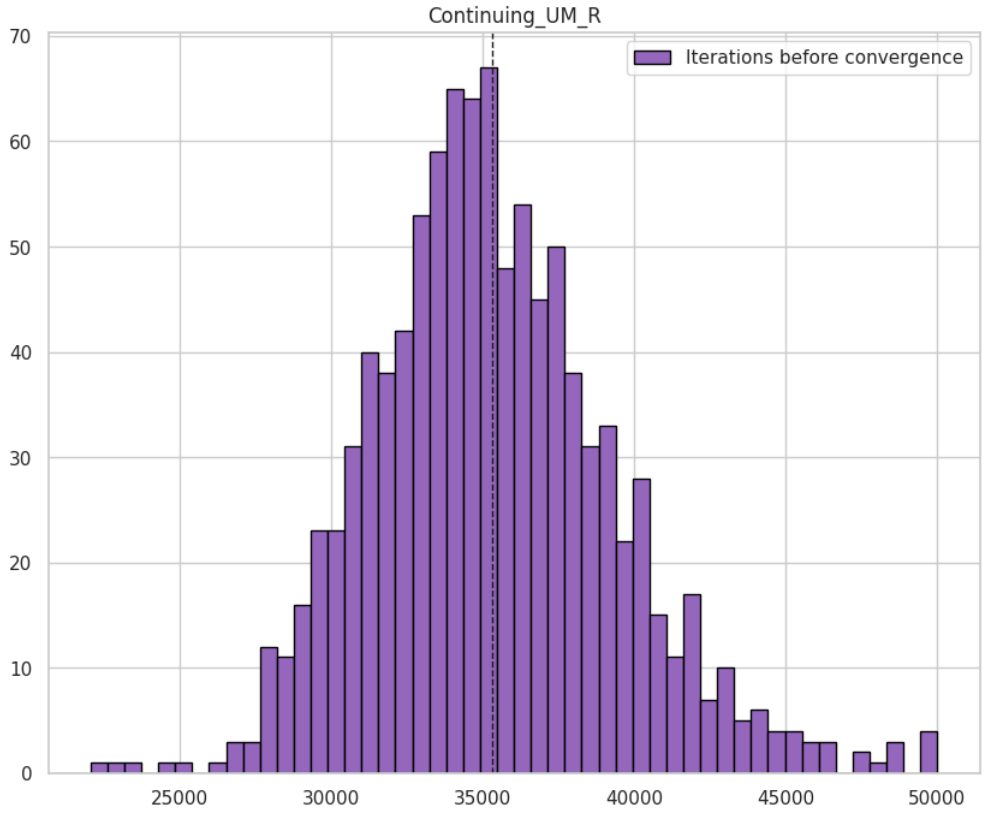


Figure 9: Histogram of the number of iterations before convergence for `Continuing_UM_R`.

The distribution has a mean of 35330 and a standard variation of 4032, and is mostly concentrated around the mean, with the majority of the seeds lying in the 30000 to 40000 range. This means that, with the optimal hyperparameters found with our grid search, we are almost guaranteed to converge before the iteration cap to a point whose value is within around 0.4% of the actual maximum.

Convergence for small θ

The remarkable performance and speed of `Continuing_UM_R` prompted some follow up experiments on its convergence. Namely, we asked ourselves if using a smaller convergence rate than the current $\theta =$ `1e-6` can achieve even better results while still converging before the iteration time. To do so we combined our grid search and multi-seed validation approach, testing 100 random seeds for each element of the following grid:

```
C_grid = [(theta, alpha, gamma, epsilon)
    for theta in [1e-9, 5e-9, 1e-8, 5e-8, 1e-7, 5e-7, 1e-6]
    for alpha in [0.0001, 0.0005, 0.001]
    for gamma in [0.7, 0.8, 0.9]
    for epsilon in [0.1, 0.2, 0.4]]
```

We chose seven small values of θ from `1e-9` to `1e-6`, and explored a neighborhood of the previous optimal weights `(0.0005, 0.8, 0.2)` for the other hyperparameters. To account for the reduction of θ we doubled the iteration cap to `max_it` $= 100000$.

In practice, doubling the iteration cap was not nearly enough to ensure convergence for the smaller values of theta, with 140 of the 189 hyperparameter combinations reaching the iteration cap for every single attempt. Reliable early convergence was achieved only for $\theta \geq$ `5e-7`, with performances comparable to the ones we observed in the previous experiments. A noticeable general trend is that, with the increased number of iterations, methods with $\gamma = 0.9$ seem to do better than the current optimal hyperparameters, with relative difference scores of about 0.2%, at the cost of needing roughly twice as many iterations to converge.

	r_d	w_b	exact	early	n_it
$\theta =$ `1e-9`	0.0008	0.73	43	0	100000.0
$\theta =$ `5e-9`	0.0008	0.73	43	0	100000.0
$\theta =$ `1e-8`	0.0008	0.73	43	0	100000.0
$\theta =$ `5e-8`	0.0008	0.73	43	0	100000.0
$\theta =$ `1e-7`	0.0008	0.73	43	0	100000.0
$\theta =$ `5e-7`	0.000809	0.74	41	68	84778.96

Table 3: Results for $\theta \leq$ `5e-7` with $\alpha = 0.0005, \gamma = 0.9, \varepsilon = 0.1$.

This preference for higher γ is also present in methods that do not converge early. In terms of absolute best performance the new optimal hyperparameters are $\alpha = 0.0005, \gamma = 0.9$ and $\varepsilon = 0.1$, which achieve a relative difference score of 0.08% for all values of $\theta \leq$ `1e-7`, as reported in Table 3. These hyperparameter combinations reach the exact maximum point in 43 out of 100 seeds with a `w_b` average of 0.73, at the cost of reaching the iteration cap in each instance. Interestingly, the next best result within our tests is reached by the same combination $(0.0005, 0.9, 0.1)$ for $\theta =$ `5e-7`, with a very slightly worse results but faster convergence. This tells us that `Continuing_UM_R` does in a way retain its speed advantage, and halving the parameter θ does improve its performance, at the cost of needing about two to three as many iterations to converge.

Behavior for increasing n

All experiments so far used the same string length value, $n = 55$. To conclude our analysis of our four best methods we want to take a look at their performance for different values of n. We started from the same idea of taking $n = \frac{k(k-1)}{2}$ as the string length needed to record graphs with k nodes. We let k range from 5 to 25, so that n ranges from 10 to 300, and tested the performance of our four methods on 100 random seeds for each n with the optimal hyperparameters from the grid search. The iteration cap was kept as `max_it` = 50000; we expect this to negatively affect runs for larger values of n, but not excessively so, since each iteration of our algorithms lasts n steps. Our results are summarized in figure 10, where we plotted the average relative difference on the random seeds for each n.

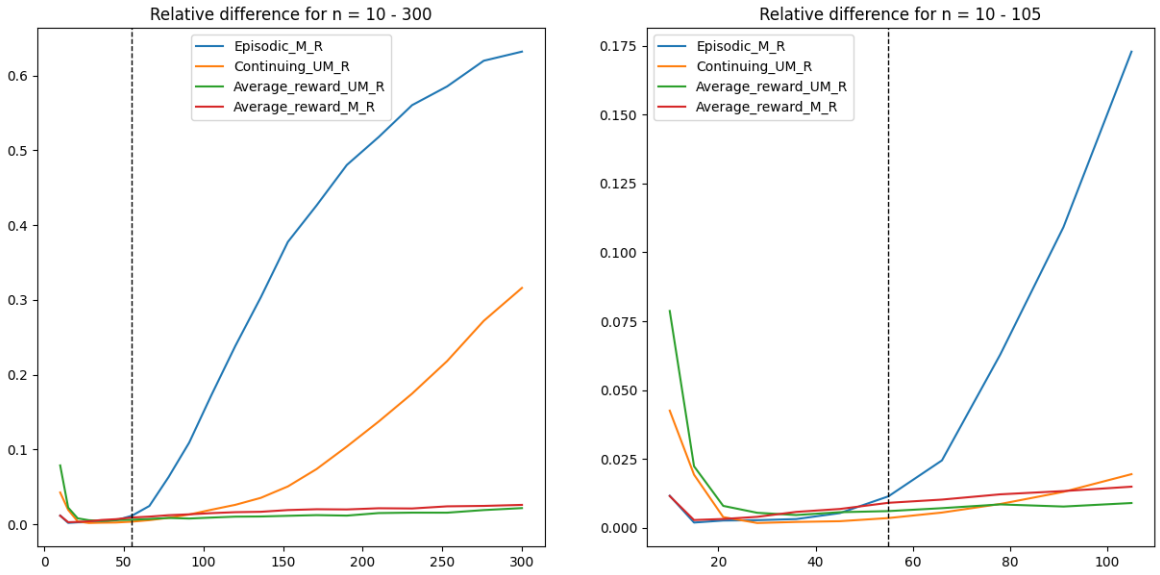


Figure 10: Average relative different for each method based on n. The dotted line represents our previous baseline $n = 55$; on the right, we zoom in on the values for $n \leq 105$.

As we can see in figure 10, the four methods start off with relatively similar performance, with spikes for the lowest values; these could by explained by the model not having enough weights to work with when n is low. The methods stay quite close to each other until around $n = 55$, after which the average relative difference score for `Episodic` starts to grow quite quickly, reaching over 60% for $n = 300$, meaning that the method is doing worse than guessing bits with coin flips. `Continuing_UM_R` has the best performance for lower values of n, as we can see in the graph on the right, but also progressively gets worse as n grows. The relative difference increase is not as steep as `Episodic` but still very significant, eventually reaching over 30%. Interestingly, the average convergence speed remains stable under the 40000 iterations threshold; further analysis could be done on this method by using lower values of θ and determining new hyperparameters better suited for large n.

Compared to the two classical discounted methods both average reward methods are remarkably stable, with the unmasked version performing consistently better than the masked version for all but the lowest values of n. For $n = 300$ the unmasked and masked versions reach a peak of, respectively, 2.16% and 2.59% relative difference, which is still quite respectable, especially compared to the less successful methods we discarded after the grid search. Again, even better results could be achieved by looking for better hyperparameters.

Experimental Conclusions

Based on our experiments, `Continuing_UM_R` is overall the best method for lower values of n, because of its great performance and the added advantage of faster convergence, but it quickly becomes unusable when n grows over 100. For higher values of n, `Average_Reward_UM_R` is consistently the best method, even before choosing ad-hoc hyperparameters. Notably, both methods were based on the looping, continuing MDP with episodic reward we introduced in Chapter 2, and produced better and more stable results than finite time horizon methods. This suggests that the same construction could be adapted to other finite time horizon tasks in order to apply algorithms for continuing methods, which we found to have better performance.

Bibliography

[1] Flora Angileri et al. *A Systematization of the Wagner Framework: Graph Theory Conjectures and Reinforcement Learning*. 2024. arXiv: 2406.12667 [cs.LG]. URL: https://arxiv.org/abs/2406.12667.

[2] Flora Angileri et al. "Analyzing RL components for Wagner's framework via Brouwer's conjecture". In: *Mach. Learn.* 114.11 (Oct. 2025). ISSN: 0885-6125. DOI: 10.1007/s10994-025-06890-2. URL: https://doi.org/10.1007/s10994-025-06890-2.

[3] Jens Flemming. *Notes on finite MDPs, Bellman equations and policy improvement 'a la Sutton/Barto*. 2024. URL: https://www2.htw-dresden.de/~fjeme691/flemming/blog/note_sutton_barto/note_sutton_barto.pdf.

[4] Francisco S. Melo, Sean P. Meyn, and M. Isabel Ribeiro. "An analysis of reinforcement learning with function approximation". In: *Proceedings of the 25th International Conference on Machine Learning*. ICML '08. Helsinki, Finland: Association for Computing Machinery, 2008, pp. 664–671. ISBN: 9781605582054. DOI: 10.1145/1390156.1390240. URL: https://doi.org/10.1145/1390156.1390240.

[5] Lina Palmborg. *Convergence of SARSA with linear function approximation: The random horizon case*. 2023. arXiv: 2306.04548 [cs.LG]. URL: https://arxiv.org/abs/2306.04548.

[6] Theodore Perkins and Doina Precup. "A Convergent Form of Approximate Policy Iteration". In: *Advances in Neural Information Processing Systems*. Ed. by S. Becker, S. Thrun, and K. Obermayer. Vol. 15. MIT Press, 2002. URL: https://proceedings.neurips.cc/paper_files/paper/2002/file/9f44e956e3a2b7b5598c625fcc802c36-Paper.pdf.

[7] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[8] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[9] C. Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis digital library of engineering and computer science. Morgan & Claypool, 2010. ISBN: 9781608454921. URL: https://books.google.it/books?id=qwtphfl7U74C.

[10] J.N. Tsitsiklis and B. Van Roy. "An analysis of temporal-difference learning with function approximation". In: *IEEE Transactions on Automatic Control* 42.5 (1997), pp. 674–690. DOI: 10.1109/9.580874.

[11] Adam Zsolt Wagner. *Constructions in combinatorics via neural networks*. 2021. arXiv: 2104.14516 [math.CO]. URL: https://arxiv.org/abs/2104.14516.